



DATABASE MODELING AND DATABASE SYSTEMS

DLBCSDMDS01

iu

INTERNATIONAL
UNIVERSITY OF
APPLIED SCIENCES

DATABASE MODELING AND DATABASE SYSTEMS

MASTHEAD

Publisher:
IU Internationale Hochschule GmbH
IU International University of Applied Sciences
Juri-Gagarin-Ring 152
D-99084 Erfurt

Mailing address:
Albert-Proeller-Straße 15-19
D-86675 Buchdorf
media@iu.org
www.iu.de

DLBCSDMDS01
Version No.: 001-v4-2026-0612

N. N.

© 2024 IU Internationale Hochschule GmbH
This course book is protected by copyright. All rights reserved.
This course book may not be reproduced and/or electronically edited, duplicated, or distributed in any kind of form without written permission by the IU Internationale Hochschule GmbH (hereinafter referred to as IU).
The authors/publishers have identified the authors and sources of all graphics to the best of their abilities. However, if any erroneous information has been provided, please notify us accordingly.

TABLE OF CONTENTS

DATABASE MODELING AND DATABASE SYSTEMS

Introduction

Signposts Throughout the Course Book	6
Basic Reading	7
Further Reading	8
Learning Objectives	9

Unit 1

Relational Database Basics	11
1.1 Basic Concepts of the Relational Data Model	12
1.2 Finding and Deleting Records in the Database	24
1.3 SQL and Relational Database Systems	29

Unit 2

Database Queries to Exactly One Table	33
2.1 Querying Data (SELECT)	36
2.2 Query Data with Condition (WHERE)	39
2.3 Sort Query Output (ORDER BY)	43
2.4 Queries with Group Formation (GROUP BY)	46
2.5 Subqueries with Nested SELECT Statements	48

Unit 3

Conception and Modeling of Relational Databases	53
3.1 The Entity Relationship Model	54
3.2 Relationships and Cardinalities in ER Models	56
3.3 Normal Forms of Databases	59

Unit 4

Creating Relational Databases	69
4.1 Logical Database Design Activities	70
4.2 Mapping of the Conceptual Data Model into the Physical Data Model	73
4.3 Creating Tables in SQL Databases from ER Diagrams	85

Unit 5	
Complex Database Queries on Multiple Tables	97
5.1 Composite Quantities (JOIN)	98
5.2 Set Operations	104
5.3 Data Views with CREATE VIEW	106
Unit 6	
Manipulating Records in Databases	109
6.1 Inserting New Data Records (INSERT)	110
6.2 Changing Existing Records	115
6.3 Transactions	119
Unit 7	
NoSQL Database Systems	129
7.1 Motivation and Basic Idea	130
7.2 Selected Groups of NoSQL Systems	132
Backmatter	
List of References	138
List of Tables and Figures	140

INTRODUCTION

WELCOME

SIGNPOSTS THROUGHOUT THE COURSE BOOK

This course book contains the core content for this course. Additional learning materials can be found on the learning platform, but this course book should form the basis for your learning.

The content of this course book is divided into units, which are divided further into sections. Each section contains only one new key concept to allow you to quickly and efficiently add new learning material to your existing knowledge.

At the end of each section of the digital course book, you will find self-check questions. These questions are designed to help you check whether you have understood the concepts in each section.

For all modules with a final exam, you must complete the knowledge tests on the learning platform. You will pass the knowledge test for each unit when you answer at least 80% of the questions correctly.

When you have passed the knowledge tests for all the units, the course is considered finished and you will be able to register for the final assessment. Please ensure that you complete the evaluation prior to registering for the assessment.

Good luck!

BASIC READING

46th VLDB (2020). *Proceedings of the International Conference on Very Large Data Bases (VLDB)*.

Date, C.J. (2019). *Database design and relational theory: Normal forms and all that jazz* (2nd ed.). Apress.

Documentation of Mondial Database (2010). *Mondial Database*.

Elmasri, R., Navathe, S. B. (2016). *Fundamentals of database systems*. Pearson Education Limited. Datenbank: ProQuest Ebook Central

Foster, E., Godbole, S. (2016). *Database systems. A pragmatic approach*. (2nd ed.). Apress.

Sumathi, S. et al (2010). *Fundamentals of relational database management systems*. Springer.

W3Schools (2020). SQL Tutorial.

FURTHER READING

UNIT 1

NA

UNIT 2

NA

UNIT 3

Faeskorn-Woyke, H. et al. (2007). Database systems: Theory and practice with SQL2003. *Oracle and MySQL*. Addison-Wesley-Publishing House, (pp. 79–111).Datenbank: Ebook Central

UNIT 4

ApacheOpenJPA (2014). *Mapping of inheritance in ApacheOpenJPA*.

Hibernate (2014). *Hibernate reference documentation*.

UNIT 5

NA

UNIT 6

NA

UNIT 7

Edlich, S. et al. (2011). *NoSQL. Entry into the world of non-relational web 2.0 databases*. Hanser Publishing House. (pp. 64–116). (HANSER eLibrary).

Kaufmann, M., Meier, A (2019). *SQL & NoSQL databases: Models, languages, consistency options and architectures for big data management*. Springer Vieweg. Datenbank: SpringerLink

NoSQL-Database.

Perdue, T (2011). *An Overview of NoSQL Databases*.

LEARNING OBJECTIVES

Stored data form the basis of many value chains found in a society that thrives on information and knowledge. A data schema provides structure through the methodical organizing of data thus enabling large amounts of information to be stored in such a way that it can be easily retrieved and edited.

The goal of this **Data Modeling and Database Systems** course is to teach you how to design, model, and create relational data models; how to use the SQL language to store and modify data in relational data models; and how to access data stored in relational databases. In addition, properties and concepts of NoSQL systems are presented.

UNIT 1

RELATIONAL DATABASE BASICS

STUDY GOALS

On completion of this unit, you will have learned ...

- what the most important relational database terms are and how they relate to each other.
- how data in relational databases is stored and read in a structured way.
- what SQL is and what typical relational database management systems exist.

1. RELATIONAL DATABASE BASICS

Introduction

This unit serves as a practical and descriptive introduction to the basics of relational databases. Important terms are introduced and illustrated through real-world examples.

1.1 Basic Concepts of the Relational Data Model

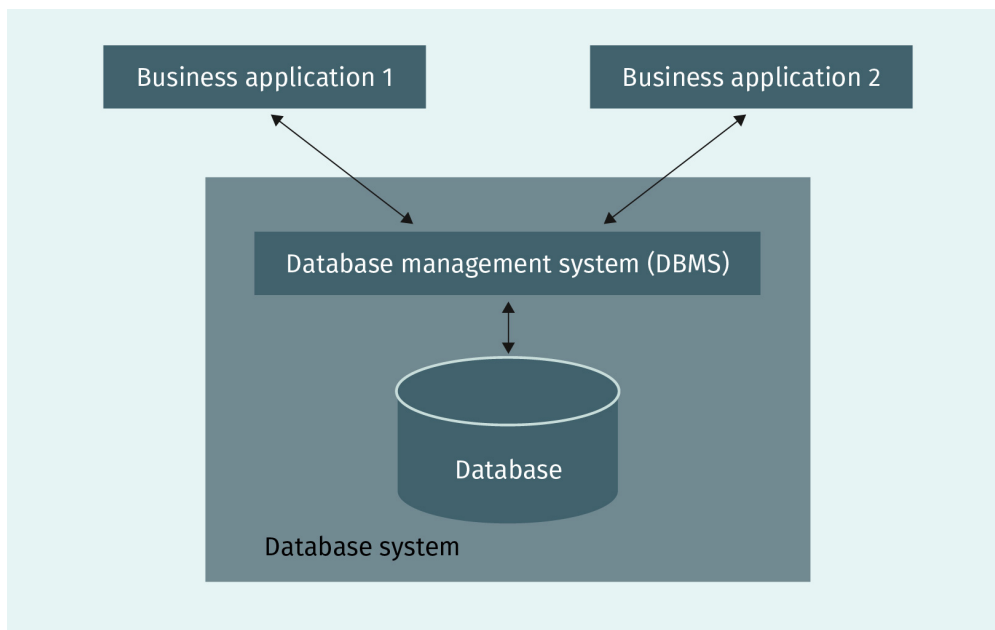
The data to be processed by industrial information systems are usually stored in databases, which are made available within database systems. Databases have a number of advantages over simple files that make them particularly relevant for large amounts of data. Specifically, databases

- provide specific and relatively easy-to-use functions for the evaluation of data, making even complex evaluations across different data sets possible with manageable effort.
- are optimized for efficiency when dealing with complex evaluations and large amounts of data – only the data that are actually required will be read and written, not the complete file.
- allow simultaneous access to the data by multiple users.
- support fine-grained access protection, i.e., the data users are allowed to access can be defined for the particular user.
- increase the reliability of systems in the sense that, in the event of a write error or crash, a defined, consistent state can normally be restored and it will be possible to trace which changes have already been implemented and which have not.

In addition to the standard database management systems (DBMSs) described here, a number of DBMSs have recently become widespread that take the form of NoSQL systems, which have a slightly different focus and de-emphasize data consistency in order to increase efficiency. However, our focus should first be on standard DBMSs.

Let us now introduce three closely related terms (see illustration):

Figure 1: Database, Database Management System, and Database System



Source: Jörg Dreikauss, 2022.

- **Databases** themselves contain data stored in an efficient format on a storage medium, e.g., a hard disk.
- **DBMSs** manage access to databases and provide corresponding functions. An application normally only accesses a database indirectly via the DBMS.
- **Database systems** are the combination of a DBMS and database.

Depending upon the author, sometimes applications are also counted within the database system. (Similarly, there is often no distinction between the database and the database system, and both are referred to as “the database”.) Users usually only work with the applications, not with the actual DBMS or the database.

The most common way to organize the storage of data is with the relational data model. Database management systems for the administration of relationally organized databases are called relational database management systems (RDBMSs).

Structure of Relational Databases

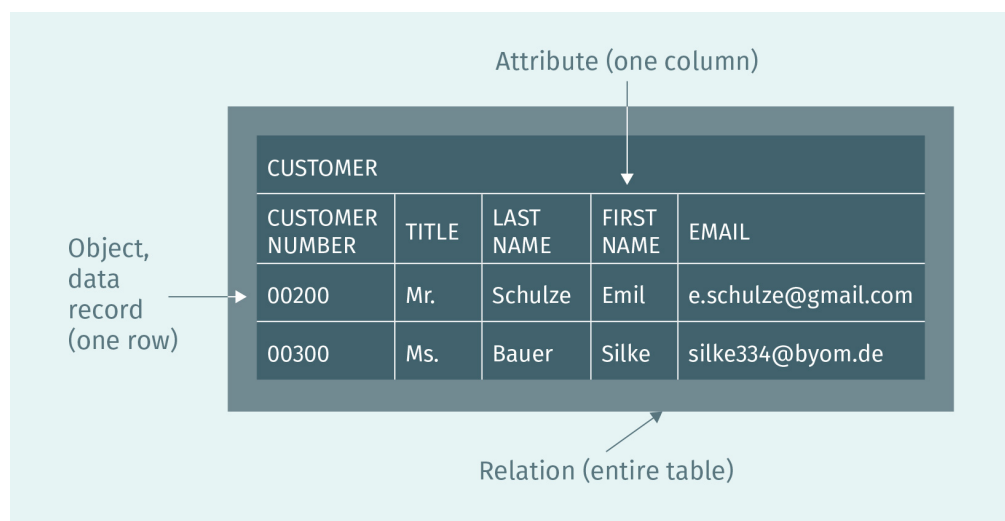
In order for the information stored in a database to be processed by information systems, the individual data records must be arranged in a predefined structure (Schubert, M., 2007). If you did not have a database, the most important data could be saved as text, such as in a Word document, as in the following example of a customer management system of an online shop:

“Mr. Emil Schulze has the email address e.schulze@gmail.com and is listed under customer number 00200.”

“Ms. Silke Bauer with the customer number 00300 can be reached by email at silke334@byom.de.”

However, customer data stored in this form cannot be automatically read and modified by IT systems. Therefore, the data records are reduced to the most important information and brought into a uniform structure. The most important characteristics of the customers just described are customer number, title, last name, first name, and email address. One way to structure these customer data records is to display them in a table whose columns each store a specific property of the data record and whose rows each contain exactly one data record.

Figure 2: Customer Relation Example



Source: Jörg Dreikauss, 2022.

All data records contained in the figure above have the same attributes and therefore the same structure. A data set is also called an object. A set of objects with the same structure – i.e., the same attributes – is stored in a relation. The CUSTOMER relation shown in the figure above has the structure (CUSTOMERNUMBER, TITLE, LASTNAME, FIRSTNAME, EMAIL). A relation, in turn, is stored in a table named after the relation.

Objects in relations must be able to be uniquely identified at all times, so two data sets in a relation may never have exactly the same values in all attributes. However, this required uniqueness is not always possible from the domain point of view. Therefore, in addition to the domain attributes, a technical attribute that can uniquely identify a data record is often stored in the relation. Such an attribute is called an identifier (ID attribute). In relational databases, identifiers are referred to as “primary keys.”

**TIP**

Always separate business IDs from technical IDs. Never use technical IDs as business IDs or business IDs as technical IDs! There are often unique identifiers, such as the customer number, as in the example above. Nevertheless, a database structure should always ensure the technical identifiability of the data records by means of its own ID.

Table 1 contains the CUSTOMER relation already introduced above, which has been extended to include the CUSTOMER ID attribute. This attribute serves as the primary key in this relation and is shown in dark gray in the table.

Table 1: Customer Data of an Online Shop

CUSTOMER					
CUSTOM-ERID	CUSTOM-ERNUM-BER	TITLE	LAST-NAME	FIRSTNAME	EMAIL
094904	00100	Mr.	Schwarz	David	david.schwarz@hkf-online.de
098483	00200	Mr.	Schulze	Emil	e.schulze@gmail.com
112536	00300	Ms.	Bauer	Silke	silke334@byom.de
234938	00400	Mrs.	Meier	Frauke	lieschen234@yahoo.com
293933	00500	Mr.	Ernst	Reiner	ernst-reiner@tkom.org
338457	00600	Mrs.	Weber	Maria	weberchen@trash-mail.com

Source: Jörg Dreikauss, 2022.

In almost all practical applications, a single relation (table) is not sufficient to store all the required information. In order to operate an online shop, it must be possible to store not only the most important customer data for each customer but also their registration address and various delivery and billing addresses. In this case, a relation ADDRESS with the attributes (ADDRESSID, STREET, HOUSENUMBER, POSTALCODE, CITY, COUNTRY) will be used, which is represented in the table below.

Table 2: Table with ADDRESS Relation

ADDRESS					
ADDRES-SID	STREET	HOUSE-NUMBER	POST-ALCODE	CITY	COUNTRY
2	Berliner Str.	55b	60311	Frankfurt	Germany

ADDRESS

ADDRES-SID	STREET	HOUSE-NUMBER	POST-ALCODE	CITY	COUNTRY
3	Marienstr.	25	10117	Berlin	Germany

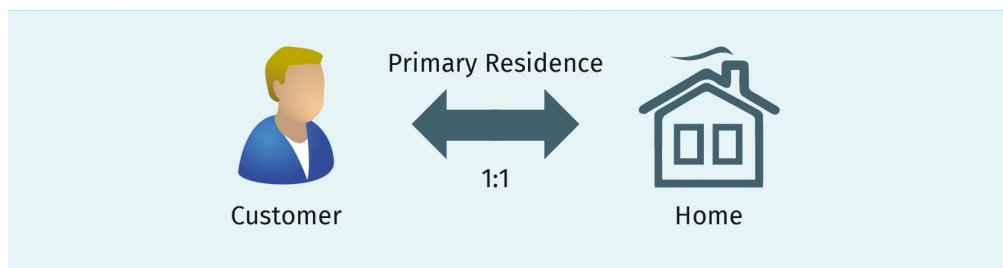
Source: Jörg Dreikauss, 2022.

With the two relations CUSTOMER and ADDRESS, it is now possible to store customer data and address data in a structured way. However, the classification of objects from the CUSTOMER relation to objects from the ADDRESS relation is still missing. This classification is made using relationships. Every relationship has a unique name. The following types of relationships are defined by their cardinality (or by arity or multiplicity): 1:1 relationship, 1:N relationship, and N:M relationship.

1:1 Relationships

The figure below illustrates the 1:1 relationship “primary residence” by way of example. This relationship connects a data set from the relation CUSTOMER with exactly one data set from the relation ADDRESS.

Figure 3: Illustration of the 1:1 Relationship Type



Source: Jörg Dreikauss, 2022.

The figure below shows the relation CUSTOMER extended by the attribute ADDRESSID. The value stored in the ADDRESSID attribute must uniquely identify an ADDRESS record. Therefore, the data model requires that only the existing identifier (primary key) of the relation ADDRESS be stored as the value of ADDRESSID. The identifier of ADDRESS is the attribute ADDRESSID. Therefore, the concrete values of ADDRESSID in the objects of type CUSTOMER must be values from the column ADDRESSID of the relation ADDRESS. The primary key of ADDRESS is thus also used in the “foreign” relation CUSTOMER. The primary keys of one table used in another table are called foreign keys; a foreign key references the primary key of another table related to the current table, thus linking them.

Figure 4: Table CUSTOMER with 1:1 Relationship to ADDRESS Relation

CUSTOMER						
CUSTOMERID	CUSTOMER-NUMBER	TITLE	LAST NAME	FIRST NAME	EMAIL	ADDRESSID
094904	00100	Mr.	Schwarz	David	david.schwarz@hfk-online.de	2
112536	00300	Ms.	Bauer	Silke	silke334@byom.de	3

Foreign key

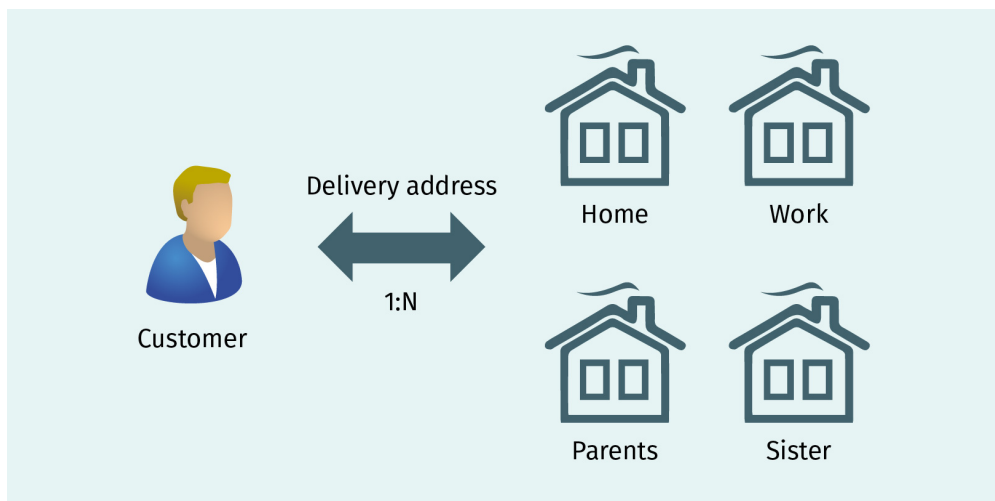
All values in this column must already occur in the table ADDRESS in the column ADDRESSID

Source: Jörg Dreikauss, 2022.

1:N Relationships

The figure below shows the 1:N relationship “delivery address”, with which exactly one object of the CUSTOMER relation is linked to several objects of the ADDRESS relation. In practice, you may be able to deliver to various locations, such as your office or the homes of your family members.

Figure 5: Illustration of the 1:N Relationship Type



Source: Jörg Dreikauss, 2022.

The figure below shows the relation ADDRESS that has been extended by the attribute CUSTOMERID. Values from the CUSTOMERID column of the CUSTOMER table are stored in this attribute. The attribute CUSTOMERID is the primary key of the relation CUSTOMER. This attribute can be used to uniquely identify the actual customers for the data records in the table ADDRESS.

Figure 6: ADDRESS table with 1:N Relationship to CUSTOMER Relation

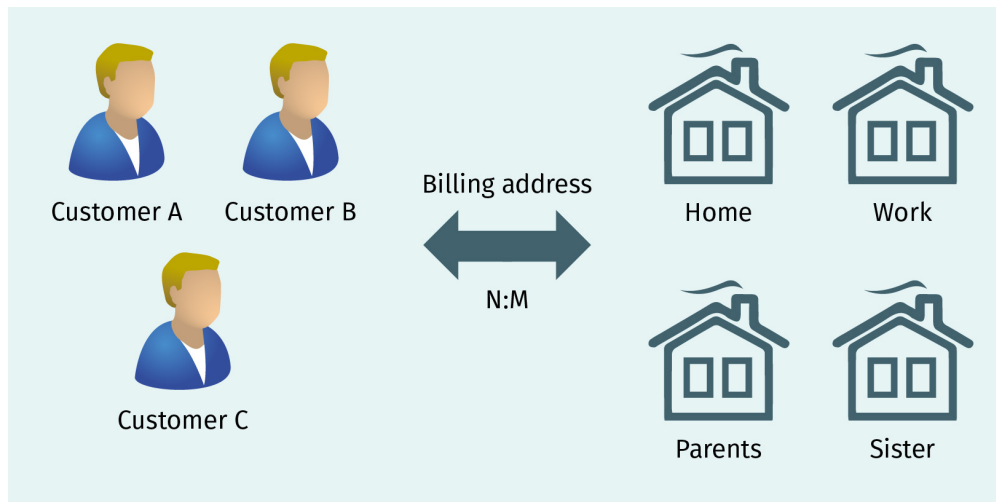
ADDRESS						
ADDRESSID	STREET	HOUSE-NUMBER	POSTAL-CODE	CITY	COUNTRY	CUSTOMERID
2	Berliner Str.	55b	60311	Frankfurt	Germany	094904
3	Marienstr.	25	10117	Berlin	Germany	094904
4	Alte Str.	49	53123	Bonn	Germany	112536
5	Salzburger Str.	27	93047	Regensburg	Germany	112536
6	Bahnhofstr.	5	69115	Heidelberg	Germany	112536
7	Domstr.	18	20095	Hamburg	Germany	348902
8	Stuttgarter Str.	73	01189	Dresden	Germany	348902

Source: Jörg Dreikauss, 2022.

N:M Relationships

The figure below illustrates the N:M relationship “billing address.” This relationship establishes a connection between M objects of the CUSTOMER relation and N objects of the ADDRESS relation. For example, a household may consist of distinct customers, such as parents or children. Or, a delivery address could be to an apartment block, where it is important to distinguish between customers living in different flats.

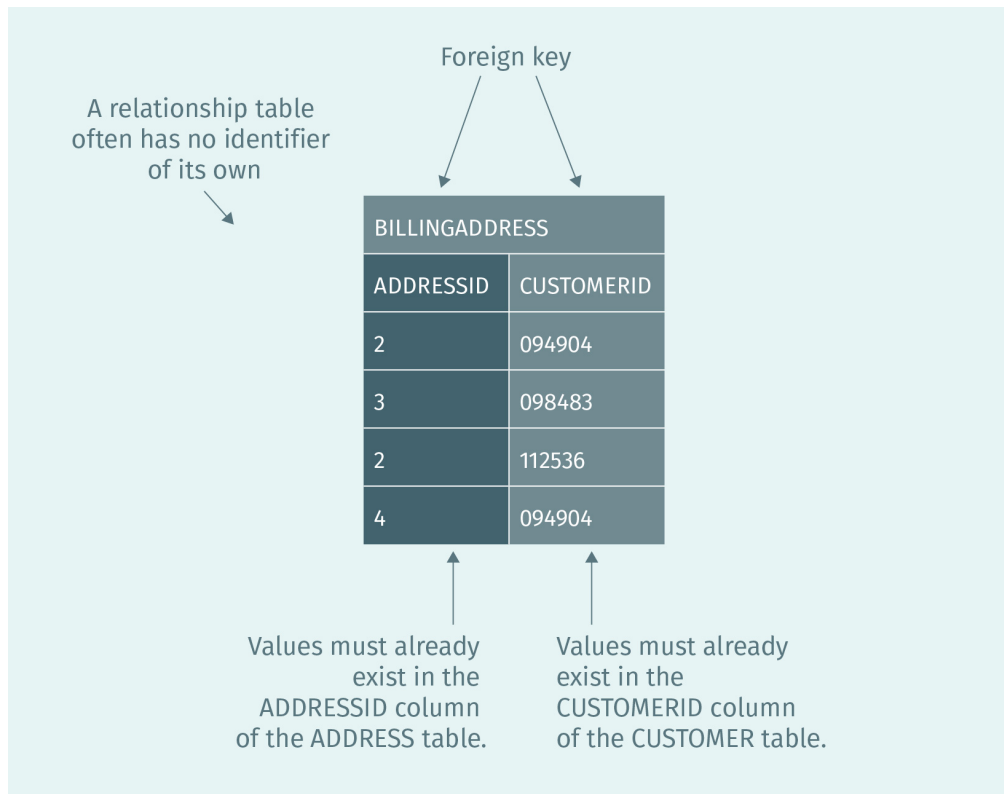
Figure 7: Illustration of the N:M Relationship Type



Source: Jörg Dreikauss, 2022.

Mapping N:M relationships in the relational data model requires “relationship tables.” The figure below shows an example of a relationship table that stores the N:M relationship between records of the CUSTOMER relation and records of the ADDRESS relation. A relationship table has as attributes the identifiers of the correlating relations. In the concrete example, this is ADDRESSID of the relation ADDRESS and CUSTOMERID of the relation CUSTOMER. Data records in this relationship table consist only of foreign keys.

Figure 8: Relationship Table for N:M Relationships



Source: Jörg Dreikauss, 2022.

The primary key of the BILLINGADDRESS table results from the combination of the ADDRESSID and the CUSTOMERID. This is a composite primary key.

Entering Data Records into Relational Databases

In general, data records are entered into relational databases according to the following scheme:

1. Select the relations (tables) in which the desired information is to be entered.
2. Insert the object (data record) with its corresponding attribute values.
3. Make sure that the primary keys are set uniquely and that no foreign keys are included that do not have corresponding primary keys.

Example

The following facts are to be stored in the relational data schema already presented above:

Mrs. Lisa Wagner, with the email address lisa-wagner@posteo.de, should be stored under the customer number 00700. Her primary residence is Domstr. 18 in Hamburg (20095) Germany. Almost all her shipments and invoices have been sent to her home so far. However, the last order, including the invoice, was shipped to the following address: Stuttgarter Str. 73, Dresden, Germany 01189.

To enter this data, three tables must be changed: CUSTOMER, ADDRESS, and BILLINGADDRESS. These tables are shown below. The new objects are added to the end of each table: one object in the CUSTOMER relation, two objects in the ADDRESS relation, and two objects in the BILLINGADDRESS relation. The values inserted in the example are marked in bold. In addition to the uniqueness of all primary keys, the following foreign key relationships must be taken into account:

- Save the primary residence in the attribute ADDRESSID of the CUSTOMER table.
- Save the delivery addresses in the attribute CUSTOMERID in the ADDRESS table.
- Save the billing addresses in the BILLINGADDRESS table with the respective foreign keys CUSTOMERID and ADDRESSID.

Table 3: CUSTOMER Relation Table

CUSTOMER						
CUSTOM-ERID	CUSTOM-ERNUM-BER	TITLE	LASTNAME	FIRST-NAME	EMAIL	ADDRESSID
094904	00100	Mr.	Schwarz	David	david.schwarz@hfk-online.de	2
112536	00300	Ms.	Bauer	Silke	silke334@byom.de	3
348902	00700	Mrs.	Wagner	Lisa	lisa-wagner@posteo.de	7

Source: Jörg Dreikauss, 2022.

Table 4: ADDRESS Table Including Sample Data

ADDRESS						
ADDRES-SID	STREET	HOUSE-NUMBER	POSTAL-CODE	CITY	COUNTRY	CUSTOM-ERID
2	Berliner Str.	55b	60311	Frankfurt	Germany	094904
3	Marienstr.	25	10117	Berlin	Germany	094904
4	Alte Str.	49	53123	Bonn	Germany	112536
5	Salzburger Str.	27	93047	Regensburg	Germany	112536

ADDRESS

ADDRESS-SID	STREET	HOUSE-NUMBER	POSTAL-CODE	CITY	COUNTRY	CUSTOMERID
6	Bahn-hofstr.	5	69115	Heidelberg	Germany	112536
7	Domstr.	18	20095	Hamburg	Germany	348902
8	Stuttgar-ter Str.	73	01189	Dresden	Germany	348902

Source: Jörg Dreikauss, 2022.

Table 5: BILLINGADDRESS Table Including Sample Data

BILLINGADDRESS	
ADDRESSID	CUSTOMERID
2	094904
3	098483
2	112536
4	094904
7	348902
8	348902

Source: Jörg Dreikauss, 2022.

Terms and Definitions

The following terms are central to dealing with relational databases and are frequently used in the broader field. Many of the following definitions are based on those from Steiner (2021), Elmasri and Navathe (2017), Unterstein and Mathiessen (2012), and Foster and Godbole (2016).

Relation

A relation R on the quantities $A_1, A_2, \dots, A_n$ is a subset of the Cartesian product $A_1 \times A_2 \times \dots \times A_n$, thus $R \subseteq A_1 \times A_2 \times \dots \times A_n$. The structure of Table 4 can be noted as the following relationship: $ADDRESS \subseteq ADDRESSID \times STREET \times HOUSENUMBER \times POSTAL-CODE \times CITY \times COUNTRY \times CUSTOMERID$, i.e., as a seven-tuple with the respective columns as components. A value range can be assigned to each component.

Table

A table is an illustration of a relation that contains the values that belong to the relation.

Data Record

All data in a table row are referred to as a data record. A data record is an element of the relation illustrated by the table.

Entity

An entity is something separate and distinct that can be uniquely identified among other similar entities, such as a business object. Each record stored in the database is an entity.

Attribute (or Property)

Concrete properties of entities are described as attributes. As in object orientation, each attribute has a name (or identifier) and a precisely-defined value range (or **domain**). Each column in a table stores the value for a specific attribute according to its name.

Domain

The designation for the value range of an attribute.



TIP

Those who know object orientation can identify entities for an ER (entity-relationship) model in the same way as classes for a data model and attributes of entities such as attributes to classes.

Entity set

Entities with the same properties are combined into entity sets. All records stored in a table form an entity set.

NULL value (or NULL)

If no value is stored in a database for an attribute in a table, this is indicated by the attribute value NULL. This means that no information has been saved for this attribute. Saving a NULL value usually requires as much memory space as a valid value from the value range of the attribute. Tables with many NULL values should therefore be avoided when designing the database.

Primary key

A key is an attribute or set of attributes of an entity that can be used to uniquely distinguish an entity from another similar entity. There must not be two entities with the same values in all attributes. If there were, they would be indistinguishable. A key that uniquely identifies an entity is called a primary key (or key attribute, identifier).

A primary key formed by exactly one attribute is a simple primary key. It is also possible to form a composite primary key where several attributes form the primary key. In this case, the combinations of all attributes involved in the primary key must be unique. A primary key must never be a NULL value.

Artificial key (Surrogate key)

An artificial key is an additional attribute added to a relation that is used as a primary key if there is no natural and unique primary key. This additional attribute has no meaning in the real world outside the database and is only used for the unique referencing of data records. An integer type is often used as an artificial key, the value of which is automatically determined by the database.

Foreign key

An attribute of a table is called a foreign key if it references a primary key of another table. In the data record of a database, it must be ensured that there is a corresponding value as a primary key in the referenced table for each value of a foreign key attribute. This condition (that there must be an actual primary key for each foreign key) is called **referential integrity**.

Referential Integrity
This is a property of a database whereby all foreign keys used are actually used as primary keys in another table and thus a valid reference to another data record exists.

Consistency

The data records stored in a database must be technically correct and consistent, and technical conditions such as referential integrity must be fulfilled. Maintaining data consistency and preventing inconsistencies is one of the central tasks of a database management system.

Database management system (DBMS)

A software system for operating databases is called a database management system (DBMS). Typical relational database management systems include MySQL, Oracle, Microsoft SQL Server, and IBM DB2.

1.2 Finding and Deleting Records in the Database

Finding Records in Relational Databases

Finding and returning data from a database is the most common operation performed on a dataset. The basic procedure is as follows:

1. Determine the relations (tables) in the database in which the searched information is stored. This includes all tables that may not directly contain the information you are looking for but are needed to identify the concrete data records. Relationship tables are a typical example of this.
2. Next, identify the concrete attributes in the relations selected in Step 1 that are to be returned as the result.
3. Restrict the output. Frequently, not all data records with the required attributes should be output, but only those data records whose attribute values meet predefined criteria. Therefore, the value set identified in Step 2 may have to be restricted in Step 3.
4. Sort the result set determined in Step 3 according to defined criteria.

As a result of a query or search operation to a relational database, a set of objects is returned that have only the requested attributes and whose values meet the restrictions specified in the query.

Example

Using the data stored in the following three tables, determine all locations and their postal codes to which the customer David Schwarz has already had shipments delivered.

Step 1: CUSTOMER and ADDRESS are the relevant relations, the DELIVERYADDRESS relationship between CUSTOMER and ADDRESS is stored in the attribute CUSTOMERID in the relation ADDRESS (see Table 6).

Step 2: Determine Mr. Schwarz's CUSTOMERID from DELIVERYADDRESS, determine all ADDRESSIDs whose CUSTOMERID value match Mr. Schwarz's CUSTOMERID (see Table 8).

Step 3: Determine all addresses using the identified ADDRESSIDs from Step 2 (see Table 7).

Step 4: Restrict the addresses found to the POSTALCODE and LOCATION attributes.

Result: (60311 Frankfurt), (53123 Bonn).

Obviously, for the formulation of search queries to a database, an exact knowledge of the data model is necessary. In particular, the technical meaning of foreign keys is often not directly recognizable from the data model, but can only be found in the documentation. For example, it is not directly apparent from the relations that the attribute ADDRESSID in CUSTOMER is the residential address. The table also does not clearly show the meaning of the foreign key CUSTOMERID in the relation ADDRESS.

Table 6: CUSTOMER Table

CUSTOMER						
CUS-TOMERID	CUS-TOMER	TITLE	LAST-NAME	FIRST-NAME	EMAIL	ADDRESS-ID
094904	00100	Mr.	Schwarz	David	david.schwarz@hkf-online.de	2
112536	00300	Ms.	Bauer	Silke	silke334@byom.de	3
348902	00700	Mrs.	Wagner	Lisa	lisa-wagner@posteo.de	7

Source: Jörg Dreikauss, 2022.

Table 7: ADDRESS Table

ADDRESS						
ADDRES-SID	STREET	HOUSE-NUMBER	POSTALCODE	CITY	COUNTRY	CUS-TOMERID
2	Berliner Str.	55b	60311	Frankfurt	Germany	094904
3	Marienstr.	25	10117	Berlin	Germany	094904
4	Alte Str.	49	53123	Bonn	Germany	112536
5	Salzburger Str.	27	93047	Regensburg	Germany	112536
6	Bahnhofstr.	5	69115	Heidelberg	Germany	112536
7	Domstr.	18	20095	Hamburg	Germany	348902
8	Stuttgarter Str.	73	01189	Dresden	Germany	348902

Source: Jörg Dreikauss, 2022.

Table 8: DELIVERYADDRESS Table

DELIVERYADDRESS	
ADDRESSID	CUSTOMERID
2	094904
3	098483
2	112536
4	094904
7	348902

DELIVERYADDRESS

ADDRESSID	CUSTOMERID
8	348902

Source: Jörg Dreikauss, 2022.

Deleting Records from Relational Databases

Unlike searching in a database (which does not change the information stored in it), deleting records is a critical operation. Especially in the case of data records that are distributed over several relations, it must be ensured that all other data records remain complete and stored consistently, even after deletion. The following steps can be identified for deleting data:

1. Select the relations (tables) in the database in which the objects to be deleted are stored. This also includes all tables that may not directly contain the information you are looking for but are needed to identify the concrete data records. Relationship tables are a typical example of this.
2. Identify the data records to be deleted based on the criteria formulated in the deletion request.
3. Ensure that there are no foreign keys that cannot be resolved after deleting the objects from the database (referential integrity).

Example

Refer to the data set shown in Table 9, Table 10, and the relationship tables (Tables 11, 12 and 13). Delete the customer Silke Bauer from the database, along with all other address data not required by other customer records.

Step 1: Identify the relations and relationships that are affected. CUSTOMER and ADDRESS as well as all relationship tables are relevant relations that must be taken into account when deleting the data for Silke Bauer.

Step 2: Determine the CUSTOMERID of Ms. Bauer, then determine and remove all records in the relationship tables whose value in CUSTOMERID matches the CUSTOMERID of Ms. Bauer. Remove the data record of the customer Ms. Bauer in the table CUSTOMER. Remove all records in ADDRESS that are no longer used in a relationship table. This applies to addresses 5 and 6 in Table 10, while addresses 2, 3, and 4 may not be deleted as they are also used by other customers.

Step 3: Make sure that after removing all data, all foreign keys in the relationship tables reference a valid record. In addition, there is the entry (3; 098483) in the table of invoice addresses, independent of the deletion of customer Silke Bauer, for which there is no suitable entry in the CUSTOMER table. This means that this data record must also be deleted.

The cells to be deleted in Tables 9–13 are marked in **bold**.

Table 9: CUSTOMER Table before Deletion

CUSTOMER					
CUSTOMERID	CUSTOM-ERNUMBER	TITLE	LASTNAME	FIRSTNAME	EMAIL
094904	00100	Mr.	Schwarz	David	david.schwarz@hkf-online.de
112536	00300	Ms.	Bauer	Silke	silke334@byom.de
348902	00700	Mrs.	Wagner	Lisa	lisa-wagner@posteo.de

Source: Jörg Dreikauss, 2022.

Table 10: ADDRESS Table before Deletion

ADDRESS					
ADDRESSID	STREET	HOUSENUMBER	POST-ALCODE	LOCATION	COUNTRY
2	Berliner Str.	55b	60311	Frankfurt	Germany
3	Marienstr.	25	10117	Berlin	Germany
4	Alte Str.	49	53123	Bonn	Germany
5	Salzburger Str.	27	93047	Regensburg	Germany
6	Bahnhofstr.	5	69115	Heidelberg	Germany
7	Domstr.	18	20095	Hamburg	Germany
8	Stuttgarter Str.	73	01189	Dresden	Germany

Source: Jörg Dreikauss, 2022.

Table 11: Relationship Table MAINRESIDENCE Before Deletion

MAINRESIDENCE	
ADDRESSID	CUSTOMERID
2	094904
3	112536
7	348902

Source: Jörg Dreikauss, 2022.

Table 12: Relationship Table DELIVERYADDRESS Before Deletion

DELIVERYADDRESS	
ADDRESSID	CUSTOMERID
2	094904
3	094904
4	112536
5	112536
6	112536
7	348902
8	348902

Source: Jörg Dreikauss, 2022.

Table 13: Relationship Table INVOICEADDRESS Before Deletion

INVOICEADDRESS	
ADDRESSID	CUSTOMERID
2	094904
3	098483
2	112536
4	094904
7	348902

Source: Jörg Dreikauss, 2022.

1.3 SQL and Relational Database Systems

In order to operate and use relational databases, a database management system (DBMS) must be available and there must also be an interface for querying and manipulating the data stored in the DBMS.

Structured Query Language

The Structured Query Language (SQL) is an internationally standardized programming language for data queries (ISO/IEC 9075). The SQL commands can each be assigned to one of the following sublanguages:

- **Data Manipulation Language (DML):** commands for reading and changing stored data
- **Data Definition Language (DDL):** commands for creating and changing the data schema, i.e., the logical structure of a database
- **Transaction Control Language (TCL):** commands for the controlled execution and monitoring of complex DML commands
- **Data Control Language (DCL):** commands for managing access rights

As such, the SQL language represents a universal interface for relational database systems. It can be used to create the database, access the data in a controlled manner, and control who has the right to access the database. The standardization of SQL has created a basis that should make it possible to develop and operate an application independently of a DBMS. No matter which DBMS is used, the SQL commands should not differ. Nevertheless, the concrete implementations of DBMSs by their manufacturers vary, so a simple exchange of DBMSs is not possible. Because the syntax specified by SQL is the same in almost all implementations and the SQL commands are also standardized, it is not surprising that the SQL syntax is the same in almost all implementations. However, there are differences in the sets of available data types for attributes and the sets of data types supported by the DBMS where functions are concerned. In addition, the execution speeds of individual DBMSs vary. Therefore, changing the database layer of an application is a complex task because every SQL command used must be tested in the new environment.

Typical Relational Database Management Systems

There are many different DBMSs available on the market. Therefore, only a small selection of the best-known will be mentioned.

MariaDB is a relational DBMS that is widely used for small and medium-sized applications. It was created by a spin-off from MySQL after MySQL separated from Oracle (MySQL, 2019). MariaDB is available as open-source software for many operating systems, meaning there are no license fees for its commercial use.

Typical representatives of commercial relational DBMSs are:

- **Oracle Database**
- **IBM DB2**
- **Microsoft SQL Server**



TIP

Install MariaDB on your device to understand and solve the self-check questions. This can be done easily, for example, as XAMPP in a package with a web server and the web programming language PHP. XAMPP also includes the easy-to-use DBMS admin program phpMyAdmin. In phpMyAdmin, you can easily create databases and edit them via SQL. Use this tool to follow the examples in this course book and solve the self-check questions. Alternatively, SQL trainers and simulators are available on the internet. These often require no additional

installation on your computer and can run completely in a browser, even on your smartphone or tablet. Please search the Course Feed for the current internet addresses of the SQL trainers and simulators, as these addresses often vary over time. A more stable reference is the W3Schools from the World Wide Web (W3) Consortium.



SUMMARY

Database management systems (DBMSs) are responsible for storing business data. Business applications access the data stored in the DBMS via an interface. The most common way to organize data storage is the relational data model, in which the data are structured in tables whose columns each store a specific property of the data record and whose rows each contain exactly one data record. Foreign keys are used to map relationships between individual tables in the database. With the standardized language SQL, there is indeed a universal interface for relational databases. However, the concrete implementations of DBMS vendors vary.

UNIT 2

DATABASE QUERIES TO EXACTLY ONE TABLE

STUDY GOALS

On completion of this unit, you will have learned ...

- how the SELECT statement for querying data is structured.
- how filter conditions are formulated with WHERE.
- how groupings are implemented with GROUP BY.
- how to formulate SELECT statements nested with subqueries.

2. DATABASE QUERIES TO EXACTLY ONE TABLE

Introduction

This lesson introduces formulations for reading database queries. Examples are used to illustrate how SQL commands for querying data in a table are structured and which elements are responsible for which tasks.

All examples in this lesson refer to the structure and records in Table 14. This table is available as a SQL file under the name Country.sql. This is an excerpt from the Mondial database that is widely used for practice exercises and can be found on MyCampus under Tutorial Documents.

Table 14: Extract from the MONDIAL Database

Country							
Name	Code	Capital	Province	Area	Population	Continent	Percentage
Australia	AUS	Canberra	Australia Capital Territory	7686850	18260863	Australia/Oceania	100
Bolivia	BOL	La Paz	Bolivia	1098580	7165257	America	100
Brazil	BR	Brasilia	Distrito Federal	8511960	162661214	America	100
Canada	CDN	Ottawa	Ontario	9976140	9976140	America	100
China	TJ	Beijing	Beijing (munic.)	9596960	1210004956	Asia	100
Egypt	ET	Cairo	El Qahira	1001450	63575107	Africa	90
Egypt	ET	Cairo	El Qahira	1001450	63575107	Asia	10
Fiji	FJI	Suva	Fiji	18270	782381	Australia/Oceania	100
France	F	Paris	Ile de France	547030	58317450	Europe	100
Germany	D	Berlin	Berlin	356910	83536115	Europe	100
India	IND	New Delhi	Delhi	3287590	952107694	Asia	100

Country

Name	Code	Capital	Province	Area	Population	Continent	Percentage
Indonesia	RI	Jakarta	Indonesia	1919440	206611600	Asia	80
Indonesia	RI	Jakarta	Indonesia	1919440	206611600	Australia/Oceania	20
Libya	LAR	Tripoli	Libya	1759540	5445436	Africa	100
Mali	RMM	Bamako	Mali	1240000	9653261	Africa	100
Mexico	MEX	Mexico City	Distrito Federal	1972550	95772462	America	100
Mongolia	MNG	Ulaanbaatar	Mongolia	1565000	2496617	Asia	100
New Caledonia	NCA	Noumea	New Caledonia	19060	227436	Australia/Oceania	100
New Zealand	NZ	Wellington	New Zealand	268680	3547983	Australia/Oceania	100
Niger	RN	Niamey	Niger	1267000	9113001	Africa	100
Norway	N	Oslo	Oslo	324220	4383807	Europe	100
Poland	PL	Warsaw	Warszawskie	312683	38642565	Europe	100
Russia	R	Moscow	Moskva	17075200	148178487	Europe	20
Russia	R	Moscow	Moskva	17075200	148178487	Asia	80
Samoa	WS	Apia	Samoa	2860	214384	Australia/Oceania	100
Saudi Arabia	SA	Riyadh	Saudi Arabia	1960580	19409058	Asia	100
South Africa	RSA	Pretoria	Gauteng	1219910	41743459	Africa	100
Sudan	SUD	Khartoum	al Khartoum	2505810	31547543	Africa	100
Sweden	S	Stockholm	Stockholm	449964	8900954	Europe	100
Turkey	TR	Ankara	Ankara	780580	62484478	Europe	5

Country

Name	Code	Capital	Province	Area	Population	Conti- nent	Percent- age
Turkey	TR	Ankara	Ankara	780580	62484478	Asia	95
United States	USA	Wash- ington DC	Distr. Colum- bia	9372610	266476278	America	100

Source: Mondial Database, n.d.

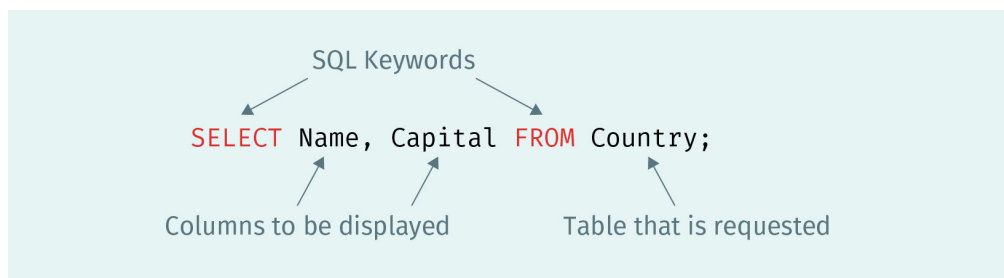
2.1 Querying Data (SELECT)

Querying data from a table in a database is a read-only operation, i.e., the data cannot be changed. The SQL command for a database query is SELECT. You can use SELECT to display the contents of one or more tables.

General Structure of SELECT

A SELECT command consists of the partially optional elements shown in Table 15. The example shown in the figure below contains a simple SQL command (or SQL statement) to query all countries and their capitals from the Country example database.

Figure 9: Example of a Simple SELECT Request



Source: Jörg Dreikauss, 2022.

Table 15: Result (Excerpt)

Name	Capital
Australia	Canberra
Bolivia	La Paz
Brazil	Brasilia
Canada	Ottawa

Name	Capital
...	...

Source: Jörg Dreikauss, 2022.

A valid **SELECT** statement to query data from a table consists of at least the elements **SELECT** and **FROM**. All other elements are optional. The end of a **SELECT** statement is marked by a semicolon (;). The result of **SELECT** is a virtual table that contains all of the records in the found set. Which columns are contained in the result table is defined in the **SELECT** statement. For example, you can use the result table for nested **SELECT** statements such as a database table, but it is not stored in the database.

Table 16: Elements of a SELECT Command

Elements of SELECT	Description
SELECT list of columns	This names the columns to be output; with *, all columns of the requested table are output. The order of the columns corresponds to the order of the columns in the requested table.
FROM table	This is the name of the table in which the search is performed; later, queries are also made to several tables. (There are also queries which do not operate on tables. In this case this part is not needed.)
optional: WHERE selection condition	These are the conditions that must be fulfilled for a data record to be taken; restricts the result set.
optional: GROUP BY list of columns	This summarizes elements of the result into groups for which specific restrictions or calculations can be made.
optional, if used together with GROUP BY : HAVING condition	These are conditions that must be fulfilled by elements of a group.
optional: ORDER BY list of columns	These are the criterion by which the result of this query is sorted. In case of more than one column, the columns are separated by a comma.

Source: Jörg Dreikauss, 2022.

The statement **SELECT** expects a list of columns to be displayed in the result. If you want to output all columns, use the character “*.” The following statement outputs the entire contents of the **Country** table:

Code

```
SELECT * FROM Country;
```

Table 17: Result (Excerpt)

Name	Code	Capital	Province	Area	Population	Continent	Percentage
Australia	AUS	Canberra	Australia Capital Territory	7686850	18260863	Australia/Oceania	100
Bolivia	BOL	La Paz	Bolivia	1098580	7165257	America	100
Brazil	BR	Brasilia	Distrito Federal	8511960	162661214	America	100
Canada	CDN	Ottawa	Ontario	9976140	9976140	America	100
...	...	...	...	...	...	...	...

Source: Jörg Dreikauss, 2022.

Elimination of Duplicates in the Result (DISTINCT)

It is often the case that data records occur several times in the result set of a database query. A typical example is the multiple occurrences of places or postal codes in a customer database. If only the number of locations from which the customers originate is relevant and not the number of customers from each location, duplicate occurrences will interfere with distilling this information. Identical data records in the result of an SQL query can be combined with `DISTINCT`. “Identical” here means that all attributes of a data set are identical. The following example displays all continents represented in the list of countries.

SQL-Statement

Code

```
SELECT DISTINCT Continent FROM Country;
```

Result

Continent
Australia/Oceania
America
Asia
Europe
Africa

Without `DISTINCT` the `Continent` attribute would be output for each data record in the table. With `DISTINCT`, multiple records containing the same continent are reduced to one data record.

2.2 Query Data with Condition (WHERE)

With a simple `SELECT` statement, the result set basically includes all data records of the requested table. Often, however, the result should be limited to a certain set of values. For example, if only those customers in the customer base who live in the city of Hamburg are of interest, the result of the database query should not contain any customers from Berlin. To restrict the result set, the keyword `WHERE` must be added to the `SELECT` statement. The following example queries all countries in the `Country` table but restricts the result to countries on the American continent:

SQL statement

Code

```
SELECT Name FROM Country WHERE Continent='America';
```

Result

Name
Bolivia
Brazil
Canada
Mexico
United States

As a selection condition of `WHERE`, the value of the `Continent` attribute is checked for equality with the character string `America`. Only records for which the selection condition is fulfilled are included in the result. If a restriction with `WHERE` is used in the `SELECT` statement, `WHERE` always follows the element `FROM`.

In the above example, a simple comparison for equal value was used as a selection condition. With SQL, however, several conditions can be linked together to create complex queries.

Logical Operators in the Selection Condition

One class of possible operators that can be used to formulate selection conditions are the logical operators `NOT`, `AND`, and `OR`. Without parentheses, these are evaluated in the order specified. The table below illustrates the use of logical operators in selection conditions.

Table 18: Examples of Logical Operators

Operator	Description	Example SELECT Name FROM Country WHERE ...
NOT	Negation of a condition; excludes entries with the value	NOT Continent = 'America'; (Names of all countries not located in America)
OR	Selects entries with either value	Continent='America' OR Continent='Africa' (Names of all countries located in America or Africa)
AND	Selects only entries with both values	Continent='Europe' AND Percentage=100 (Names of all countries whose base area is 100 percent in Europe)

Source: Jörg Dreikauss, 2022.

Logical operators can also be used in combination. However, the correct use of parenthesis must be ensured here. The following examples illustrate this connection:

SQL statement

Code

```
SELECT Name FROM Country WHERE Continent='Europe' AND NOT Percentage=100;
```

Result: Returns all countries in Europe whose base area is not 100 percent in Europe.

SQL statement

Code

```
SELECT Name FROM Country WHERE NOT(Continent='Europe'  
AND Percentage=100);
```

Result: All countries that are not 100% located in Europe.

SQL statement

Code

```
SELECT Name FROM Country WHERE NOT Continent='Europe' AND Percentage=100;
```

Result: All countries that are not located in Europe but with a base area on only one continent.

Comparison Operators in the Selection Condition

In addition to checking for equality with the operator "=", other comparison operators can also be used in SQL to formulate selection conditions. Typical examples of such conditions would be a request for all customers whose last order was placed more than three months

ago or a request for all customers whose highest invoice amount is greater than 1,000 euros. Similarities can also be found with comparison operators, such as a request for all customers whose surname begins with “S.”

Table 19: Comparison Operators

Operator	Description	Example SELECT Name FROM Country WHERE ...
=	checks for equality of values	Continent='America' or Percentage=100
<>, also: !=	checks for inequality	Continent != 'America' or Percentage <> 100
>, <	greater than, less than	Continent > 'America' or Percentage < 100
>=, <=	greater than or equal to, less than or equal to	Continent <= 'America' or Percentage >= 100
BETWEEN	restricts to certain value ranges	Population BETWEEN 6000000 AND 100000000
LIKE	restricts to certain patterns of attributes: _ stands for any single character % stands for any sequence of zero or more characters.	Code LIKE '_' (single digit value); Code LIKE 'C__' (Starts with C and consists of 3 characters); Name LIKE '%A' (Value ends with A); Name LIKE 'G%Y' (Value begins with G and ends with Y)
IS NULL	no value stored in database	Code IS NULL
IS NOT NULL	assurance that a value is stored in the database	Code IS NOT NULL
IN	restricts to values of a given value set	Continent IN ('America', 'Europe')

Source: Jörg Dreikauss, 2022.

SQL statement

Code

```
SELECT * FROM Country WHERE Continent='Africa' AND Name LIKE 'M%';
```

Result: Output of all countries that are located in Africa and whose name begins with the letter M.

SQL statement

Code

```
SELECT Name FROM Country WHERE Continent='Asia' AND Name  
BETWEEN 'C' AND 'I';
```

Result

Name
China
Egypt

Result: Output of all countries located in Asia whose name has an initial letter between C and I. China is included in the result quantity, but Indonesia and India are not.



TIP

The way BETWEEN operates differs depending on the DBMS. For example, in MySQL, the first parameter of BETWEEN is part of the result, but the second is not.

Selected Operators for Strings and Dates

The following operators for strings and dates are supported by most DBMSs. In addition, DBMSs usually support a large number of other functions, some of which are very DBMS-specific. Therefore, the developers of SQL queries have to inform themselves about which functions are actually available and how they are implemented based on the documentation of the DBMS used.

Date functions

Date functions are functions that output the current date or time and are used either to filter records or to write date and time information. The table below presents CURRENT_DATE and CURRENT_TIMESTAMP as examples of two date functions.

Table 20: Date Functions

Operator	Description	Example
CURRENT_DATE	returns the current calendar date, often in the form YYYY-MM-DD.	SELECT Name FROM Country WHERE (CURRENT_DATE - orderdate) >= 3; determines all orders placed three or more days ago
CURRENT_TIMESTAMP	returns a current timestamp, often in the form YYYY-MM-DD HH:MM:SS.	SELECT CURRENT_TIMESTAMP; determines and outputs time and date

Source: Jörg Dreikauss, 2022.

In addition, relational DBMSs usually provide conversion functions for date and time values. The concrete details must be read in the documentation of the DBMS used in each case.

String functions

SQL provides a number of functions for editing character strings. The table below shows examples of functions that link character strings.

Table 21: String Functions

Operator	Description	Example
CONCAT(arg1, ... argN)	combines the values passed as parameters to a character string	SELECT CONCAT(Name, ' ', Code) FROM Country;
CONCAT_WS(separator, arg1,...,argN) WS means “with separator”.	combines the values passed as parameters to a character string; always inserts the characters passed as the first parameter as a separator between two values	SELECT CONCAT_WS (' ', Name, Code, Area, ';') FROM Country;

Source: Jörg Dreikauss, 2022.

A concatenation of character strings is used, for example, to generate output texts (such as addresses) or to export data in structured text (such as XML) for technical interfaces.

2.3 Sort Query Output (ORDER BY)

Typically, the results of an SQL query are output in the order in which they are read from the database. However, SQL also offers the option of sorting the result set of a query according to defined criteria. The keyword `ORDER BY` is used for this purpose. On the one hand, presorted results are easier for the user to process – on the other hand, when using DBMS sorting functions, the implementation of a separate sorting function within the application can be dispensed with if necessary.

SQL statement

Code

```
SELECT Name FROM Country WHERE Continent='America'
ORDER BY Population DESC;
```

Result

Name
United States

Result

Name
Brazil
Mexico
Canada
Bolivia

The keyword `ORDER BY` initiates the sorting of the query result. In our example, the list of country names is sorted according to the number of inhabitants so that the country with the most inhabitants is named first, followed by all other countries in descending order.

As a selection condition of `WHERE`, the value of the `Continent` attribute is checked for equality with the character string `America`. Only records for which the selection condition is fulfilled are included in the result.

Sequence Indicators

When `ORDER BY` is used, the sort sequence is specified in addition to the column used for sorting. The table below shows the sequence indicators that can be used there.

Table 22: Sequence Indicator

Sequence indicator	Description
ASC	Ascending; A–Z or 0–9
DESC	Descending; Z–A and 9–0, respectively
-- (not specified)	Default value: Ascending

Source: Jörg Dreikauss, 2022.

Structure of `ORDER BY`

The `ORDER BY` keyword can be used as follows:

Variant 1

Sort by a column, ascending, or descending.

Scheme:

Code

```
ORDER BY column1 sequence;
```

Example:

Code

```
ORDER BY Population DESC;
```

Variant 2

Sort by several columns. The first column specified is the first criterion, the second column the second criterion, and so on. All criteria are sorted in the same order, i.e., ascending or descending.

Scheme:

Code

```
ORDER BY column1, column2, ..., columnN sequence;
```

Example:

Code

```
ORDER BY Continent, name ASC;
```

Variant 3

Sort by several columns. The first column specified is the first criterion, the second column the second criterion, and so on. For each column, the system explicitly specifies whether it is sorted in ascending or descending order.

Scheme:

Code

```
ORDER BY column1 sequence1, column2 sequence2, ..., columnN sequenceN;
```

Example:

Code

```
ORDER BY Continent ASC, Percentage DESC, Name ASC;
```

2.4 Queries with Group Formation (GROUP BY)

The `GROUP BY` keyword is used to group data. Groupings are usually used together with evaluation functions in SQL statements. In this way, the individual data records can be grouped in the dataset and then individually evaluated for each group. Thus, for example, it is possible to directly query the database about how many customers bought a certain product or how high the average turnover per postal code is.

The following example illustrates the use of `GROUP BY`. For each continent, the number of countries stored in the database is searched for.

SQL statement

Code

```
SELECT Continent, COUNT(*) FROM Country GROUP BY Continent;
```

Continent	COUNT(*)
Africa	5
America	5
Asia	8
Australia/Oceania	6
Europe	7

As many groups are formed for the relation `Country` as there are different values of `Continent`. So here, every record with `Asia` as a value for `Continent` is sorted into one group, every record with `Africa` into another group, and so on. For the example table, this grouping by continent resulted in five different groups. The aggregate function `COUNT(*)` is then used to count the number of data records in each group. The data records of the result set are collected from the grouping criterion (`Continent`) and the result of the aggregate function executed for each group (`COUNT(*)`).

Aggregate Functions

Aggregate functions are used to aggregate several values into one value – that is, to summarize or combine them. With these functions, calculations can be carried out very efficiently on the stored data records in the database. Table 23 explains the most important aggregate functions in SQL. These functions can also be used without `GROUP BY`, in which case they will refer to the entire table or to the result set restricted by `WHERE`.

Naming Columns in the Query Result with AS

The AS keyword can be used to rename the column names of the result table. This is particularly helpful when using aggregate functions. Examples are shown together with the aggregate functions in Table 23.

Table 23: Important Aggregate Functions in SQL

Operator	Description	Example
COUNT(*)	This counts the number of data records in a group. The example includes an operation together with GROUP BY and an operation without GROUP BY.	<pre>SELECT Continent, COUNT(*) AS NumberOfCountriesPerContinent FROM Country GROUP BY Continent; SELECT COUNT(*) AS 'number of countries in the world' FROM Country;</pre>
SUM(column)	This totals the values of a column within a group. In the example, the area of all countries of a continent.	<pre>SELECT Continent, SUM(Area) AS AreaofContinents FROM Country GROUP BY Continent;</pre>
AVG(column)	This calculates the average value of a column within a group.	<pre>SELECT Continent, AVG(Area) AS AverageAreaPerCountry FROM Country GROUP BY Continent;</pre>
MAX(column)	This calculates the maximum value of a column within a group; can be used for numbers, strings, and dates.	<pre>SELECT Continent, MAX(Area) AS SizeOfTheLargestCountry FROM Country GROUP BY Continent;</pre>
MIN(column)	This calculates the minimum value of a column within a group; can be used for numbers, strings, and dates.	<pre>SELECT Continent, MIN(Area) AS SizeOfTheSmallestCountry FROM Country GROUP BY Continent;</pre>

Source: Jörg Dreikauss, 2022.

Restriction of Groups with HAVING

With the keyword HAVING, it is possible to restrict the output of the **grouped** results. While WHERE refers to the selection of data records before grouping, HAVING restricts groupings that have already been created. If, for example, you want to determine all articles in an online shop that have been sold more than 500 times, the data records of your sales database are first grouped according to article number, then the number per group is determined, and finally only those groups whose number is greater than 500 are output. The following example illustrates the use of HAVING using the country relation.

We are looking for a list of continents with at least five countries that are completely located on one continent.

SQL statement

Code

```
SELECT Continent FROM Country
WHERE Percentage=100
GROUP BY Continent
HAVING COUNT(*)>=5;
```

Continent
Africa
America
Australia/Oceania
Europe

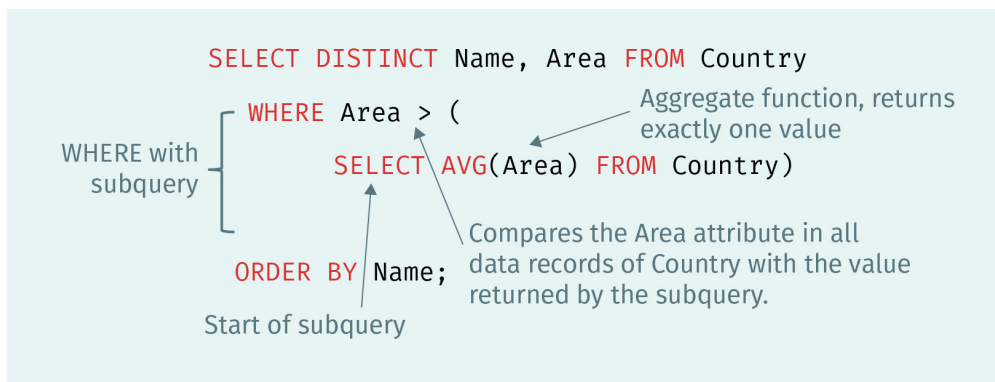
In the Country relation, all countries whose Percentage attribute value equals 100 are taken into account first. These records are grouped by the attribute values in the Continent field. The HAVING now refers to the grouping: only those groupings that contain more than five data records are output. The result table contains only the Continent column.

2.5 Subqueries with Nested SELECT Statements

In SQL, it is possible to formulate further SELECT statements within SELECT statements. These nested SELECT statements are called subqueries or subselects. They are often used to formulate filter conditions in the context of WHERE or HAVING.

The following example illustrates the use of subqueries using the Country table: a duplicate-free, sorted list is to be output with names and areas of all countries that are larger than the average area of all countries. The SQL statement used for this purpose is shown in the figure below.

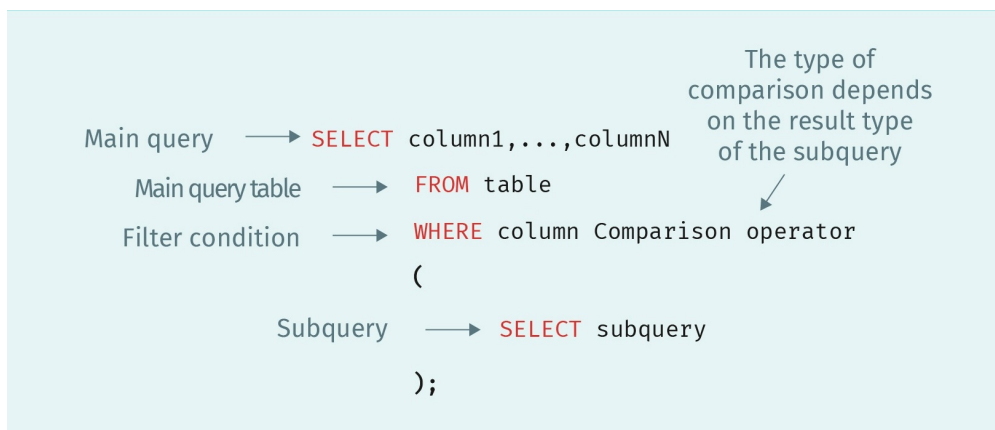
Figure 10: Example of Subqueries



Source: Jörg Dreikauss, 2022.

The subquery shown here is used to determine a dynamic comparison value from the current dataset. A general schema for subqueries in SELECT statements is shown in the figure below.

Figure 11: Schema of Subqueries SELECT



Source: Jörg Dreikauss, 2022.

Subqueries are used when the comparison criterion is dynamic and not known at the time the SQL query is created. In the above example, the comparison value was calculated on the basis of the current dataset of a table. Which comparison operators are possible is determined by the result type of the subquery. Subqueries with exactly one return value can be distinguished from subqueries with multiline returns.

Subqueries with Exactly One Return Value

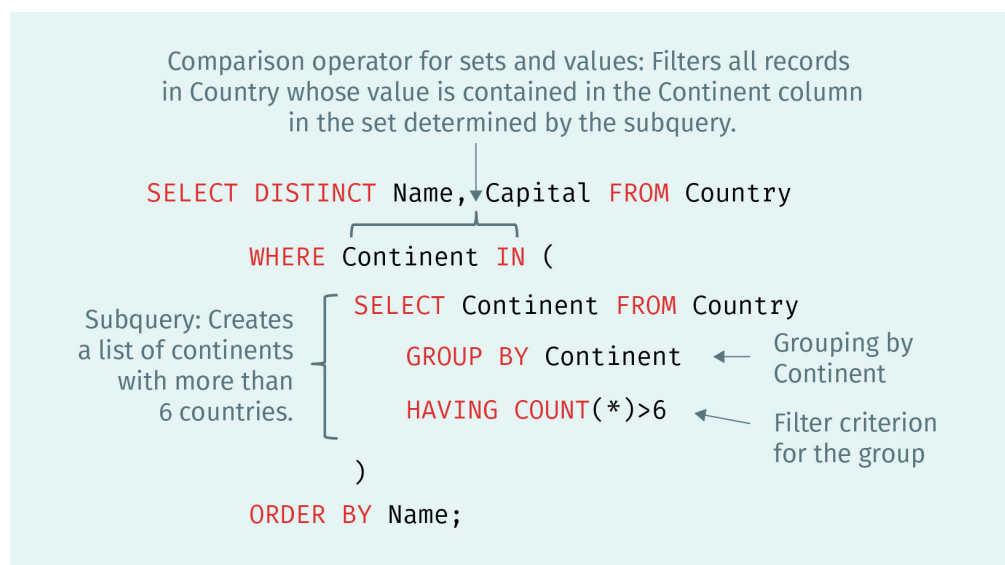
Although the result set of a SELECT statement is always a table, it can be required that this table contain exactly one value. In this case the result set of the SELECT statement of the subquery will contain exactly one column that contains exactly one data record. Only if these conditions are fulfilled can the already-known comparison operators (see section

2.2) be used. As shown in the above example, these conditions are fulfilled by the use of aggregate functions. In the event that no aggregate function is used, it must be ensured that the conditions for the result set are still fulfilled. Otherwise, an error message will be issued when the SELECT statement is executed.

Subqueries with Multiline Returns

An example of subqueries with multiline returns is shown in the figure below, which queries the names and capitals of the countries in the Country table that are located on a continent for which more than six countries are stored in Country.

Figure 12: Example of Subquery with Multiline Return



Source: Jörg Dreikauss, 2022.

The result table of the subquery contains two records:

Result
Continent
Asia
Europe

The comparison operator IN checks whether a particular value (specified to the left of IN) is contained in a set of values (specified to the right of IN). Further typical comparison operators for quantities are explained in Table 24.

Table 24: Typical Comparison Operators for Quantities

Operator	Description	Example
Column comparative operator ALL value set	The value from the named column is compared with all values of the value set by a comparison operator (e.g., <, >, >=, <=). ALL is evaluated to TRUE if the comparison for <i>all</i> records of the subquery evaluate to TRUE.	<pre>SELECT Population FROM Country WHERE (Population/Area) > ALL (SELECT Population/Area FROM Country WHERE Continent='Europe') ORDER BY Name;</pre> This returns the countries whose population density is greater than that of all countries in Europe. Note: This uses division as a mathematical function of SQL.
Column comparative operator ANY value set	The value of the named column is compared with all values of the value set by a comparison operator (e.g., <, >, >=, <=). ANY is evaluated to TRUE if the comparison is evaluated to TRUE for at least <i>one</i> subquery record.	<pre>SELECT Name, Area FROM Country WHERE Area >= ANY (SELECT SUM(Area) FROM Country GROUP BY Continent);</pre> This returns the countries whose area is greater than or equal to the total area of a continent.
EXISTS	Checks whether the result set contains at least one data record. Usually requires a link from the main query to the subquery in the WHERE part of the subquery.	<pre>SELECT Capital FROM Country a WHERE EXISTS (SELECT Province FROM Country b WHERE b.Province = a.Capital);</pre> This returns the names of capitals for whom the province is the same (e.g., Berlin, Oslo, and Stockholm).

Source: Jörg Dreikauss, 2022.



SUMMARY

The SQL command for a read database query that does not change the dataset is SELECT. You can use SELECT to display the contents of one or more tables. The result of SELECT is a table whose columns are exactly defined.

The WHERE keyword can be used to specify selection conditions that can be used to filter the data records of the result set before outputting them. With GROUP BY, the elements of the result set can be grouped and specific restrictions or calculations can be formulated for the groups.

The output is sorted with the keyword `ORDER BY`. If data records are to be filtered using a dynamic comparison criterion, `SELECT` statements within `SELECT` statements are used to create subqueries.

UNIT 3

CONCEPTION AND MODELING OF RELATIONAL DATABASES

STUDY GOALS

On completion of this unit, you will have learned ...

- which elements make up the entity relationship model and how they are graphically represented.
- which relationship types exist in the entity relationship model and how they are modeled.
- what normal forms are and which properties data schemas in the first to third normal forms must fulfill.

3. CONCEPTION AND MODELING OF RELATIONAL DATABASES

Introduction

Before a relational database can be created, a model of the data to be stored is created and then optimized for efficient use as a relational data schema. This lesson introduces the Entity Relationship Model – a standard data modeling structure – and teaches which relationship types to consider. Subsequently, the concept of the normal form is explained and then data schemas transferred into the third normal form will be demonstrated.

3.1 The Entity Relationship Model

The creation of the structure for a relational database is an iterative process in which a technical data model is created on the basis of a business data model. The physical data model determines exactly how the data are structured in a DBMS. The procedure is roughly comparable with the design of classes in an object-oriented system, except the goal is not the implementation of a system but the implementation of a database structure. As with system design, a graphical modeling language can also be used for the design and documentation of databases. Usually, an Entity Relationship Diagram (or ER Diagram) is used to display the all-important elements of a database structure.

Unlike the universal UML class diagram that can be used to model data structures and systems, the ER diagram is only used to model data schemas. In contrast to UML as the standard modeling language for IT systems, there are several variants of the ER diagram with differing notation elements.

Elements of an ER Model

The elements in an ER Model are entities, attributes, keys, and relationships. In practice, artificial keys (surrogate keys) are often used as primary keys. An artificial key is a purely technical attribute that carries no other information and has no equivalent in the world outside the DBMS. It is often a whole number and is automatically filled and managed by the DBMS.



TIP

Read more about this topic in Harkins (2011).

Relationship

Entities can be related to each other. A relationship is two-digit/two-figures if two entities are related. But relationships can also be multi-digit, i.e., they can relate more than two entities. A relationship itself can also have its own attributes.



TIP

Relationships in ER models are similar to relationships in UML class diagrams.

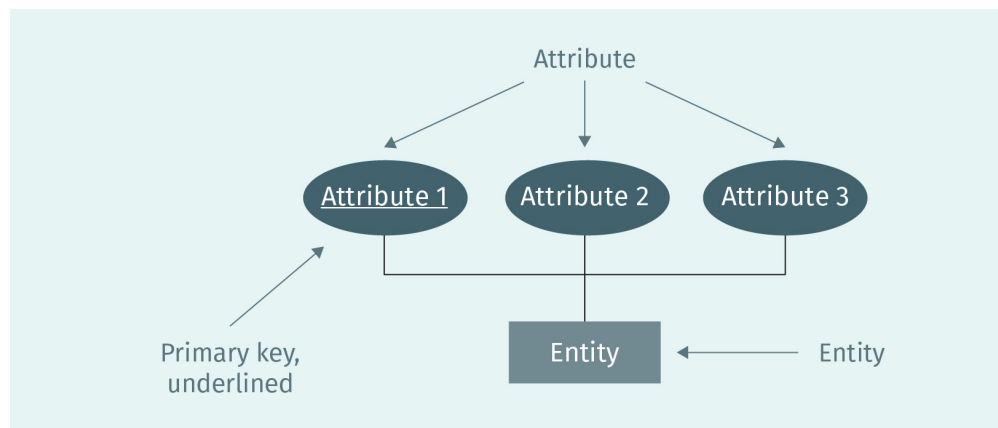
Graphical Notation of Entities and Attributes in ER Models

The most important elements in graphical ER diagrams are presented in the following three notation forms: Chen notation, Martin notation (or crow's foot notation), and as UML class diagrams.

Chen notation

The figure below shows entities and attributes in Chen notation. In this notation, entities are represented by a rectangle and attributes by an oval that are connected to the corresponding entity. Primary keys are identified by underlining the attribute name.

Figure 13: Conception and Modeling of Relational Databases

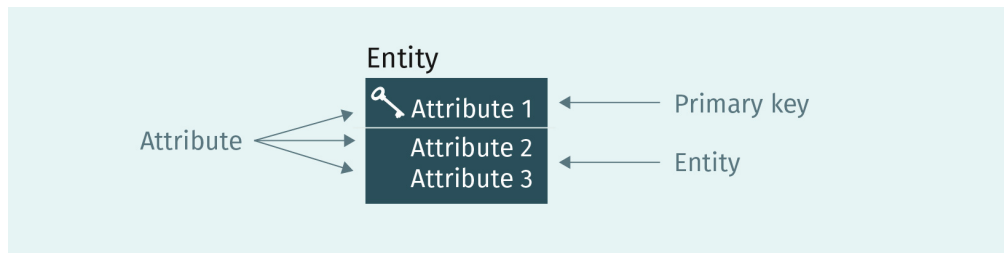


Source: Jörg Dreikauss, 2022.

Martin notation (crow's foot notation)

The figure below shows an entity with attributes in Martin notation. The name of the entity is noted over a rectangle. The primary key is noted in the upper part of the rectangle (the key-icon is optional), all other attributes are in the lower part.

Figure 14: Entities and Attributes in Martin Notation

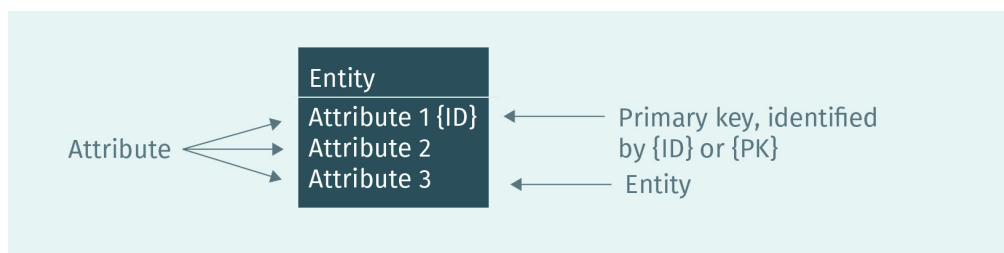


Source: Jörg Dreikauss, 2022.

UML class diagram

The UML class diagram is used to model the entity as a class with attributes, as shown in the figure below. Primary keys are noted as property values of the attribute in curly brackets after the name.

Figure 15: Entities and Attributes in UML



Source: Jörg Dreikauss, 2022.

In practice, the UML and the Martin notations are more compact because the modeling of the individual attributes does not take up much space and the affiliation of attributes to relations can be determined at a glance.

3.2 Relationships and Cardinalities in ER Models

Basically, three different types of relationships can be distinguished in the relational data model: 1:1 relationships, 1:N relationships, and N:M relationships. In the following, the notation "C" (or "CN" or "CM") as described in Steiner (2014) is used for a better differentiation of optional (or "conditional") elements of the relationships, even if this is not widespread.

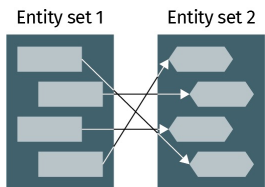
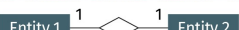
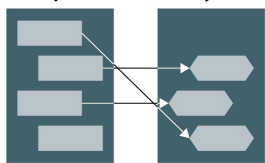
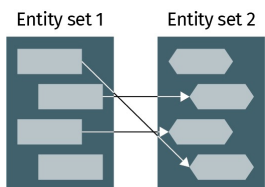
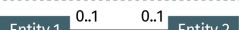
Types of 1:1 Relationships

In 1:1 relationships, exactly two entities are related to each other. The table below shows the possible variants of 1:1 relationships that result when the concept of **optionality** is added. A schematic representation is shown for each variant of the 1:1 relationship, the notation elements are given in the ER diagram, and an example of use is given. First, the table shows the classic 1:1 relationship. Each element of entity set 1 is assigned exactly one element of entity set 2. This is a bijective image. The 1:C and C:C relationships are variants of the 1:1 relationship that cannot be represented with Chen notation. Therefore, the 1:1 relationship in the Chen notation has a slightly different meaning than described here and stands for the relationship “0 or 1 to 0 or 1.”

Optionality

This is the possibility of an optional relationship which also allows for explicit omission.

Table 25: Types of 1:1 Relationships Including Optionality

	Schematic representation	Top: Martin notation Middle: UML notation Bottom: Chen notation	Application example
1:1		  	Exactly one article is assigned to a customer; each article belongs to exactly one customer.
1:C		 	Each customer is assigned no or exactly one article; each article belongs to exactly one customer.
C:C		 	Each customer is assigned no or exactly one article; each article is assigned no or exactly one customer.

Source: Jörg Dreikauss, 2022.

Types of 1:N Relationships

In 1:N relationships, one entity is related to several other entities. The table below compares the different 1:N relationships. The 1:CN, C:CN, and C:N relationships are variants of the 1:N relationship that cannot be represented by Chen notation. In Chen notation, the 1-N or N-M relationships are used instead, but they take on a slightly different meaning as a result. For example, 1-N stands for “zero or more” instead of “at least one”.

Table 26: Types of 1:N Relationships Including Optionality

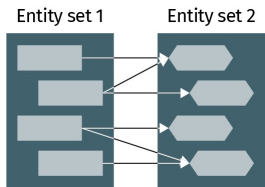
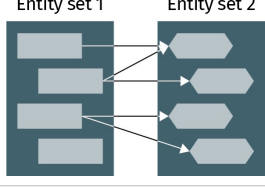
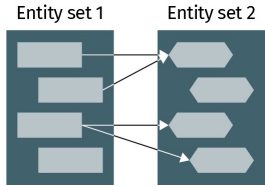
	Schematic representation	Top: Martin notation Middle: UML notation Bottom: Chen notation	Application example
1:N		 	Every customer belongs to one or more articles, and every article belongs to exactly one customer.
1:CN		 	A customer can belong to any number of articles, and each article belongs to exactly one customer.
C:CN		 	A customer can belong to any number of articles, and each article belongs to exactly one or to no customers.
C:N		 	Each customer belongs to at least one article, but each article does not have to belong to any customer.

Source: Jörg Dreikauss, 2022.

Types of N:M Relationships

In an N:M relationship, there are several entities involved on both sides of the relationship. The table below compares the different N:M relationships. The N:CM and CN:CM relationships are variants of the N:M relationship that cannot be represented by Chen notation.

Table 27: Types of N:M Relationships Including Optionality

	Schematic representation	Top: Martin notation Middle: UML notation Bottom: Chen notation	Application example
N:M		  	Each customer belongs to at least one article, and each article belongs to at least one customer.
N:CM		 	Each customer can belong to any number of articles, and each article belongs to at least one customer.
CN:CM		 	Each customer can belong to any number of articles, and each article can belong to any number of customers.

Source: Jörg Dreikauss, 2022.

3.3 Normal Forms of Databases

In order to avoid possible errors when working with stored data, the structure of tables in databases is evaluated and improved on the basis of objective criteria, in particular on whether the structure conforms to identified **normal forms**. Normalization optimizes the structure of a database and the normal form of a database is a measure of this structure. It provides information on how redundant data are stored in a database. Data contain **redundancy** if parts can be omitted without any loss of information (e.g., if a name as an attribute is stored in more than one table). If data are stored redundantly in a database, the risk of inconsistencies is high. When creating the structure of a database table, you should therefore attempt to keep redundancy as minimal as possible. The higher the normal form, the less redundant the data are. To determine the normal form, consider the structure of attributes of a relation and their interdependencies. Figure 15 shows the dependencies of the normal forms explained in more detail. In addition, the Boyce Codd normal form and the fourth normal form take further steps above the third normal form. For most practical projects the third normal form is sufficient. Often the third normal form won't be achieved because of reasons of efficiency.

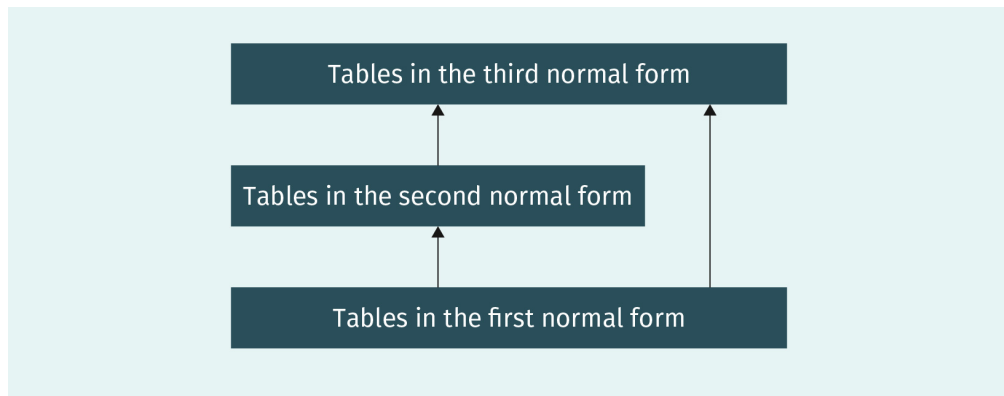
Normal form

A standard, fundamental form of a statement to which other statements can be reduced.

Redundancy

This is the property of a dataset that describes that the deletion of data does not necessarily result in a loss of information. For this, the information must be saved more than once.

Figure 16: Dependencies Between Normal Forms



Source: Steiner, 2014, p. 78.

Depending on requirements for data stock, a data schema will be transferred into the normal form that is needed by normalization. Using the example given in the table below, the first, second, and third normal forms of tables are explained below. The example in the table below shows which vendor supplied which customer with which article. There are three columns in the table that contain information about the vendor, customer, and article supplied.

Table 28: Table for Article Delivery Example

ArticleDelivery		
Vendor	Customer	Articles
MediaService, Hattstadt	Sophie Weber, 182932	Edge Of Tomorrow; EAN 838883
MovieGroup, Hamburg	Sophie Weber, 182932	Cry Baby; EAN 1222233
MovieGroup, Hamburg	Ralf Fischer, 488389	In the Morning; ISBU XX-NHN-223
FilmService, Berlin	Maria Bolz, 299376	Edge Of Tomorrow; EAN 838883

Source: Jörg Dreikauss, 2022.

First Normal Form

The following two conditions must both be fulfilled for a table to be in the first normal form:

1. All attributes must contain single values. Composite values are not allowed. Exactly one value must be stored for each attribute.
2. Each individual data record can be uniquely identified using a primary key.

In the table above, these criteria are not met because all attributes are composite values. The normalization step for the first normal form splits all occurring compound values, value enumerations, and nested values, and creates new columns for the sub-elements.

Key attributes are identified by which data records in the table can be uniquely distinguished from each other. Furthermore, no primary key has yet been defined. The table below shows our example table placed into the first normal form. The composite attributes supplier, customer, and article have been broken down into their individual components and defined as separate attributes. In addition, a composite primary key has been determined, namely the attributes “CustomerNumber” and “ArticleID.” In the following and all subsequent tables, attributes used as primary keys are set in boldface.

Table 29: First Normal Form with Compound Primary Key

ArticleDelivery						
Supplier-Name	SupplierCity	Customer Last-Name	Customer First-Name	<u>Customer Number</u>	ArticleName	<u>ArticleID</u>
MediaService	Hattstadt	Weber	Sophie	182932	Edge of Tomorrow	EAN 838883
Movie Group	Hamburg	Weber	Sophie	182932	Cry Baby	EAN 1222233
Movie Group	Hamburg	Fischer	Ralf	488389	In the Morning	ISBU XX-NHN-223
FilmService	Berlin	Bolz	Maria	299376	Edge of Tomorrow	EAN 838883

Source: Jörg Dreikauss, 2022.

The tables below show a possible alternative where an artificial key is selected as the primary key. The column ArticleDeliveryID has been added to the table for this purpose.

Table 30: First Normal Form with Transformed Table Including Simple Primary Key.

ArticleDelivery							
<u>Article-DeliveryID</u>	SupplierName	SupplierCity	CustomerLast-Name	CustomerFirst	CustomerNumber	Article-Name	ArticleID
L01	Media-Service	Hattstadt	Weber	Sophie	182932	Edge of Tomorrow	EAN 838883
L02	Movie Group	Hamburg	Weber	Sophie	182932	Cry Baby	EAN 1222233
L03	Movie Group	Hamburg	Fischer	Ralf	488389	In the Morning	ISBU XX-NHN-223

ArticleDelivery

<u>Article-DeliveryID</u>	SupplierName	SupplierCity	CustomerLastName	CustomerFirst	CustomerNumber	ArticleName	ArticleID
L04	FilmService	Berlin	Bolz	Maria	299376	Edge of Tomorrow	EAN 838883

Source: Jörg Dreikauss, 2022.

With the first normal form, it is possible to sort according to certain attributes and to clearly reference and change data records.

Second Normal Form

The second normal form is only relevant for tables with compound primary keys. In order to achieve the second normal form, all criteria for the first normal form must be met and, each non-key attribute is always dependent on the entire compound primary key and not on only a part of the key.

If the primary key consists of at least two attributes, all other attributes of the relation must always be dependent on the entire key. It is not sufficient for an attribute to be dependent on only part of a key.

Table 29, which is in the first normal form, shows the dependencies illustrated in the figure below:

- The attributes CustomerLastName and CustomerFirstName are dependent on the customer number, since the attributes CustomerLastName and CustomerFirstName follow from the attribute CustomerNumber. However, the customer number is only part of the primary key.
- The ArticleName attribute is dependent on the ArticleID, since the ArticleID attribute can be used to infer the value of ArticleName. However, ArticleID is only part of the primary key.

Figure 17: Dependencies in First Normal Form

ArticleDelivery						
Supplier Name	Supplier City	Customer LastName	Customer FirstName	Customer Number	ArticleName	ArticleID
Media Service	Hattstadt	Weber	Sophie	182932	Edge of Tomorrow	EAN 838883
Movie Group	Hamburg	Weber	Sophie	182932	Cry Baby	EAN 1222233
Movie Group	Hamburg	Fischer	Ralf	488389	In the Morning	ISBU XX-NHN-223
Film Service	Berlin	Bolz	Maria	299376	Edge of Tomorrow	EAN 838883

Elements of the Primary Key

The value of CustomerLastName and CustomerFirstName can be inferred from the CustomerNumber; ArticleID is not required.

The value of ArticleName can be inferred from the ArticleID; CustomerNumber is not required.

Source: Jörg Dreikauss, 2022.



TIP

The analysis of dependencies requires specialized knowledge. Normalization is usually not possible on the basis of the documented data schema alone.

The following three tables show the results of Table 29 transformed into the second normal form. The dependencies on parts of the primary key have been resolved by creating two new tables: Customer and Article.

Table 31: ArticleDelivery, Second Normal Form

ArticleDelivery			
SupplierName	SupplierCity	<u>CustomerNumber</u>	<u>ArticleID</u>
MediaService	Hattstadt	182932	EAN 838883

ArticleDelivery

SupplierName	SupplierCity	CustomerNumber	ArticleID
MovieGroup	Hamburg	182932	EAN 1222233
MovieGroup	Hamburg	488389	ISBU XX-NHN-223
FilmService	Berlin	299376	EAN 838883

Source: Jörg Dreikauss, 2022.

Table 32: Customer, Second Normal Form

Customer

CustomerLastName	CustomerFirstName	CustomerNumber
Weber	Sophie	182932
Fischer	Ralf	488389
Bolz	Maria	299376

Source: Jörg Dreikauss, 2022.

Table 33: Article, Second Normal Form

Article

ArticleName	ArticleID
Edge of Tomorrow	EAN 838883
Cry Baby	EAN 1222233
In the Morning	ISBU XX-NHN-233

Source: Jörg Dreikauss, 2022.

There is no second normal form for Table 28 shown above because a simple primary key was selected for this table.

By normalizing to the second normal form, each relation or table represents exactly one issue. Redundancy is eliminated by placing previously repeated information, such as article or customer, in separate tables and referencing these tables in the other tables. In the example, CustomerLastName and ArticleName are now stored exactly once as opposed to several times, as was the case in the first normal form.

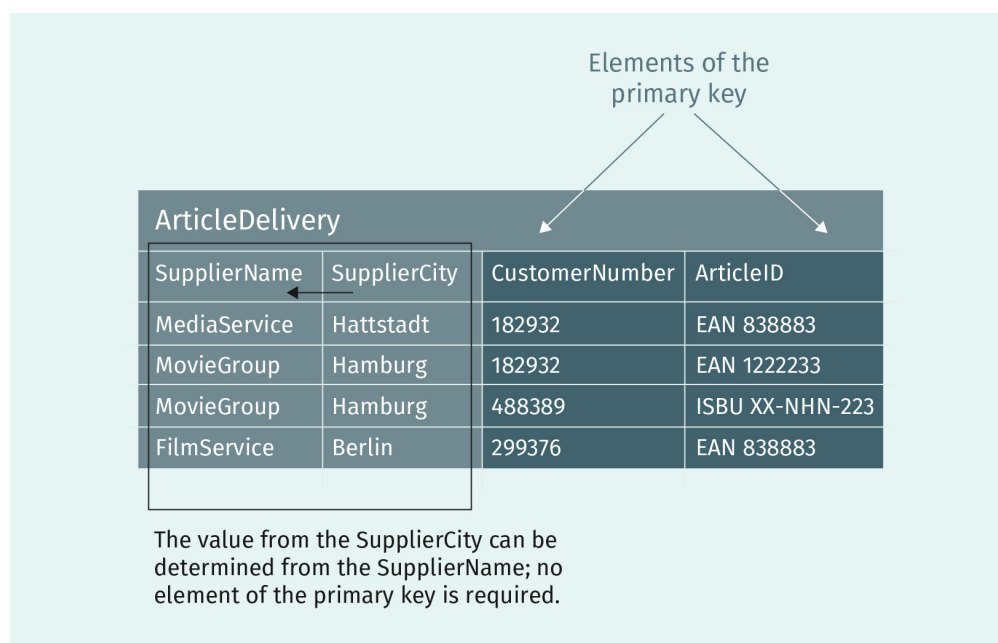
Third Normal Form

For a table to be in the third normal form, first the criteria of the first normal form must be fulfilled and in the case of tables with a composite primary key, the criteria of the second normal form. In addition, the following must apply: Any non-key attribute depends directly on the key attribute. There are no functional dependencies between attributes that are not part of the primary key.

With the second normal form, the remaining non-key attributes of a table are thematically linked to each other. However, it must be examined to see whether there are dependencies between the attributes that should be stored in a separate table.

The figure below shows the dependencies between the SupplierName and SupplierCity attributes from Table 31. These two attributes have dependencies that are not related to any part of the primary key. Therefore, these dependencies must first be resolved in order to reach the third normal form.

Figure 18: Dependencies in the Second Normal Form



Source: Jörg Dreikauss, 2022.

The third normal form of the original Table 26 is shown in Figure 18. A total of four tables have been created.

Figure 19: Third Normal Form with Compound Primary Key

ArticleDelivery			Supplier	
SupplierName	CustomerNumber	ArticleID	SupplierName	SupplierCity
MediaService	182932	EAN 838883	MediaService	Hattstadt
MovieGroup	182932	EAN 1222233	MovieGroup	Hamburg
MovieGroup	488389	ISBU XX-NHN-223	FilmService	Berlin
FilmService	299376	EAN 838883		

Article		Customer		
ArticleName	ArticleID	Customer LastName	Customer FirstName	CustomerNumber
Edge of Tomorrow	EAN 838883	Weber	Sophie	182932
Cry Baby	EAN 1222233	Fischer	Ralf	488389
In the Morning	ISBU XX-NHN-223	Bolz	Maria	299376

Source: Jörg Dreikauss, 2022.

The third normal form of the original Table 26 with a newly introduced simple primary key is shown in the figure below. Four tables were also created for this third normal form.

Figure 20: Third Normal Form with a Simple Primary Key

ArticleDelivery			
ArticleDeliveryID	SupplierName	CustomerNumber	ArticleID
L01	MediaService	182932	EAN 838883
L02	MovieGroup	182932	EAN 1222233
L03	MovieGroup	488389	ISBU XX-NHN-223
L04	FilmService	299376	EAN 838883

Supplier		Article	
SupplierName	SupplierCity	ArticleName	ArticleID
MediaService	Hattstadt	Edge of Tomorrow	EAN 838883
MovieGroup	Hamburg	Cry Baby	EAN 1222233
FilmService	Berlin	In the Morning	ISBU XX-NHN-223

Customer		
Customer LastName	Customer FirstName	CustomerNumber
Weber	Sophie	182932
Fischer	Ralf	488389
Bolz	Maria	299376

Source: Jörg Dreikauss, 2022.

In the third normal form, all the transitive dependencies of the data are immediately visible, since they are directly represented by the structure of the tables.



SUMMARY

The structuring of a relational database is an iterative process in which a technical data model is created on the basis of a business data model. As with system design, a graphical modeling language can be used for the design and documentation of databases. Usually, an Entity Relationship Diagram (or ER Diagram) is used to illustrate the elements Entity, Attribute, Key, and Relationship of a database structure. In principle, the three different relationship types (1:1 relationship, 1:N relationship, and N:M relationship) and their variants can be modeled in a relational data model.

Determining the normal form of a database provides a measure of the quality of the data schema. The normal form provides information on how redundant data are stored in a database. By using normalization to convert a data schema into a normal form, the redundancy of a database can be reduced.

UNIT 4

CREATING RELATIONAL DATABASES

STUDY GOALS

On completion of this unit, you will have learned ...

- what activities are performed to design relational databases.
- how the various relationship types from the ER model can be mapped in the tables of a database structure.
- how to define tables in a relational database using the SQL language.

4. CREATING RELATIONAL DATABASES

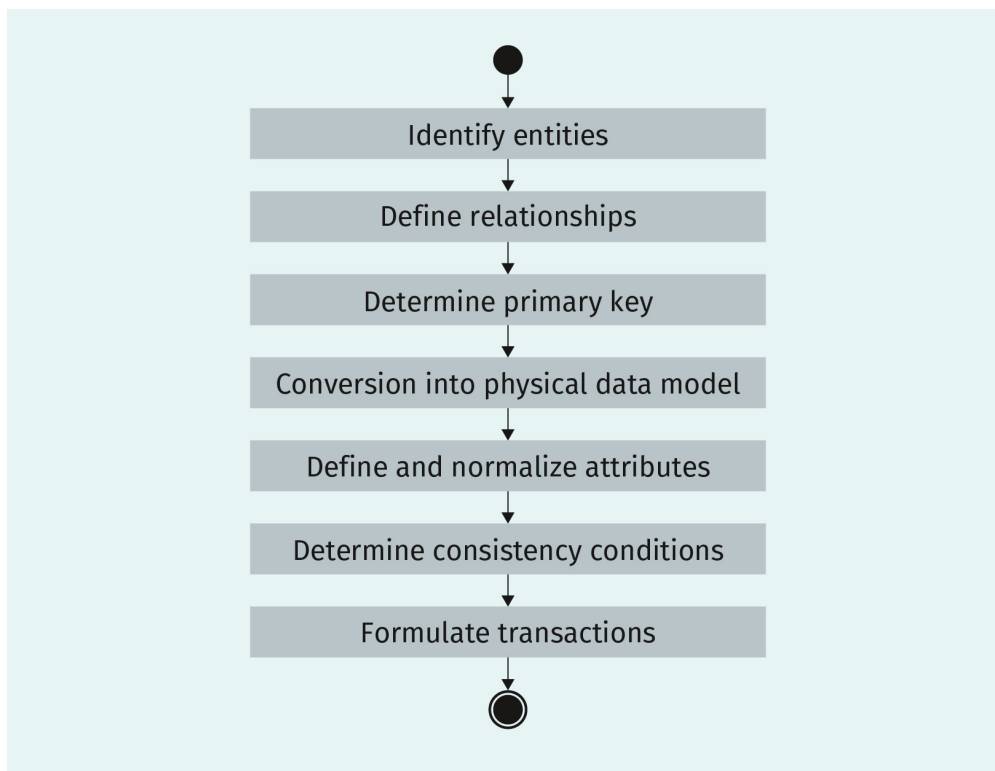
Introduction

After the conception and modeling of relational databases, the data model must be transferred into concrete tables of a database management system (DBMS). This lesson first introduces the logical database design activities required to create a physical database. It then explains how the various types of relationships are mapped into the table structure before introducing the Structured Query Language (SQL) elements that can be used to generate tables as data schemas from relational databases.

4.1 Logical Database Design Activities

The path from modeling to the concrete creation of a database involves various activities that are usually repeated for each database project. The figure below shows an overview of these activities in their typical order, based on Steiner (2014). The sequence of activities is not strictly sequential – in practice, it takes place with some iterations and returns to the starting point. Figure 20 therefore serves more as an overview and illustration of the context. The individual activities are described in more detail below.

Figure 21: Activities in the Design Process for Databases



Source: Steiner, 2014, p. 83.

Identify Entities

This step includes the identification of relations. (In the language of object orientation, one would say that classes are identified here.) The starting point is technical entities, which are refined step by step by technical details. This step has often already been done by requirements engineering and specification activities. In this case, the design of the database does not start with a blank page but rather with an already agreed upon technical data model.

Define Relationships

Once the relevant entities have been identified, their relationships must be determined. (The comparable step in object orientation is the modeling of relationships between classes.) Cardinalities are also used to specify quantities for the relationships (Databases Online Lexicon, 2012). The model resulting from the identification of the entities and the determination of the relationships is called the conceptual data model.

Determine Primary Key

The primary keys for the relations are determined first. Either the attributes that serve as simple or compound primary keys are identified or artificial keys are used. In the latter case, the model must be supplemented with the corresponding attributes.

Conversion into Physical Data Model

After a set of entities and their relationships have been identified, the transformation into a physical data model takes place. All relationships between the entities are transformed into a model supported by a relational DBMS. The concrete illustrations from a conceptual to a physical data model are described in detail in Section 4.2.

Defining and Normalizing Attributes

This identifies all attributes that are required to describe the entities. (The comparable step in object orientation is the identification of instance attributes.) Attributes are often already identified together with the entities. In this step, the concrete data type in the database must be determined in addition to the attributes. Next, the database is normalized, i.e., it is converted to the normal form sufficient for the requirements of the project. The attributes of relations are examined for structure and dependencies and, if necessary, further relations are added to the model. As a rule, the third normal form is sufficient for most practical projects.

Determine Consistency Conditions

In addition to the pure structure of the database in the form of its relations, specific consistency conditions may also have to be guaranteed by the DBMS. Explicitly formulated consistency conditions are used to ensure that the data are available in a technically “clean” form. Typical consistency considerations ensure the following:

- that all values in an ID attribute are truly unique
- compliance with the specified data type and value range of attributes
- the preservation of referential integrity and the possibility of securely resolving foreign keys

The conversion into a physical data model and the normalization of the database usually results in a complex data schema with several tables that are dependent on each other via foreign key relationships. For this reason, it must be ensured – particularly when changing and deleting data records – that exactly the same number of relations are changed or deleted at the same time so that all technical and functional consistency conditions are guaranteed.

Formulate Transactions

The term “transaction” refers to operations on the dataset of a database. In daily use, data must be read out, created, changed, and also deleted. Especially in industrial software systems, queries often consist of more than one SQL statement. For this reason, sets of SQL statements are combined into transactions. It is therefore necessary to determine in detail which SQL commands make up a query for reading, adding, changing, and deleting data so that all consistency conditions are met.

Defining transactions ensures that the dataset remains consistent in the event of an error during the execution of changes. This means that all changes already made to a transaction that has not been successfully completed can be reversed.

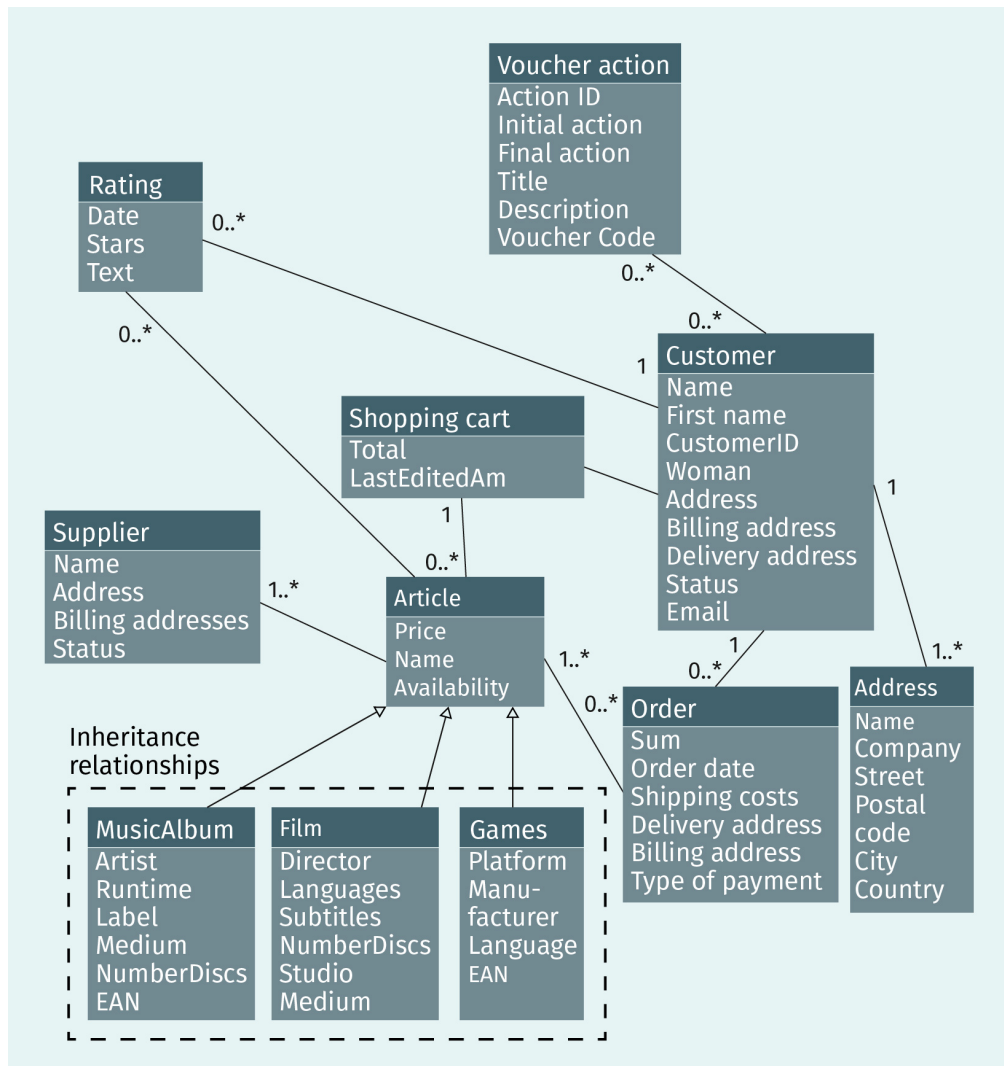
4.2 Mapping of the Conceptual Data Model into the Physical Data Model

Motivation and Terms

Conceptual data model

At the beginning of database design, all relevant entities along with their relationships and attributes are identified and documented. The result of these activities is a domain-oriented data model. The business data model is often started during requirements engineering activities. Ideally, as a result of the specifications, a detailed technical data model will be available. This data model is called a “conceptual data model” in the database environment. A conceptual data model does not yet contain any database-specific information. The figure below shows an example of a conceptual data model that contains an excerpt of a model for an online shop. A conceptual model is usually a network of entities (or “classes”) and, as shown in the example, also contains inheritance relationships.

Figure 22: Example of a Conceptual Data Model in the Form of a UML Class Diagram



Source: Jörg Dreikauss, 2022.

Physical data model

A physical data model is a data model that takes into account all of the constraints and properties of relational databases. Thus, only 1:C relationships and 1:CN relationships can be mapped directly in a relational database. Based on a physical data model, the concrete table structure of a relational database can then be created. In addition, the actual structure of a database is documented in a physical data model. Such a model contains tables, attributes, and primary keys, as well as relationships modeled using foreign keys. Also, any identifiers in the physical data model should not contain special characters such as umlauts.

Despite the limitations of relational databases, it is possible to map the structure of the conceptual data model in the tables of a DBMS. All relationships from the conceptual data model that cannot be mapped directly into a database are transformed into the physical data model beforehand. 1:1 relationships and 1:N relationships are mapped with the help of 1:C or 1:CN relationships and additional conditions. N:M relationships and inheritance relationships must first be mapped to 1:N relationships before the concrete database schema can be created.



TIP

More detailed information and other variants of the illustrations presented below can be found in Elmasri and Navathe (2016).

Mapping of 1:1 Relationships into the Physical Data Model

A selection of possible mappings of 1:1 relationships in the physical data model is shown in the table below.

Table 34: Illustration of 1:1 Relationships

	Conceptual data model	Physical data model	Description
1:1	Customer CustomerNR Name First name 1 1 Address Street Postal code City	Customer CustomerID {ID} CustomerNR LastName FirstName AddressID {FK} 1 1 Address AddressID {ID} Street PostalCode City	Customer and Address are each stored in a table; in Customer, the primary key of Address is stored in the attribute Address-sID. Condition: Each value Address.Address-sID must occur in Customer.AddressID exactly once. Customer Customer {ID} CustomerNR LastName FirstName AddressID {FK; Address->AddressID} Address AddressID {ID} Street Postal code City

	Conceptual data model	Physical data model	Description
		Customer CustomerID {ID} CustomerNR LastName FirstName 1 1 Address AddressID {ID} Street PostalCode City CustomerID {FK}	OR Customer and Address are each stored in a table; in Address, the primary key of the Customer is stored in the attribute CustomerID. Condition: Each value Customer.CustomerID must occur in Address.CustomerID exactly once. Customer CustomerID {ID} CustomerNR LastName FirstName Address AddressID {ID} Street PostalCode City CustomerID {FK, Customer->CustomerID}
		Customer Customer {ID} CustomerNR LastName FirstName Street PostalCode City	OR Summary in a table. This is only possible if the Address table has no other relationships to other tables. Condition: The combination of the Address attribute Street, PostalCode, and City must be unique.

Source: Jörg Dreikauss, 2022.

The figures of 1:C and C:C relationships in the physical data model are shown in the table below.

Table 35: Illustration of C:C Relationships

	Conceptual data model	Physical data model	Description
1:C			The Customer and Address are each stored in a table. The primary key of the Customer is stored in the attribute CustomerID as a foreign key in Address. Condition: The attribute CustomerID in Address must not be NULL.
C:C			Customer and Address are each stored in a table. There is also a CustomerAddress table with the two foreign keys AddressID and CustomerID as attributes. The CustomerAddress table is provided here with the artificial key CustomerAddressID as an example. Condition: In the CustomerAddress table, the values AddressID and CustomerID are not NULL, and in both columns, each value is contained at most once.

Source: Jörg Dreikauss, 2022.

Mapping of 1:N Relationships into the Physical Data Model

The mappings of relations of the form 1:N into the physical data model are shown in the table below.

Table 36: Illustration of 1:N relationships

	Conceptual data model	Physical data model	Description
1:N	Customer CustomerNR Name First name 1 1..* Rating Date Stars Text	Customer CustomerID {ID} CustomerNR LastName FirstName 1 1..* Rating RatingID {ID} Date Stars Text CustomerID {FK}	The customer and address are each stored in a table. The primary key of the customer is stored in the attribute CustomerID as a foreign key in Rating. Condition: The attribute CustomerID in Rating must not be NULL and must contain each value of CustomerID in Customer at least once (This is an additional condition). Remark: The additional condition could only be fulfilled by implemented application logic, otherwise, the relationship had to be implemented as 1:CN relationship. Customer CustomerID {ID} CustomerNR LastName FirstName Rating RatingID {ID} Date Stars Text CustomerID {FK, Customer->CustomerID}
1:CN	Customer CustomerNR Last name First name 1 0..* Rating Date Stars Text	Customer CustomerID {ID} CustomerNR LastName FirstName 1 0..* Rating RatingID {ID} Date Stars Text CustomerID {FK}	The customer and address are each stored in a table. The primary key of the customer is stored in the attribute CustomerID as a foreign key in Rating. Condition: The attribute CustomerID in Rating must not be NULL. Customer CustomerID {ID} CustomerNR LastName FirstName Rating RatingID {ID} Date Stars Text CustomerID {FK, Customer->CustomerID}

	Conceptual data model	Physical data model	Description
C:CN			The customer and address are each stored in a table. The primary key of the customer is stored in the attribute CustomerID as a foreign key in Rating. In this case, the attribute CustomerID in Rating can be NULL.
C:N			The customer and address are each stored in a table. The primary key of the customer is stored in the attribute CustomerID as a foreign key in Rating. In this case also, the attribute CustomerID in Rating can be NULL. Condition: Each value of CustomerID in customer must be available at least once as a value in CustomerID of Rating (This is an additional condition). Remark: The additional condition could only be fulfilled by implemented application logic, otherwise, the relationship had to be implemented as C:CN relationship.

Source: Jörg Dreikauss, 2022.

Mapping of N:M Relationships into the Physical Data Model

The representations of relationships of the form N:M in the physical data model are shown in the table below.

Table 37: Mapping of N:M Relationships

	Conceptual data model	Physical data model	Description				
N:M	Customer CustomerNR Last name First name 1..* 1..* Voucher campaign Initial campaign Final campaign Voucher code	Customer CustomerID {ID} CustomerNR LastName FirstName 1 1..* VoucherSolution CustomerID {ID, FK} ActionID {ID, FK} 0..1 1 VoucherAction ActionID {ID} StartAction EndAction VoucherCode	Customer and Voucher Action are each stored in a table. There is also a Voucher Solution table with the two foreign keys CustomerID and ActionID as attributes. Because the values in both CustomerID and ActionID can occur more than once, a primary key composed of both values can be used. It is also possible to use an artificial key for redeeming vouchers. Condition: Each value of CustomerID in the Customer table must occur at least once as a value of CustomerID in the VoucherSolution table. Each value of ActionID in the VoucherAction table must be contained in ActionID of the VoucherSolution table at least once. Customer CustomerID {ID} CustomerNR LastName FirstName VoucherAction ActionID {ID} StartAction EndAction VoucherCode VoucherSolution CustomerID {ID, FK, Customer->CustomerID} ActionID {ID, FK, VoucherAction->ActionID} <tr><td>N:CM</td><td> Customer CustomerNR Last name First name 1..* 0..* Voucher campaign Initial campaign Final campaign Voucher code </td><td> Customer CustomerID {ID} CustomerNR LastName FirstName 1 0..* VoucherSolution CustomerID {ID, FK} ActionID {ID, FK} 1..* 1 VoucherAction ActionID {ID} StartAction EndAction VoucherCode </td><td> The conversion into tables of a database takes place as with the N:M relationship. Condition: Each value of ActionID in the VoucherAction table must be contained in ActionID of the VoucherSolution table at least once. </td></tr>	N:CM	Customer CustomerNR Last name First name 1..* 0..* Voucher campaign Initial campaign Final campaign Voucher code	Customer CustomerID {ID} CustomerNR LastName FirstName 1 0..* VoucherSolution CustomerID {ID, FK} ActionID {ID, FK} 1..* 1 VoucherAction ActionID {ID} StartAction EndAction VoucherCode	The conversion into tables of a database takes place as with the N:M relationship. Condition: Each value of ActionID in the VoucherAction table must be contained in ActionID of the VoucherSolution table at least once.
N:CM	Customer CustomerNR Last name First name 1..* 0..* Voucher campaign Initial campaign Final campaign Voucher code	Customer CustomerID {ID} CustomerNR LastName FirstName 1 0..* VoucherSolution CustomerID {ID, FK} ActionID {ID, FK} 1..* 1 VoucherAction ActionID {ID} StartAction EndAction VoucherCode	The conversion into tables of a database takes place as with the N:M relationship. Condition: Each value of ActionID in the VoucherAction table must be contained in ActionID of the VoucherSolution table at least once.				

	Conceptual data model	Physical data model	Description
CN:CM			The conversion into tables of a database takes place as with the N:M relationship. Condition: However, there are no requirements for the single use of CustomerID or ActionID in VoucherSolutions.

Source: Jörg Dreikauss, 2022.

Mapping of Recursive Relationships Into the Physical Data Model

The illustrations of recursive relationships in the physical data model are shown in the table below.

Table 38: Mapping of Recursive Relationships

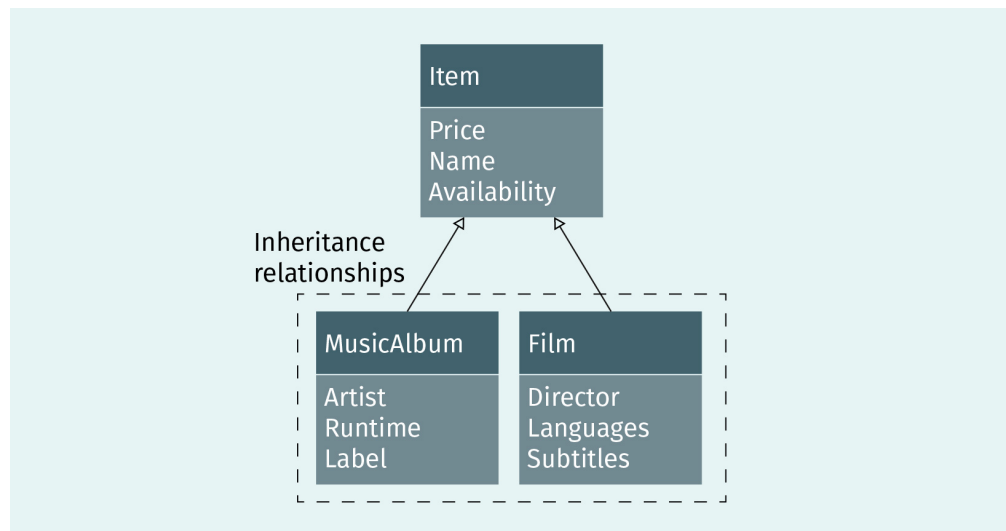
Conceptual data model	Physical data model	Description
		Articles are stored in a table. The recursion is modeled using a foreign key that points back to the Article table. Remark: The foreign key can be NULL.

Source: Jörg Dreikauss, 2022.

Mapping of Inheritance Relationships Into the Physical Data Model

Relational databases do not support inheritance relationships. Therefore, these types of relationships must also be mapped to tables and their relationships. Different strategies for the required illustrations are presented below using the simple example shown in the figure below.

Figure 23: Scenario for Inheritance Example



Source: Jörg Dreikauss, 2022.

In practice, the following mapping strategies for inheritance are widely used:

- single table
- joined subclass table
- table per class

Single table

The mapping strategy “Single Table” (also: single table per class hierarchy strategy) maps an entire class hierarchy to a table, as in the figure below. All objects for all classes in the inheritance hierarchy are stored in exactly one table.

Figure 24: Physical Data Model for Single-Table Strategy



Source: Jörg Dreikauss, 2022.

With the optional attribute “Discriminator,” the type (the name of the class) of each record can be saved explicitly. The advantages and disadvantages of Single Table are compared in the table below.

Table 39: Advantages and Disadvantages of the Single-Table Strategy

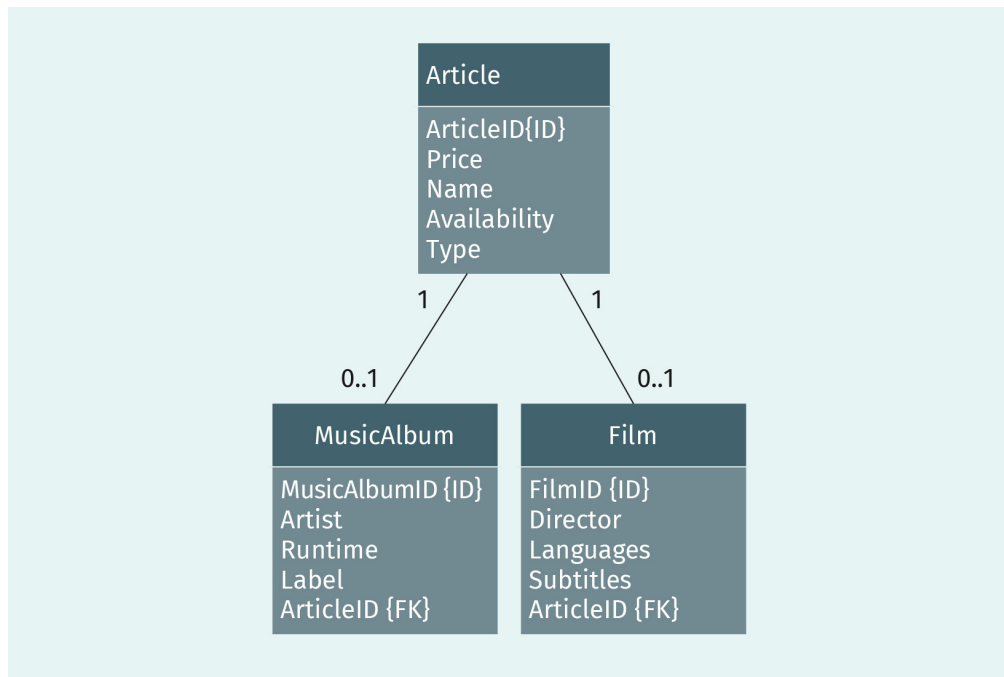
Advantages	Drawbacks
Very fast reading and writing access to data, since only one table has to be read.	The more complex the inheritance hierarchy, the larger a single table becomes. This can lead to significant performance losses with large amounts of data. In complex inheritance hierarchies, the table will contain a large number of NULL elements. The storage space reserved for these elements is therefore not used efficiently.
Exactly one SQL statement is sufficient for reading and writing to the database.	
Efficient access to relationships between classes in the inheritance hierarchy.	

Source: Jörg Dreikauss, 2022.

Joined subclass table

In the joined subclass table strategy, each class in the inheritance hierarchy is stored in its own table, as shown in the figure below. This also applies to abstract classes – classes that may not have instances themselves, but only their subclasses. Each subclass contains a foreign key to the data records of the attributes of its superclass. To read or write all attributes of a class from the database, the data must also be read or written from all tables of all superclasses.

Figure 25: Example for Joined Subclass Table Strategy



Source: Jörg Dreikauss, 2022.

The advantages and disadvantages of the joined subclass table strategy are compared in the table below.

Table 40: Advantages and Disadvantages of the Joined Subclass Table Strategy

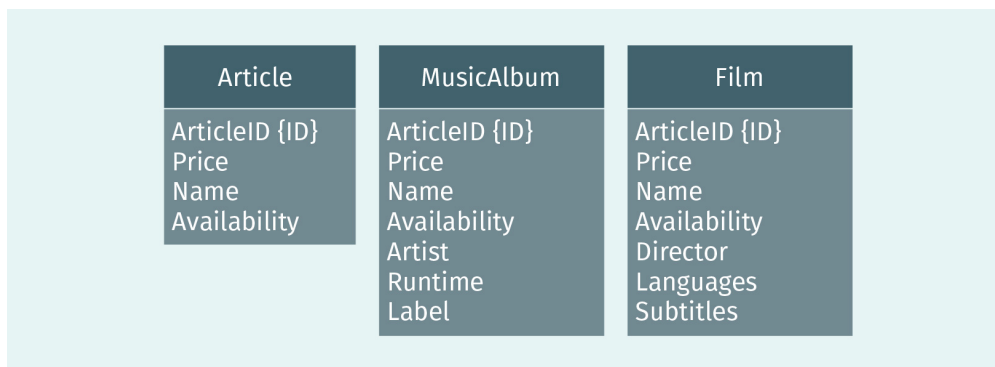
Advantages	Drawbacks
This is the strategy with the least redundancy in stored data.	Compared with the other strategies, it has the slowest mapping of inheritance.
The simple addition of further subclasses is very easy. Existing tables do not have to be changed in their structure.	To read and write data, several tables must always be requested. Therefore, the queries consist of several SQL statements.
The data from superclasses can be loaded using simple database tools (foreign key and join).	Exception: Access to the top superclass is very fast.

Source: Jörg Dreikauss, 2022.

Table per class

The table per class strategy also creates a separate table for each class in the inheritance hierarchy, as shown in the figure below. However, this only applies to concrete (i.e., not abstract) classes. Unlike the joined subclass table strategy, however, a separate table for each class contains all the attributes that are stored for concrete objects of that class. To load a record of a certain class, one access to the corresponding table is sufficient.

Figure 26: Example of Table Per Class Strategy



Source: Jörg Dreikauss, 2022.

The advantages and disadvantages of the table per class strategy are compared in the table below.

Table 41: Advantages and Disadvantages of the Table per Class Strategy

Advantages	Drawbacks
If the class name is known, the database can be accessed very efficiently because all of the required information is stored in one table.	Relationships to objects in a superclass where it is not clearly recognizable in which subclass the individual object is located are very difficult and time consuming to implement.
Adding additional classes is very easy. Existing tables do not have to be changed in their structure.	If the concrete subclass is not known, the table in which the data record is stored cannot be found either.
	Evaluations across the entire superclass require a large number of SQL commands because the objects of the superclass are distributed across different subclasses.

Source: Jörg Dreikauss, 2022.

4.3 Creating Tables in SQL Databases from ER Diagrams

Before data can be stored in relational databases, the appropriate table structure must be created. (In object-oriented programming, this step corresponds to creating classes before concrete objects of the class can be created.) Tables are created using Data Definition Language (DDL) expressions. The SQL language provides a number of keywords for this purpose, of which the most frequently used are presented below.

Data Types in SQL

A data type must be specified for each attribute of a table in SQL. When the data type is defined, the storage space to be reserved by the database is also automatically determined. In addition, the selection of the data type determines how the stored data is internally organized by the DBMS.

SQL does specify a number of data types, but whether these are actually implemented in a DBMS depends on the manufacturer of the DBMS. In addition, concrete DBMSs have their own data types that are not part of the SQL standard but have proven practical in daily work. Thus, the amount of data types actually available in a database can vary considerably from manufacturer to manufacturer. Before defining a concrete data schema for a database, the developer must know the name, version number, and manufacturer of the DBMS. In addition, the technical documentation of the database must be available so that the concrete implementation and specific features can be read when using data types.



TIP

If standard data types that are supported by almost all DBMSs are used to define the data schema, it is easier to port a database to another DBMS later.

Text types

Basically, there are text types that store either a fixed, a maximum, or a virtually unlimited number of characters, as shown in the table below.

Table 42: Text Types in SQL, Selection

Form	SQL data type	Examples
Character strings with always exactly <i>n</i> characters	CHAR(<i>n</i>) alternative: CHARACTER(<i>n</i>)	Country codes: DE, CN, FR, NL etc.: CHAR(2) ISBN 13 numbers: 9783836216999 CHAR(13) UUIDs: 21EC2020-3AEA-1069-A2DD-08002B30309D CHAR(36) (German) Postal codes: 45147 CHAR(5)
Character strings with variable but limited length (max. <i>n</i> characters)	VARCHAR(<i>n</i>) alternative: CHARACTER VARYING(<i>20</i>)	proper names, designations, short descriptions: VARCHAR(55), VARCHAR(160), VARCHAR(500)

Form	SQL data type	Examples
Very long character strings; the actual allowed size is limited by technical parameters within the DBMS.	TEXT	Texts, blog entries, detailed descriptions: TEXT

Source: Jörg Dreikauss, 2022.

There may be a requirement for the database to store text types in a specific character format, such as UTF-8 or latin1. In this case, the character format in which the text characters are stored must also be taken into account when defining text attributes. This is particularly important for error-free handling of special characters and umlauts (ä, ü, ö).

Number Types

The table below shows a selection of number types. The data type DECIMAL is recommended for storing monetary amounts because this type specifies the exact number of decimal places required.

Table 43: Number Types in SQL, Selection

Form	SQL data type	Examples
Integers; 32 bits, value range from -2,147,483,648 to 2,147,483,647	INTEGER alternative: INT	Number, index, numbering: INTEGER
Decimal numbers with a maximum of n digits in total and exactly m digits after the decimal point.	DECIMAL(n, m)	Amounts of money: DECIMAL(9,2) , Key figures: DECIMAL(4,2) ,
Floating point numbers, 32 bit	REAL	Measured values: REAL

Source: Jörg Dreikauss, 2022.

Date types

The table below gives an overview of frequently used data types for storing date and time information.

Table 44: Date Types in SQL, Selection

Form	SQL data type	Examples
Calendar data accurate to the day, from year 1000 to year 9999 in YYYY-MM-DD format	DATE	Date, calendar days: 2014-06-14
Time data in the format HH:MM:SS	TIME	Time, to the second: 13:43:56

Form	SQL data type	Examples
Exact determination of a time, exact to the second, often in the format YYYY-MM-DD HH:MM:SS	TIMESTAMP	Selected times: 2014-06-14 14:00:02; Automatic saving of creation or modification times

Source: Jörg Dreikauss, 2022.

Other data types

The table below presents one data type each for storing binary data and Boolean values. Concrete DBMSs often offer their own data types for storing binary types. A common terminology used for large binary types is the binary large object (BLOB). A BLOB could hold a large amount of binary data like a document, picture or video. The concrete implementation of Boolean also varies widely; some manufacturers store the values TRUE/FALSE, others the values 0/1.

Table 45: Additional Data Types in SQL, Selection

Form	SQL data type	Examples
Binary types with variable length, but not more than n characters (n bits)	VARBINARY(n)	Store binary data such as images, audio, video, and other binary files; BINARY VARYING(50000); often also called BLOB in DBMS
Boolean Values	BOOLEAN	Boolean Values

Source: Jörg Dreikauss, 2022.



TIP

Familiarize yourself with the data types available for the DBMS MariaDB. All examples in this script have been implemented in this DBMS and can thus be traced.

Creating Tables with SQL

When you create a table in a DBMS, you are creating the structure in which the data records will later be physically stored. The following specifications can be defined for a table:

- name of the table that is unique throughout the database
- columns of the table, one unique name per column, the data type of the column, prohibition of NULL values, obligation to have unique values
- primary key of the table (single or compound)

- foreign key relationships and their dependencies
- professional integrity conditions

The SQL command for creating a table is `CREATE TABLE`. The following example of an SQL statement creates a table named `Customer`. This table has four columns: `CustomerID` for storing integer values, `CustomerNR` for storing a customer number with a maximum length of fifteen characters, and then the columns `LastName` and `FirstName` for storing character strings with a maximum length of 500 characters.

SQL statement

Code

```
CREATE TABLE Customer (
    CustomerID INTEGER,
    LastName VARCHAR(500),
    FirstName VARCHAR(500),
    CustomerNR VARCHAR(15)
)
```

Result

Customer			
CustomerID	Last name	First name	CustomerNR



TIP

If changes must be made to the table definition after the initial creation, the SQL command `ALTER TABLE` is used.

General Structure of CREATE

The general structure of `CREATE TABLE` consists of the elements described in detail below:

Code

```
CREATE TABLE Name of the table (
    Column definition,
    Primary key definition,
    [Integrity rules, background],
    [Foreign key definition],
    [Specifying conditions for columns]
);
```

Column Definition

The following properties can be defined for a table column with SQL: name, data type, default value, prohibition of NULL values, conditions to be observed, and the obligation to uniqueness of values. The general schema for defining columns can be specified as follows, [elements] in square brackets are optional:

Code

```
name Data type [NOT NULL] [DEFAULT default value] [UNIQUE] [CHECK (condition)]
```

The table below describes the elements and contains concrete application examples.

Table 46: Elements for Column Definition

Elements for column definition	Description	Example
name	table-wide unique identifier for the column	LastName, FirstName, CustomerID
data type	concrete data type of the column, specifies the format of the values stored in the column	CustomerID INTEGER, LastName VARCHAR(500), OrderDate DATE
Optional: NOT NULL	prohibits NULL values in the column and forces the entry of a valid value from the value range of the data type	CustomerID INTEGER NOT NULL, LastName VARCHAR(500) NOT NULL
Optional: DEFAULT 'default value'	defines a default value for this attribute if no value was passed for this attribute and prevents NULL values	Stars INTEGER DEFAULT '0', Text VARCHAR(5000) DEFAULT 'no Text entered'
Optional: UNIQUE	forces the uniqueness of all values in this column, i.e., no value may occur more than once in this column	CustomerID INTEGER UNIQUE
Optional: CHECK (condition)	specification of conditions that must be fulfilled for the entry of the data set, e.g., checking of permitted values, dependencies on other attributes of the data set	StartDate DATE, EndDate DATE CHECK (EndDate > StartDate),

Source: Jörg Dreikauss, 2022.

Primary Key Definition with PRIMARY KEY

After defining the individual columns of the table, you can use the keyword PRIMARY KEY to determine the set of columns from which the primary key for the table is formed. A column defined as a primary key may not contain NULL values, and no value may be

assigned more than once for the combination of the different primary key columns. (If a primary key consists of several columns, each of these columns may contain multiple values.)

SQL statement

Code

```
CREATE TABLE Address (  
    AddressID INTEGER,  
    Street VARCHAR(500),  
    HouseNumber VARCHAR(50),  
    City VARCHAR(500),  
    PostalCode CHAR(5),  
    PRIMARY KEY (AddressID)  
);
```

Table 47: Address

AddressID	Street	HouseNumber	City	Postal-Code

Source: Jörg Dreikauss, 2022.

In the above example, an artificial key was defined for this table with the attribute AddressID. This key is only used within the DBMS and has no technical equivalent in the real world. When artificial keys are used, the DBMS can also automatically generate unique values. Provided that the data type of the artificial key is an integer type, an unassigned, and thus unique value, can be generated and stored for each new data record by automatic incrementing after specifying a manufacturer-dependent SQL keyword. In the following example for the DBMS MariaDB, the keyword is `AUTO_INCREMENT` and is part of the column definition:

Code

```
CREATE TABLE Address (  
    AddressID INTEGER AUTO_INCREMENT,  
    Street VARCHAR(500),  
    HouseNumber VARCHAR(50),  
    City VARCHAR(500),  
    PostalCode CHAR(5),  
    PRIMARY KEY (AddressID)  
);
```

The keyword `AUTO_INCREMENT` may only be used once per table.

Foreign Key Definition with FOREIGN KEY

Foreign keys of a table are defined using the SQL keyword `FOREIGN KEY` and a table column of another table whose values are to be used as foreign keys. The following example creates an “Address” table with the foreign key `CustomerID`:

Code

```
CREATE TABLE Address (  
    AddressID INTEGER AUTO_INCREMENT,  
    Street VARCHAR(500),  
    HouseNumber VARCHAR(50),  
    City VARCHAR(500),  
    PostalCode CHAR(5),  
    PRIMARY KEY (AddressID),  
    CustomerID INTEGER NOT NULL,  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)  
);
```

First, another column with the name `CustomerID` is created. The `FOREIGN KEY` statement then defines that the values in the `CustomerID` column are foreign keys that refer to the `CustomerID` column of the `Customer` table.



TIP

The introduction of the foreign key in the table “Address” is only possible if, in the table “Customer,” the reference to Customer ID was defined as the primary key.

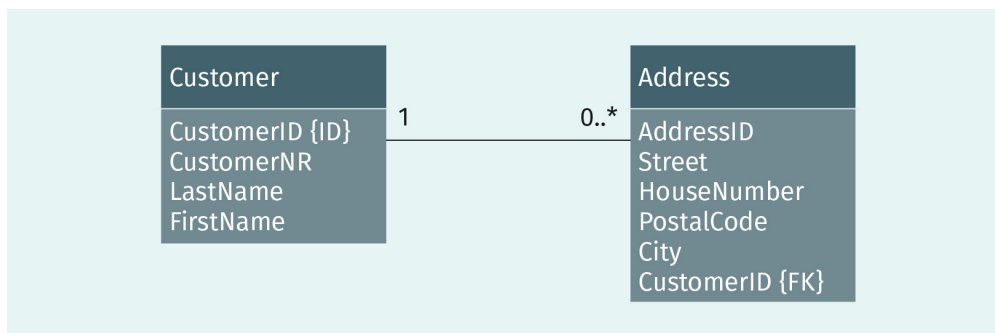
Table 48: Address

AddressID	Street	HouseNum- ber	City	Post- alCode	CustomerID (FK, Customer → CustomerID)

Source: Jörg Dreikauss, 2022.

The established foreign key relationship between customer and address corresponds to a 1:CN relationship between the two tables shown in the figure below.

Figure 27: Example for Foreign Keys



Source: Jörg Dreikauss, 2022.

Even for foreign key attributes, the **UNIQUE** and **NOT NULL** keywords can be used to force the uniqueness of all values of this attribute or the prohibition of **NULL** values. In this way, 1:1 and 1:CN relationships can be implemented.

Preservation of referential integrity

Once a foreign key relationship has been established, a logical dependency of the referencing table on the referenced table is established. These dependencies must be taken into account when updating and deleting data records of the referenced table so that referential integrity is maintained. When you define a foreign key, you can also specify how the DBMS reacts if the referenced table is changed. The keywords **ON UPDATE** and **ON DELETE** are used, as in the following example.

Code

```
CREATE TABLE Address (  
    AddressID INTEGER AUTO_INCREMENT,  
    [... ],  
    CustomerID INTEGER NOT NULL,  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

A different reaction can be defined for changes. The behavior when changing the referenced record is defined with the keyword **ON UPDATE**; the behavior when deleting is defined with the keyword **ON DELETE**. The possible actions that SQL provides to maintain referential integrity are described in the table below. The actual implementation, however, is manufacturer-dependent, as is the case with the data types.

Table 49: Preservation of Referential Integrity

Action	Key word	Description
Automatic acceptance of changes	CASCADE	If the referenced record is changed or deleted, the referencing record is automatically changed or deleted as well.
Set foreign key to NULL value	SET NULL	In the event of a change or deletion, the value of the foreign key attribute of the referencing table is set to NULL. However, the foreign key attribute must not be defined as NOT NULL.
Prohibit deletions or changes to the referenced table	RESTRICT	Deletions (ON DELETE) or changes (ON UPDATE) to the referenced table are prohibited by the DBMS; that is, they are not executed.
No changes to the referencing table	NO ACTION	Although provided for in the SQL standard, this would interfere with referential integrity.
Reset to default value	SET DEFAULT	Sets the foreign key to a default value.

Source: Jörg Dreikauss, 2022.

Technical Integrity Conditions with CHECK

You can use the SQL keyword CHECK to formulate conditions for columns in a table that are checked before the data are changed in the database. In this way, compliance with certain technical conditions can be assured before a data record is entered into the database. The following example checks whether the value in the start action is before the value in the end action.

Code

```
CREATE TABLE VoucherAction (  
    ActionID INTEGER AUTO_INCREMENT,  
    StartAction DATE,  
    EndAction DATE,  
    VoucherCode CHAR(8),  
    PRIMARY KEY (ActionID),  
    CHECK (StartAction < EndAction)  
);
```



TIP

The DBMS MariaDB does not support conditions formulated with CHECK.



SUMMARY

Before creating the actual table structure in a database, the conceptual data model must be converted to a physical data model. All relationships from the conceptual data model that cannot be mapped directly into a database are transformed into a physical data model beforehand. 1:1 relationships and 1:N relationships are mapped with the help of 1:C or 1:CN relationships and additional conditions. In addition, these additional conditions have to be implemented and become part of the application logic.

N:M relationships and inheritance relationships must first be mapped to 1:CN relationships before a concrete database schema can be created.

The SQL command `CREATE TABLE` creates a structure in the DBMS in which the data records are later physically stored. In addition to the name of the table and the column definitions, primary and foreign keys as well as functional integrity conditions are defined for a table.

UNIT 5

COMPLEX DATABASE QUERIES ON MULTIPLE TABLES

STUDY GOALS

Upon completion of this unit, you will have learned ...

- what compound sets are, what different compound types there are in SQL, and how data can be read out in several tables with compound sets.
- which set operations are supported by SQL and how they are used.
- how complex database queries can be reused with SQL.

5. COMPLEX DATABASE QUERIES ON MULTIPLE TABLES

Introduction

In practice, relational databases usually consist of several – often well over one hundred – different tables. Queries to databases must therefore take several tables into account. The composite quantities and quantity operations used for this purpose are presented in this lesson. We also show how complex database queries can be stored so that they can be reused multiple times.

5.1 Composite Quantities (JOIN)

Composite quantity, or join

This is a relation that is composed of already-existing relations.

As a rule, the data stock in relational databases is distributed over several tables. However, composite quantities are used to retrieve specific information from the database that is distributed over several tables. The basic idea behind **composite quantities** is to generate a new relation from several already existing relations. The `SELECT` statement is used to generate a result table from a database table. With the composite quantities presented in the following, targeted information from several database tables can also be compiled in a virtual table.

Basic Principle of Composite Quantities

The following example illustrates how a compound set works: There are two tables, `Article` and `Film`, in a database, where the `ArticleID` attribute of the `Film` table is a foreign key that references the `ArticleID` attribute of the `Article` table:

Table 50: Article

ArticleID	ArticleNR	Price	Name	Availability
1	UZEHN78126	10.99	On the Road to Nothingness	Immediate
2	ZHBH789123	13.99	Behind Us	Immediate
5	KJNSM7873	9.99	Help you	In 3 days
6	UIHJKHSD7	8.99	Around the Planet	In 2 days

Source: Jörg Dreikauss, 2022.

Table 51: Film

Film-ID	Director	Languages	Subtitles	NumberOfDiscs	Studio	Medium	ArticleID (FK, Article -> ArticleID)
3	Sophie Little	DE, EN, FR	DE, EN	1	DreamPictures	DVD	1
4	Fred van Heldt	EN, FR	EN, FR, DE	2	MovieFactory	DVD	2

Source: Jörg Dreikauss, 2022.



TIP

We are looking for all films that can be distributed as well as for their titles, directors, prices, and availability.

With the help of a composite, both tables `Article` and `Film` are merged into one table (also: combined). The new table created in this way has all the columns from `Article` and `Film`.

<u>ArticleID</u>	ArticleNR	Price	Name	Availability	<u>FilmID</u>	Director	Languages	Subtitles	NumberOfDiscs	Studio	Medium	ArticleID
------------------	-----------	-------	------	--------------	---------------	----------	-----------	-----------	---------------	--------	--------	-----------

When tables are merged, the data records are also merged. Because each row in a table corresponds to a data record, you must specify which data records are to be merged for a compound. For this purpose, a compound criterion must be determined. In the example, the records that have the same value in the `ArticleID` column are merged. If a record in the `Article` table has the same `ArticleID` as a record in `Film`, these records are combined in the new table. The formal comparison criterion is therefore `Article.ArticleID=Film.ArticleID`.

After merging the data records, the resulting table is as follows:

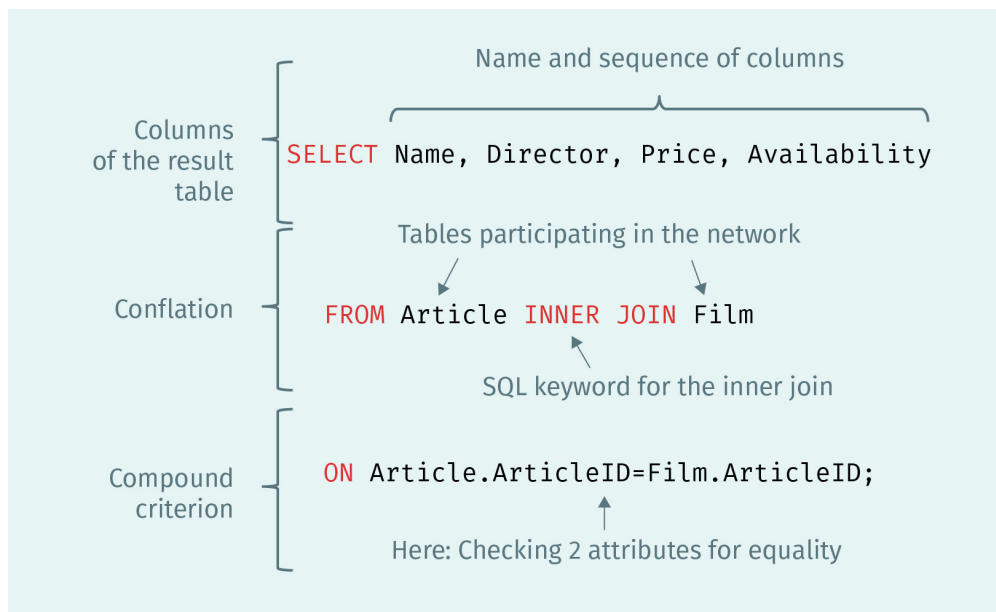
ArticleID	ArticleNR	Price	Name	Availability	FilmID	Director	Languages	Subtitles	NumberOfDiscs	Studio	Medium	ArticleID
1	UZEH N781 26	10,99	On the Road to Noth- ingness	Imm- edi- ate	3	Sophie Little	DE, EN, FR	DE, EN	1	Dream- Pictures	DVD	1
2	ZHBH 78912 3	13,99	Behind Us	Imm- edi- ate	4	Fred van Heldt	EN, FR	EN, FR, DE	2	Movie- Factory	DVD	2

However, because not all information on each film will be searched for but only the attribute's name, director, price, and availability, only these desired columns are selected in the compound set and displayed in the chosen order. All other columns are not required. Therefore, the outcome of the request is as follows:

Name	Director	Price	Availability
On the Road to Nothingness	Sophie Little	10.99	Immediate
Behind Us	Fred van Heldt	13.99	Immediate

The SQL syntax for the example just described is shown and explained in the figure below.

Figure 28: SQL Example of a Compound Set



Source: Jörg Dreikauss, 2022.

Composite Types

Different types of alliances are distinguished in relational databases. In addition to the inner join, there is also frequent use of the natural join, the left join, and the right join. The example database shown in the table below shows the most important types of alliances.

Figure 29: Example Database for the Illustration of Collaborations Database: JOINTEST

Table: Article			Table: Film		
ArticleID	ABC	Number	ArticleID	Number	DEF
1	U	11	1	11	A
2	V	33	3	22	B
3	W	55	5	33	C
4	X	66	7	44	D

Source: Jörg Dreikauss, 2022.

The behaviors of the different composite types are explained in the table below using a concrete application example.

**TIP**

Create a practice database in a DBMS of your choice that you can use to reproduce the compound types shown in the table below.

Table 52: Important Compound Types of Relational Databases

Composite type with SQL statement	Result																		
INNER JOIN with ON Compound criterion <pre>SELECT * FROM Article INNER JOIN Film ON Article.ArticleID=Film.ArticleID;</pre>	This merges all data records for which the compound criterion is fulfilled. All columns of all tables are included in the result. <table><tr><th>ArticleID</th><th>ABC</th><th>Number</th><th>ArticleID</th><th>Number</th><th>DEF</th></tr><tr><td>1</td><td>U</td><td>11</td><td>1</td><td>11</td><td>A</td></tr><tr><td>3</td><td>W</td><td>55</td><td>3</td><td>22</td><td>B</td></tr></table>	ArticleID	ABC	Number	ArticleID	Number	DEF	1	U	11	1	11	A	3	W	55	3	22	B
ArticleID	ABC	Number	ArticleID	Number	DEF														
1	U	11	1	11	A														
3	W	55	3	22	B														
INNER JOIN with USING(columns); used if the columns of the compound criterion have the same name. <pre>SELECT * FROM Article INNER JOIN Film USING(ArticleID);</pre> Note: USING can contain one or more columns as parameters.	This merges all data records whose values match in all columns named as parameters of USING. The comparison columns only exist once in the result table. <table><tr><th>ArticleID</th><th>ABC</th><th>Number</th><th>Number</th><th>DEF</th></tr><tr><td>1</td><td>U</td><td>11</td><td>11</td><td>A</td></tr><tr><td>3</td><td>W</td><td>55</td><td>22</td><td>B</td></tr></table>	ArticleID	ABC	Number	Number	DEF	1	U	11	11	A	3	W	55	22	B			
ArticleID	ABC	Number	Number	DEF															
1	U	11	11	A															
3	W	55	22	B															
INNER JOIN is often abbreviated as JOIN, i.e., a JOIN command without a more precise specification is an INNER JOIN.																			
NATURAL JOIN: Variant of INNER JOIN <pre>SELECT * FROM Article NATURAL JOIN Film;</pre>	This compares the records of all columns with the same column name. In the example, this would be the columns ArticleID and Number. The data record is only included in the result table if there is a complete match. The columns with the same name exist only once in the result. <table><tr><th>ArticleID</th><th>Number</th><th>ABC</th><th>DEF</th></tr><tr><td>1</td><td>11</td><td>U</td><td>A</td></tr></table>	ArticleID	Number	ABC	DEF	1	11	U	A										
ArticleID	Number	ABC	DEF																
1	11	U	A																
Instead of ON, you can also specify the JOIN condition with USING for LEFT JOIN and RIGHT JOIN.																			

Composite type with SQL statement	Result																														
<pre>LEFT JOIN SELECT * FROM Article LEFT JOIN Film ON Article.ArticleID=Film.ArticleID;</pre>	This merges all data records for which the compound criterion is fulfilled. All columns of all tables are included in the result. In addition, all data records of the table to the left of the JOIN statement (here: Article) are included in the result. The columns copied from the right table that have no entries matching the left table IDs are filled with NULL. <table><tr><th>ArticleID</th><th>ABC</th><th>Number</th><th>ArticleID</th><th>Number</th><th>DEF</th></tr><tr><td>1</td><td>U</td><td>11</td><td>1</td><td>11</td><td>A</td></tr><tr><td>3</td><td>W</td><td>55</td><td>3</td><td>22</td><td>B</td></tr><tr><td>2</td><td>V</td><td>33</td><td>NULL</td><td>NULL</td><td>NULL</td></tr><tr><td>4</td><td>X</td><td>66</td><td>NULL</td><td>NULL</td><td>NULL</td></tr></table>	ArticleID	ABC	Number	ArticleID	Number	DEF	1	U	11	1	11	A	3	W	55	3	22	B	2	V	33	NULL	NULL	NULL	4	X	66	NULL	NULL	NULL
ArticleID	ABC	Number	ArticleID	Number	DEF																										
1	U	11	1	11	A																										
3	W	55	3	22	B																										
2	V	33	NULL	NULL	NULL																										
4	X	66	NULL	NULL	NULL																										
<pre>RIGHT JOIN SELECT * FROM Article RIGHT JOIN Film ON Article.ArticleID=Film.ArticleID;</pre>	This merges all data records for which the compound criterion is fulfilled. All columns of all tables are included in the result. In addition, all data records from the table to the right of the JOIN statement (here: film) are included in the result. The columns copied from the left table that have no entries matching the right table IDs are filled with NULL. <table><tr><th>ArticleID</th><th>ABC</th><th>Number</th><th>ArticleID</th><th>Number</th><th>DEF</th></tr><tr><td>1</td><td>U</td><td>11</td><td>1</td><td>11</td><td>A</td></tr><tr><td>3</td><td>W</td><td>55</td><td>3</td><td>22</td><td>B</td></tr><tr><td>NULL</td><td>NULL</td><td>NULL</td><td>5</td><td>33</td><td>C</td></tr><tr><td>NULL</td><td>NULL</td><td>NULL</td><td>7</td><td>44</td><td>D</td></tr></table>	ArticleID	ABC	Number	ArticleID	Number	DEF	1	U	11	1	11	A	3	W	55	3	22	B	NULL	NULL	NULL	5	33	C	NULL	NULL	NULL	7	44	D
ArticleID	ABC	Number	ArticleID	Number	DEF																										
1	U	11	1	11	A																										
3	W	55	3	22	B																										
NULL	NULL	NULL	5	33	C																										
NULL	NULL	NULL	7	44	D																										

Source: Jörg Dreikauss, 2022.

The use of associations does not fundamentally change the structure of the SELECT statement. The result set of a compound can also be grouped with GROUP BY and sorted with ORDER BY. The following example determines the number of instantly available movies delivered on more than one disc, grouped by the number of discs:

Code

```
SELECT NumberOfDiscs, COUNT(*) AS Number
FROM Article NATURALJOIN Film
WHERE Availability='Immediate' AND NumberOfDiscs > 1
GROUP BY NumberOfDiscs;
```


Link to WHERE

An outdated alternative to maintaining inner networks can be implemented with the keyword `WHERE`, as shown in the table below. This variant returns the same result as `INNER JOIN`, but mixes the compound criterion and the filter condition and is therefore considered bad programming style.

Table 53: Internal Bond with WHERE

Composite Type with SQL Statement	Result																		
INNER JOIN with WHERE <pre>SELECT * FROM Article, Film WHERE Article.ArticleID=Film.ArticleID;</pre>	This creates a result with all records from articles and films for which WHERE conditions are met. <table><tr><th>ArticleID</th><th>ABC</th><th>Number</th><th>ArticleID</th><th>Number</th><th>DEF</th></tr><tr><td>1</td><td>U</td><td>11</td><td>1</td><td>11</td><td>A</td></tr><tr><td>3</td><td>W</td><td>55</td><td>3</td><td>22</td><td>B</td></tr></table>	ArticleID	ABC	Number	ArticleID	Number	DEF	1	U	11	1	11	A	3	W	55	3	22	B
ArticleID	ABC	Number	ArticleID	Number	DEF														
1	U	11	1	11	A														
3	W	55	3	22	B														

Source: Jörg Dreikauss, 2022.



TIP

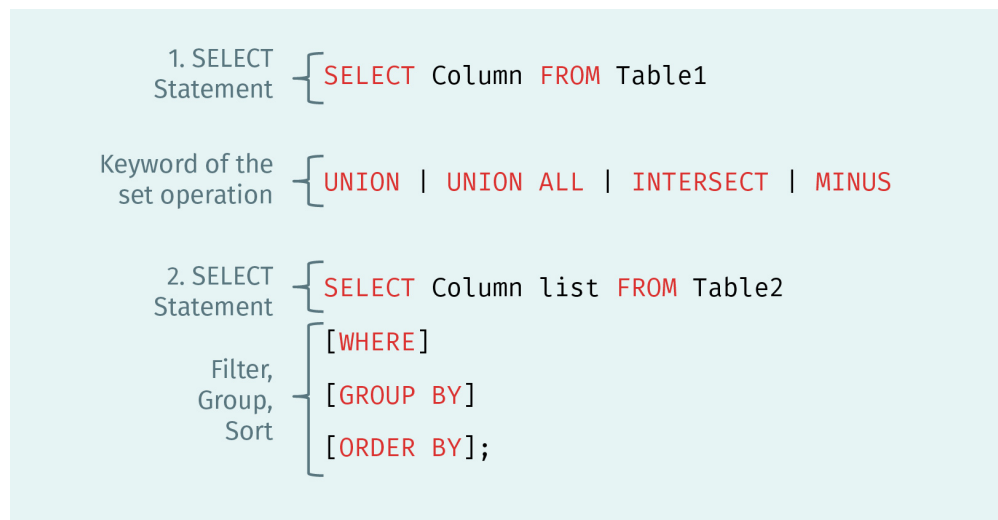
Load the provided Mondial database (consisting of the SQL files `mondial-inputs-mysql.sql` and `mondial-schema-mysql.sql`) into a database system of your choice. Familiarize yourself with the structure of the Province, City, and Country tables. Please use the website of the Department of Mathematics and Computer Science of the University of Göttingen (Mondial Database, n.d.), or see MyCampus.

5.2 Set Operations

The result of a `SELECT` request is a set of identically structured data records. Therefore, in principle it is possible to perform set operations on these results. The SQL standard specifies the set operations `Union`, `Intersection`, and `Minus`. However, many DBMSs do not implement the operations `Intersection` and `Difference Set`, since they are rarely needed in practice and can be replicated with `WHERE` using conditions.

Because set operations deliver sets as results, they can be used to clean up the result sets of several `SELECT` queries and then process them further. The general form for the set operation is shown in the figure below.

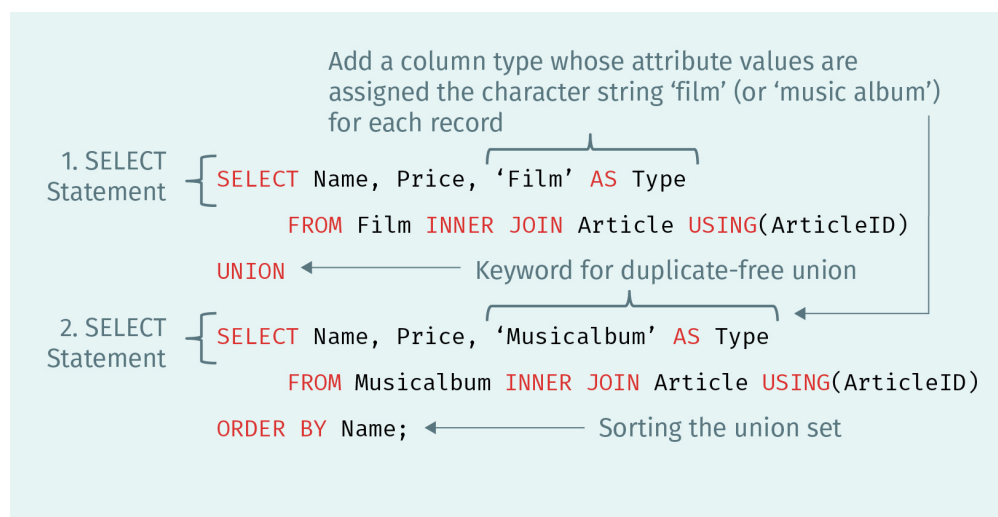
Figure 30: Schema for Set Operations



Source: Jörg Dreikauss, 2022.

The keyword for a duplicate-free union is UNION. If duplicates are not to be eliminated during unification, the keyword UNION ALL must be used. The keyword INTERSECT is used for an intersection and, depending upon the DBMS used, the keyword MINUS (e.g., Oracle) or EXCEPT (e.g., MariaDB, since version 10.3.0) is used for differences. The following example illustrates the use of the set operation union: The stock of all articles in the online shop, with name, price, and article type sorted by name, is to be indicated regardless of which concrete article type it concerns. The SQL statement shown and explained in the figure below can be formulated for this purpose.

Figure 31: Example for the Set Operation UNION



Source: Jörg Dreikauss, 2022.

The result set of the individual SELECT requests must have the same number of columns, otherwise the set operation will fail. If, however, data records with different attributes have to be combined, the unequal columns can be filled with NULL or another constant. In the above example, adding the Type column and filling the attributes with the fixed values “Film” or “Music album” provides information about the origin of the records to be merged.

5.3 Data Views with CREATE VIEW

In order not to have to formulate complex, frequently required database queries anew each time as SQL statements, relational databases offer the possibility to create views. Another application of views is the implementation of access restrictions on data for different user groups. In principle, these are SELECT statements that are stored under a specific name in the database. No additional data records are stored in a view. A view is therefore sometimes called an “imaginary” or “virtual” table. A generated view can be addressed in other SQL statements like a table in the database.

Example

A view is to be created in the ONLINESHOP database that contains a list sorted by name of all immediately available films and their detailed information.

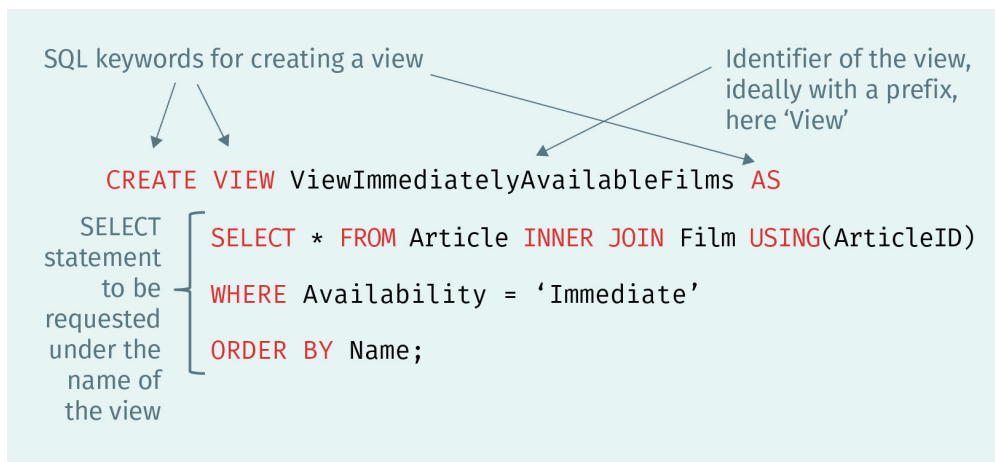
SELECT statement

Code

```
SELECT * FROM Article INNER JOIN Film USING(ArticleID)
WHERE Availability = 'Immediate' ORDER BY Name;
```

To create a view, use the SQL statement CREATE VIEW as shown in the figure below. Using a prefix, views can be clearly distinguished from tables when used later.

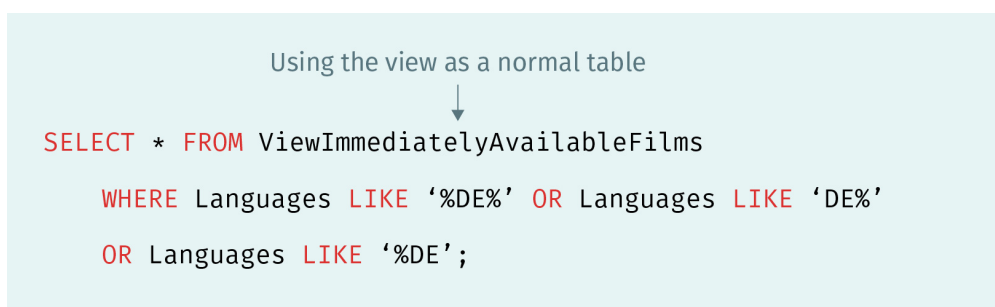
Figure 32: Example for CREATE VIEW



Source: Jörg Dreikauss, 2022.

The view created above can now be reused under the name ViewImmediatelyAvailableFilms. The SQL command in the figure below uses the created view to output all immediately available movies that are available in German. The view is used there like a normal database table in the SELECT statement.

Figure 33: Reuse of a VIEW



Source: Jörg Dreikauss, 2022.

Views are primarily used to read data. Under the following conditions, you can also make changes to the dataset directly in the view:

- The view does not contain an aggregate function, constant, or expression;
- The view only refers to one table, so no JOIN is allowed;
- The view does not use DISTINCT and does not use GROUP BY;
- The view does not contain a subquery.

If all these conditions are met, the view is writable. Otherwise, a Read Only View is defined.



SUMMARY

Composite quantities are used to retrieve information that is distributed over several tables. The basic idea behind the composite quantities is to generate a new relation from several already existing relations. Typical compound types in SQL are `INNER JOIN`, `NATURAL JOIN`, `LEFT JOIN`, and `RIGHT JOIN`. In addition, SQL supports set operations such as `UNION` on similar result sets of SQL queries. The keyword `CREATE VIEW` can be used to create virtual tables in the database that can be used to easily reuse complex database queries.

UNIT 6

MANIPULATING RECORDS IN DATABASES

STUDY GOALS

Upon completion of this unit, you will have learned ...

- how to use SQL to insert new records into a database.
- how to use SQL to change both data records and the data schema of a database.
- what transactions are, what properties they have, and what transaction strategies are possible for a multiuser operation.

6. MANIPULATING RECORDS IN DATABASES

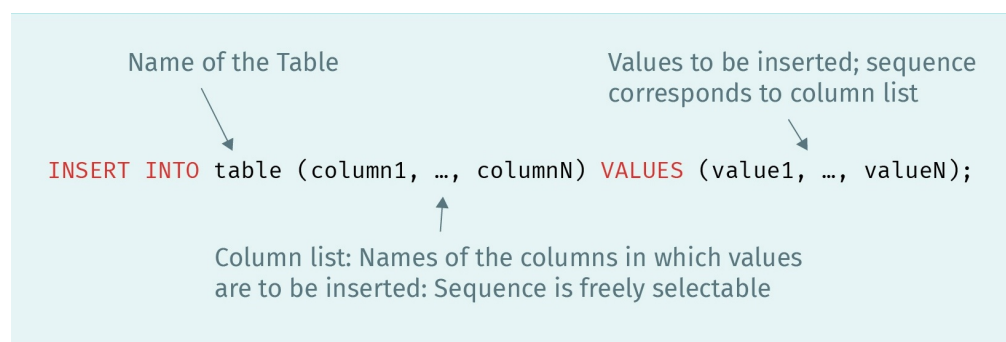
Introduction

In daily use, data are not only read from a database but are also changed. This lesson introduces the elements of SQL that can be used to insert, modify, and delete data in a database. It also introduces SQL elements that can be used to change the data schema of a database. It then introduces the transaction concept and describes how transactions can be used to ensure the consistency of databases.

6.1 Inserting New Data Records (INSERT)

Inserting new records into a relational database is done with the SQL statement `INSERT`. The general schema of an `INSERT` statement is shown in the figure below and is described in detail in the following table.

Figure 34: Schema of an `INSERT` Statement



Source: Jörg Dreikauss, 2022.

Table 54: Elements in the `INSERT` Statement

Elements of <code>INSERT</code>	Description
<code>table</code>	This is the name of the table into which the records are to be inserted. Exactly one table can be changed in each <code>INSERT</code> statement.
Optional: (<code>column 1, ..., column N</code>)	This lists the names of the columns in which the values are to be entered. At least one column must be specified; the order of the columns can be freely selected. If the list of columns is omitted, the values for all columns of the table must be inserted.

Elements of INSERT	Description
(value1,...,valueN)	This is the list of values to be inserted. The order of the values must match those in the column list. If no column list has been defined, the values must be specified for all columns in the table. The order is then given by the order of the columns in the database table specified in the CREATE TABLE statement.

Source: Jörg Dreikauss, 2022.

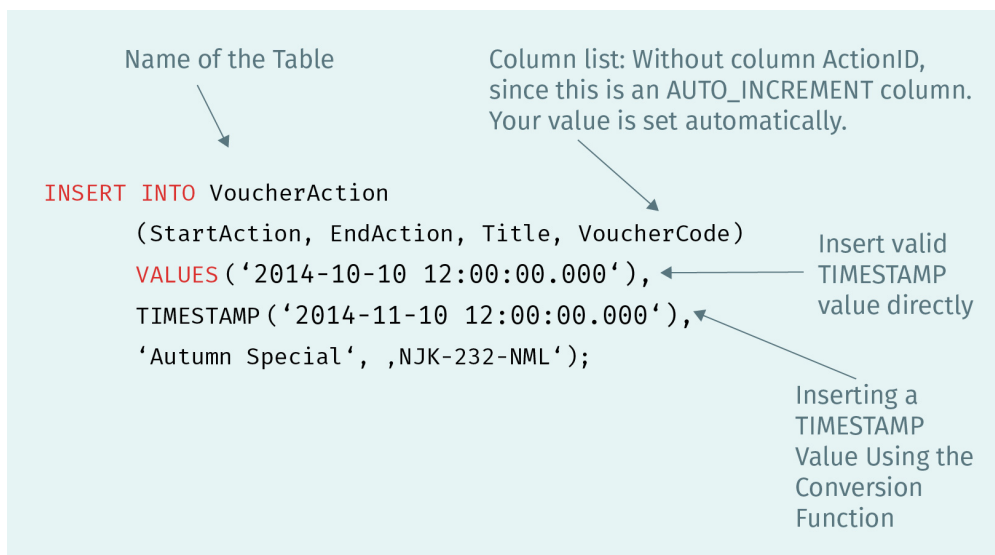
The following examples illustrate the INSERT statement using the following example table, VoucherAction, in which data on voucher actions of an online shop are stored:

Code

```
CREATE TABLE VoucherAction (
    ActionID INTEGER AUTO_INCREMENT,
    StartAction TIMESTAMP NOT NULL,
    EndAction TIMESTAMP NOT NULL,
    Title VARCHAR (500) NOT NULL,
    Description TEXT,
    VoucherCode VARCHAR(100) NOT NULL UNIQUE,
    PRIMARY KEY (ActionID)
);
```

An example of inserting a new record into the VoucherAction table is shown in the figure below.

Figure 35: Schema of an INSERT Statement



Source: Jörg Dreikauss, 2022.

Filling Columns Not Specified with INSERT

Each INSERT command adds a new record to a table. A new line is appended to the bottom of the table. Although not all columns must be specified in the INSERT statement, values must also be entered in these missing columns. SQL provides the following mechanisms for a DBMS to handle missing values:

- **AUTO_INCREMENT:** The value of the attribute is generated by the DBMS and used for artificial keys.
- **DEFAULT value:** If a default value has been defined for this attribute, this value is saved for the new data record.
- **NULL value:** If no default value has been specified, the DBMS saves a NULL value for this column. Note: Some DBMSs generate a default value instead of a NULL.
- **Error message:** If no default value has been specified and this column is defined with NOT NULL, an error message is issued and the execution of INSERT is terminated without changes to the DB.

Automatic Type Conversion

If, when inserting values into a table, a transferred value does not have the correct data type for the corresponding column, the data type is automatically converted (if possible). This conversion is lossy, i.e., information may be lost without notification during insertion. For example, character strings are truncated when the maximum length is exceeded, and numerical values are rounded. If the type conversion can be carried out without errors, a warning or error message will be returned by some DBMS but not by others, despite the loss of information. If a loss of information is to be avoided, a type check must take place in the business application before the values are transferred to the DBMS. The following example illustrates an automatic type conversion. After you have created Table t (shown here)

Code

```
CREATE TABLE t (  
    i INTEGER, d DECIMAL(5,2), f FLOAT, c VARCHAR(3), b BOOLEAN  
);
```

and the insertion of the two data records (shown here)

Code

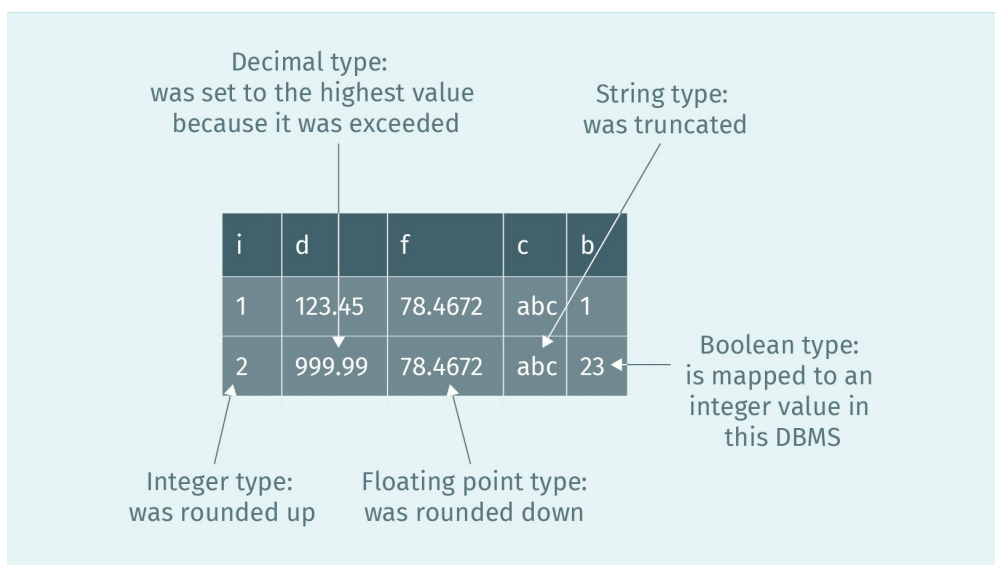
```
INSERT INTO t VALUES (1, 123.45, 78.46723, 'abc', TRUE);  
INSERT INTO t VALUES (1.5, 12345.678, 78.46723456789, 'abcdefghij', 23);
```

HINT

In this example to insert values into a table, errors may already occur depending on the DBMS, SQL trainer and simulator you are using! So the second INSERT INTO may cause an error message 'Data Transaction: Out of range value for ...'

the values shown in the figure below are stored in the database.

Figure 36: Examples of Automatic Type Conversions



i	d	f	c	b
1	123.45	78.4672	abc	1
2	999.99	78.4672	abc	23

Decimal type: was set to the highest value because it was exceeded

String type: was truncated

Boolean type: is mapped to an integer value in this DBMS

Integer type: was rounded up

Floating point type: was rounded down

Source: Jörg Dreikauss, 2022.

Complex Data Types

Complex data types such as DATE, TIME, and TIMESTAMP are supported by many DBMSs with conversion functions. As shown above, the values of these data types can be inserted into the database either directly or as parameters of the corresponding conversion functions with INSERT. How these functions are implemented and how they behave in the event of an error should be described in the DBMS manual. A NULL value is often generated when the conversion functions fail, while a default value is stored when saving without explicitly calling the conversion functions.

For example, the following table is created:

Code

```
CREATE TABLE KTEST (  
    dateNull DATE, timeNull TIME, timestampNull TIMESTAMP,  
    dateNotNull DATE NOT NULL, timeNotNull TIME NOT NULL,  
    timestampNotNull TIMESTAMP NOT NULL  
);
```

And then the following INSERT statements are called:

Code

```
INSERT INTO KTEST VALUES ('2014-11-11', '12:12:12',  
'2014-11-11 12:12:12', '2014-11-11', '12:12:12',  
'2014-11-11 12:12:12');  
INSERT INTO KTEST VALUES ('2014', '122X', '2014 122',  
'2014', '122', '2014 122');
```



HINT

In this example to insert values into a table, errors may already occur depending on the DBMS, SQL trainer and simulator you are using! So the second INSERT INTO may cause an error message 'Data Transaction: Out of range value for ...'

This is available in the DBMS (MySQL 5.6) of the following database:

dateNull	time-Null	timestampNull	dateNotNull	timeNot-Null	timestampNot-Null
2014-11-11	12:12:12	2014-11-11 12:12:12	2014-11-11	12:12:12	2014-11-11 12:12:12
0000-00-00	00:01:22	0000-00-00 00:00:00	0000-00-00	00:01:22	0000-00-00 00:00:00

However, if the data of the second INSERT statement are explicitly inserted with the following conversion function:

Code

```
INSERT INTO KTEST VALUES (DATE('2014'), TIME('122X'),  
TIMESTAMP('2014 122'), DATE('2014'), TIME('122'),  
TIMESTAMP('2014 122'));
```

then the DBMS will report an error and will not enter this data record. If the conversion fails, the DBMS will try to save a NULL value here, but this is not possible with NOT-NULL attributes.

Copying Data Records

With SQL, it is possible to create new records by copying existing records. To do so, the `INSERT` statement is combined with the `SELECT` statement. As a prerequisite for this, the number of columns and their data types in the result table of the `SELECT` statement must match the table in which the data records are to be inserted. The general schema for copying records is as follows:

Code

```
INSERT INTO table1 (column1,...,columnN)
SELECT column1,...,column FROM table1 WHERE condition;
```

The following example creates new data in the table created above by copying:

Code

```
INSERT INTO t SELECT * FROM t WHERE b=TRUE;
```

Preservation of Referential Integrity

Data that are related to a subject are often divided into several individual data records and distributed across several tables. When inserting such data records, special attention must be paid to ensuring that no incomplete data are included in the database even in the event of an error—databases with transactions presented in Section 6.3 support this requirement.

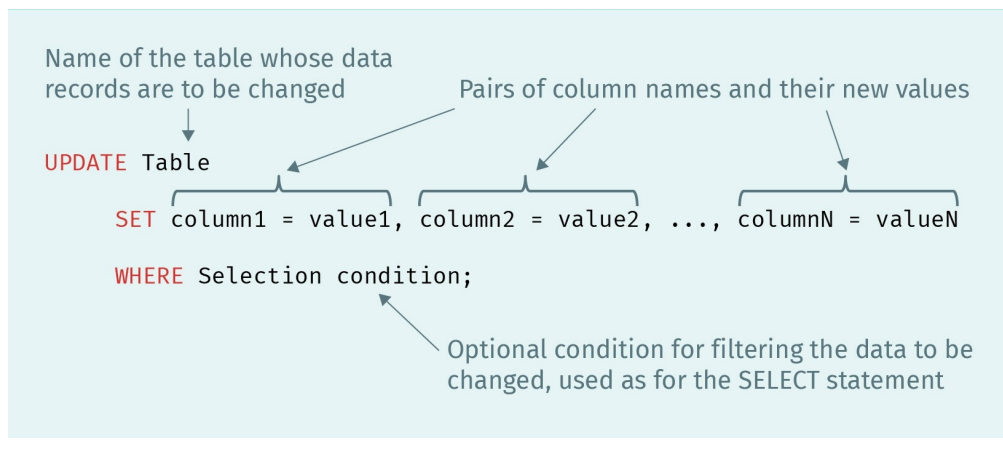
6.2 Changing Existing Records

Data records that already exist in the database can be changed. SQL offers the `UPDATE` and `DELETE` commands, which can be used to selectively change attributes in tables and completely delete data records.

Updating Data (UPDATE)

The `UPDATE` statement is used to change existing data records. The update schema is shown in the figure below.

Figure 37: Schema of UPDATE



Source: Jörg Dreikauss, 2022.

UPDATE can be used to enter specific values in data records. An example of changing the value in the Availability field to the immediate string of a specific article stored in the Article table follows:

Code

```
UPDATE Article SET Availability = 'Immediate' WHERE Name='Help you';
```

In addition, UPDATE can be used to recalculate values with reference to the old value. Here is an example where the price of all items costing less than 10 EUR is increased by 2 EUR:

Code

```
UPDATE Article SET Price = Price+2 WHERE Price < 10;
```

Each record in the table that matches the selection condition is changed. If no selection condition is formulated, all data records of the table are affected.

Deleting Data (DELETE)

The SQL statement DELETE is used to delete data records from the database. The structure of DELETE is very simple:

Code

```
DELETE FROM table WHERE condition;
```

Only the table name must be specified along with a selection condition. All data records for which the selection condition applies are irretrievably removed from the dataset. Without a condition, the entire contents of the table are deleted, but the structure of the table is retained.

**TIP**

Relational DBMSs usually do not have an undo function. Successfully deleted records cannot be recovered without backup.

Changing Tables (ALTER TABLE)

When creating tables in databases and also during the maintenance and further development of applications, it is usually necessary to adapt the current definition of tables. Adding, deleting, and changing columns in the table are typical change requests, as are adjusting primary and foreign keys or constraints. Changes to a table are possible with the SQL statement `ALTER TABLE`. The elements of `ALTER TABLE` are presented and described in table 55.

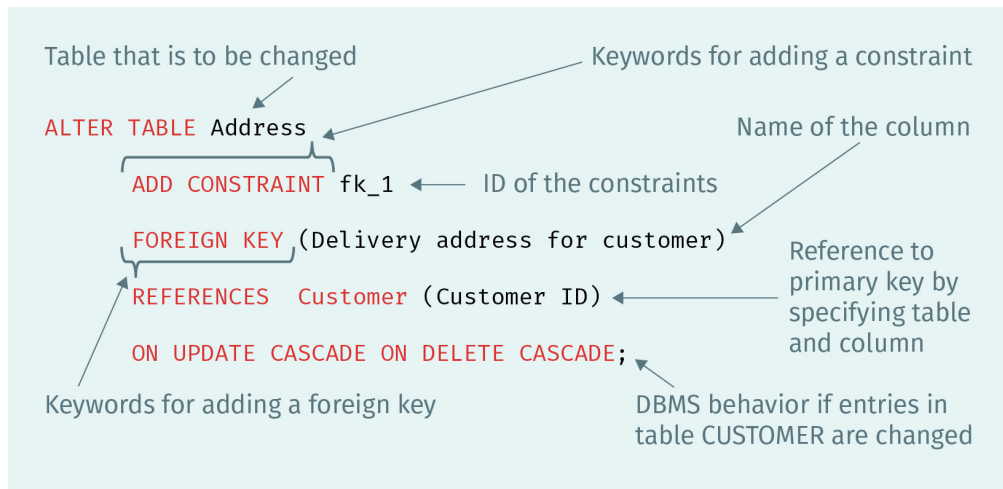
Table 55: Elements of ALTER TABLE

Elements of ALTER TABLE	Description
<code>ALTER TABLE table</code>	defines the table to be changed.
<code>ADD column definition</code>	adds a column; syntax as for <code>CREATE TABLE</code>
<code>ALTER column definition</code> in some DBMSs (like MySQL): <code>MODIFY column definition</code>	Changes properties of a column by specifying all properties; syntax as for <code>CREATE TABLE</code>
<code>DROP column</code>	deletes a column by specifying the column name
<code>ADD CONSTRAINT</code> <code>constraintdefinition</code>	adds a constraint: includes primary and foreign keys as well as <code>NOT NULL</code> and <code>UNIQUE</code> conditions, the latter also for multiple columns
<code>DROP CONSTRAINT</code> <code>constraintname</code>	deletes a constraint: includes primary and foreign keys as well as <code>NOT NULL</code> and <code>UNIQUE</code> conditions, the latter also for multiple columns

Source: Jörg Dreikauss, 2022.

A concrete example of adding and deleting constraints using a foreign key in DBMS MySQL 5.6 is shown in the figure below. However, the concrete syntax of `ALTER TABLE` varies depending on the DBMS used.

Figure 38: Example of a Constraint: Foreign Key



Source: Jörg Dreikauss, 2022.

An example of defining of a primary key with `ALTER TABLE` follows:

Code

```
ALTER TABLE Address ADD CONSTRAINT pk_1 PRIMARY KEY (AddressID);
```

Here, a constraint with the ID `pk_1` in the `Address` table determines the column `AddressID` for the primary key.

Primary keys in the DBMS MariaDB are removed with the keyword `DROP PRIMARY KEY`. Here is an example for the table `Address`:

Code

```
ALTER TABLE Address DROP PRIMARY KEY;
```

Foreign keys in the DBMS MariaDB are removed with the keyword `DROP FOREIGN KEY` followed by the constraint ID. Here is an example for the table `Address`:

Code

```
ALTER TABLE Address DROP FOREIGN KEY fk_1;
```

Deleting Tables (DROP)

Complete deletion of a table is very easy with SQL, but has very drastic effects: the contents and structure of the table will be irrevocably removed from the database. The keyword for this is `DROP TABLE`, where only the name of the table has to be specified. Here is an example of the delete command for the `Address` table:

Code

```
DROP TABLE Address;
```

Preservation of Referential Integrity

When formulating SQL statements to modify and delete existing data, the developer must ensure that all technical and business integrity conditions are met. If foreign key relationships have been defined in a database, these must be given special consideration when changing the dataset in order to maintain referential integrity. One possibility is to specify the automatic behavior of the DBMS when updating foreign keys with `ON DELETE` and `ON CASCADE`. When formulating the SQL statements, the developer must take into account how the DBMS is to behave when it is changed or deleted. In addition, there are often functional dependencies in the data model that can be automatically ensured by the DBMS. The development team must be aware of these so that they can avoid possible consistency violations when creating SQL commands to manipulate the dataset.

As a result, there are often a lot of SQL statements that need to be executed “in one piece”, as partial execution would violate consistency requirements. This is supported in relational database systems with the transaction concept described in the following section.

6.3 Transactions

The changes made to the database so far have been simple in that they relate to a single table. However, if, for example, dependencies between different tables were defined by foreign keys, consistency must be ensured by inserting, changing, or deleting data when changing the dataset. This applies in particular to the guarantee of referential integrity.

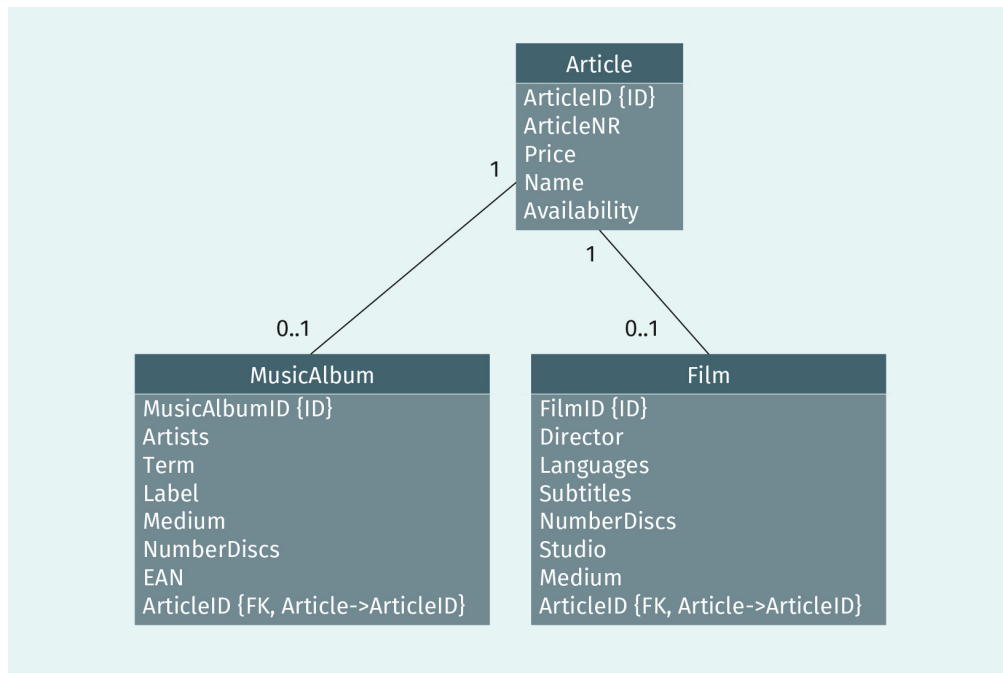
Motivation for Transactions

Dangers to the consistency of a database can be identified at various levels (Unterstein & Matthiessen, 2013):

- power or network failure and abrupt interruption of all operations
- hardware error, e.g., a defect hard disk
- operating system errors, such as unauthorized high consumption of resources like main memory, disk storage, or buffer
- program errors in applications that lead to incorrect database commands
- simultaneous access by multiple users to the same data

In all these possible scenarios, the transaction management of a DBMS must ensure that incompletely executed operations do not change the dataset. The following example scenario illustrates the need for transactions in which articles are to be added to the database of an online shop. The data schema for this is shown in the figure below. This is a joined subclass table mapping of an inheritance hierarchy.

Figure 39: Data Schema for Articles



Source: Jörg Dreikauss, 2022.

An article consists of two data records: one record in the Articles table and one record in the Music Album table or the Film table. If an article is to be listed or delisted in the inventory, two data records must always be changed together:

Code

```
INSERT INTO MusicAlbum [...]  
INSERT INTO Article [...]
```

If, for example, only the first of the two INSERT statements is executed for one of the reasons mentioned above, the database would contain a music album record without the corresponding article record. To prevent this, both INSERT statements are combined in one transaction.

Properties of Transactions

The following criteria, also known as ACID properties, must be fulfilled by a transaction in a DBMS:

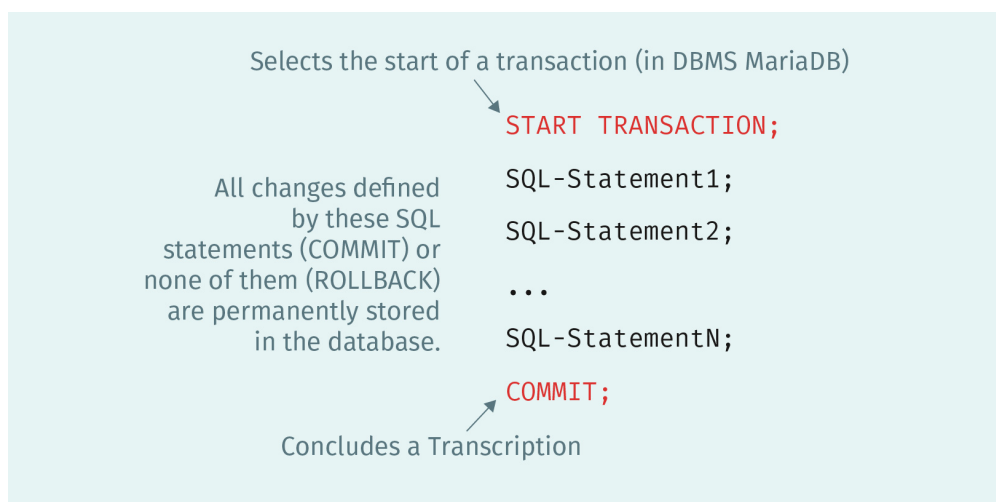
- Atomicity: A transaction is atomic. It is executed only completely or not at all, but in no case only partially.
- Consistency: A transaction maintains the consistency of the database. It transfers the dataset from one consistent state to another consistent state.

- Isolation: Transactions do not influence each other; parallel transactions are implemented as if they had taken place one after the other. This ensures the simultaneous execution of several transactions and allows for multiuser operation.
- Durability: Once a transaction has been successfully completed, the resulting changes are permanently stored in the database.

Control Transactions with SQL

A simple way to combine several SQL statements into one transaction is shown in the figure below. The concrete keyword depends on the selected DBMS.

Figure 40: Schema of a Transaction



Source: Jörg Dreikauss, 2022.

The above example could be implemented as follows:

Code

```
START TRANSACTION;
INSERT INTO MusicAlbum [...]
INSERT INTO Article [...]
COMMIT;
```

A transaction begins with `START TRANSACTION` and is concluded with either the keyword `COMMIT` or the keyword `ROLLBACK`. With `COMMIT`, all changes are persistent; with `ROLLBACK`, all changes are undone. If an error is detected during the processing of a transaction, you can use `ROLLBACK` to restore the status of the dataset to what it was before the start of the transaction.



TIP

As a rule, DBMSs offer an AUTOCOMMIT option. If AUTOCOMMIT is activated, each SQL statement is automatically executed individually as a transaction.

Strategies for Multiuser Operation

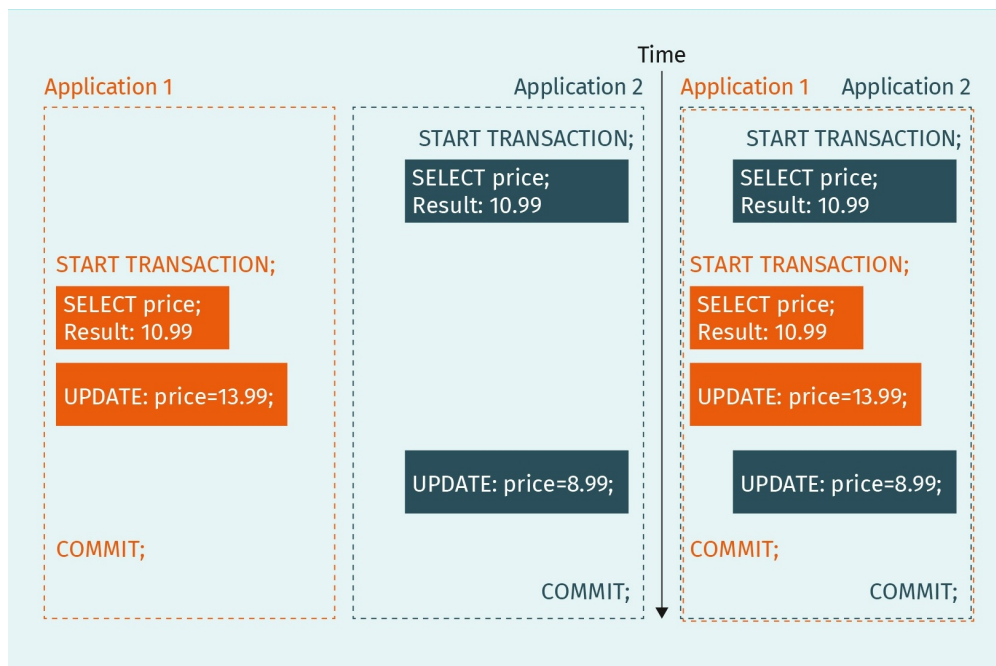
Industrial information systems are usually used not only by one but by several hundred to several thousand users simultaneously—all of whom want to read and change the data stored in the application's database at the same time. The development team faces the following challenge: if all transaction properties are to be guaranteed, the actions of all users can only be executed sequentially, i.e., one after another. In order to ensure that transactions do not influence each other, large parts of the database would have to be blocked for access. However, this often would lead to a system that is very slow from the user's point of view and therefore unusable. If, however, the simultaneous access of several users to the database is to be guaranteed, this **isolation phenomena** cannot be avoided. Isolation phenomena are effects that can occur when transactions that are correct are executed by different users in a database at the same time. Four different isolation phenomena are presented below: Lost Update, Dirty Read (Temporary Update), Non-Repeatable Read, and Phantom Read (see also Elmasri & Navathe, 2016, where a slightly different subdivision is used).

Isolation Phenomena

These are unwanted effects that can occur with the simultaneous execution of multiple transactions on the same data records.

The “lost update” phenomenon describes changes that occur when transactions are executed concurrently. Concurrency of several transactions means that they can be executed simultaneously (in parallel) or sequentially. The figure below illustrates the lost update phenomenon. Two correct transactions are executed from the point of view of the respective applications, as shown on the left side of the figure. However, in actuality, the change made by application 1 is overwritten by application 2 without application 1 becoming aware of these changes.

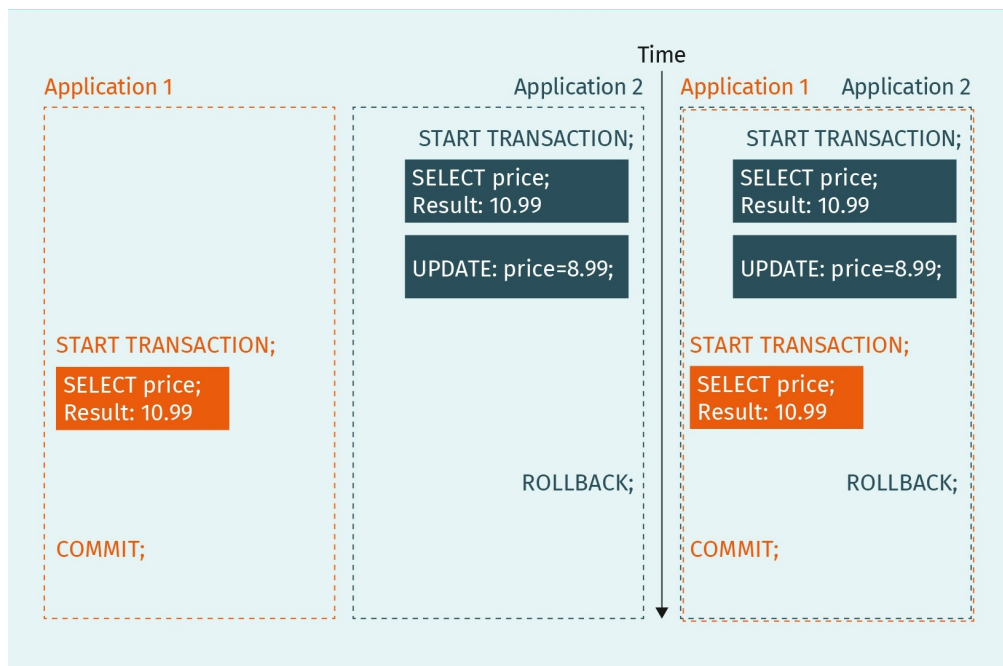
Figure 41: Example of Lost Update



Source: Jörg Dreikauss, 2022.

The “dirty read” phenomenon is illustrated in the figure below. This refers to the effect that occurs when changes are read before they are actually saved in the database by a COMMIT. If, as shown in the example, the changes are reversed with a ROLLBACK, application 1 has a price that is not valid.

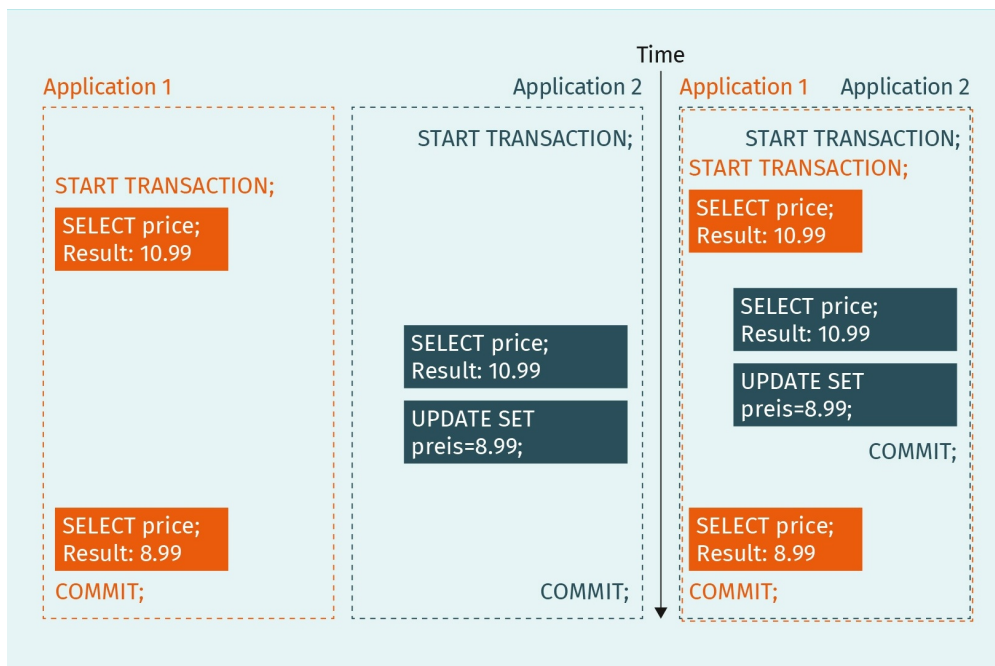
Figure 42: Example of Dirty Read



Source: Jörg Dreikauss, 2022.

The “non-repeatable read” phenomenon occurs when reading the same data several times returns different results. As shown in the example in the figure below, a dataset from application 1 is read several times. The modification of the data by application 2 leads to different results.

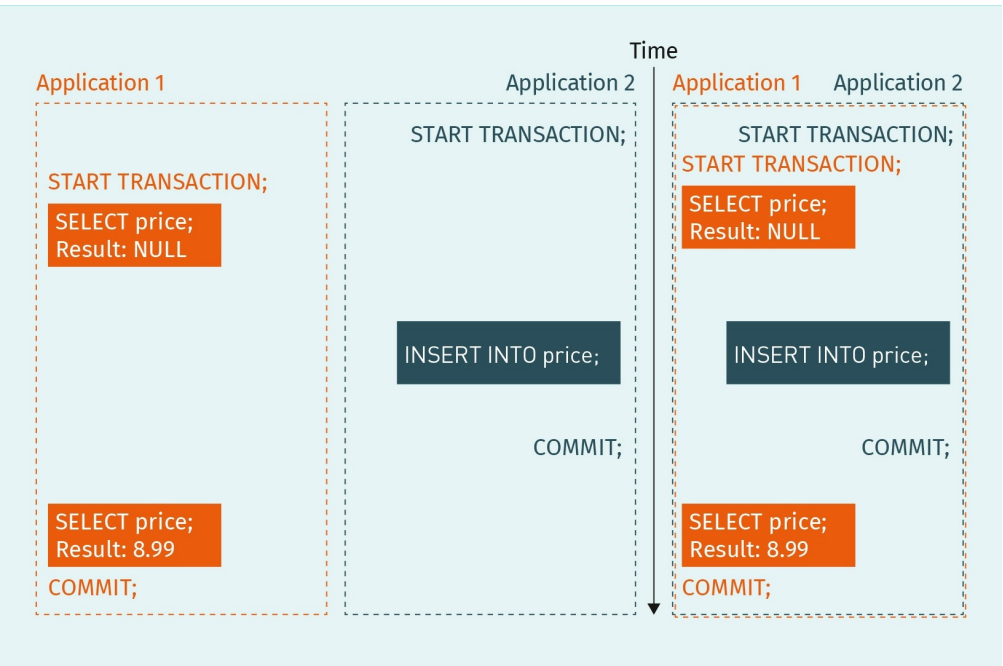
Figure 43: Example of Non-Repeatable Read



Source: Jörg Dreikauss, 2022.

The “phantom read” phenomenon refers to finding different results when reading the same tables, caused by inserting (INSERT) or deleting (DELETE) records from the database. The figure below illustrates this effect, in which no element is included in the result set for the first read access of application 1, but a value is included for the second read access.

Figure 44: Example of Phantom Read



Source: Jörg Dreikauss, 2022.

Isolation level
This describes settings that govern the compromise in a DBMS between data consistency and ease of multiuser operation. The degree of simultaneous execution of transactions is specifically configured.

SQL offers the possibility to define a specific **isolation level** for each transaction, which allows or forbids isolation phenomena. In this way, a compromise can be made between data consistency and ease of multiuser operation when creating transactions. Four isolation levels can be distinguished: Read Uncommitted, Read Committed, Repeatable Read, and Serialize. The next table describes the isolation levels and indicates which isolation phenomena are permitted by them.

Table 56: Isolation Levels of Transactions

Isolation level	Description	Permitted phenomena
Read uncommitted	lowest isolation level at which changes not confirmed by COMMIT can also be read	dirty read, non-repeatable read, phantom read
Read committed	allows reading only data that have already been confirmed by COMMIT, but does not prevent inconsistencies in multiple reads	non-repeatable read, phantom read
Repeatable read	ensures that the same data are available when reading the same dataset more than once ; prevents no deviations by inserting or deleting	phantom read
Serialize	ensures that the same amount of data is always available when reading multiple times within a transaction; prevents all isolation phenomena	none

Source: Jörg Dreikauss, 2022.

Setting transaction isolation levels allows for a controlled violation of the isolation of transactions to enable efficient multiuser operation of databases.



SUMMARY

The SQL statement keyword `INSERT` is used to insert new records into a relational database. An `UPDATE` statement is executed to change data records, and a data record is deleted from a table with `DELETE`.

The schema of a database table is executed with the SQL statement `ALTER TABLE`. The complete deletion of a table is executed with `DROP TABLE`. The referential integrity must be guaranteed for all change operations.

Transactions are defined if multiple SQL statements have to be executed, for example, because a domain-oriented data record is distributed across multiple tables. Different multiuser strategies can be defined depending on the database requirements.

UNIT 7

NOSQL DATABASE SYSTEMS

STUDY GOALS

On completion of this unit, you will have learned ...

- what the term NoSQL means and which properties distinguish NoSQL systems.
- which categories of NoSQL systems exist.
- what the basic idea of data organization in selected categories of NoSQL systems is.

7. NOSQL DATABASE SYSTEMS

Introduction

In addition to the widely used relational databases, there are also database management systems (DBMSs) in which the data are not stored on the basis of the relational data model. This category of databases is called NoSQL (“not only Structured Query Language”). This lesson explains the basic idea behind NoSQL systems and the basic data organization of selected NoSQL systems.

7.1 Motivation and Basic Idea

With the advent of fast-growing, globally distributed web applications, which often have several hundred thousand users and an equally fast-growing amount of data, database systems were needed whose capacity could be increased quickly and without the procurement of expensive, special hardware. Under the term NoSQL, these different, new database systems purposefully address the requirements of fast-growing internet applications, such as high availability, simple scaling, and the processing of large amounts of data. NoSQL is not a standardized term but was coined by the community. A widely accepted meaning is “Not only SQL”, alluding to the fact that NoSQL databases are not to be used (or not only) with the standard DB query language SQL (“NoSQL database, n.d.”).

Meaning of NoSQL

It is not always clear in some cases when a DBMS falls into the NoSQL system category. Some typical NoSQL properties are listed below (see Edlich, 2011). According to Edlich, whether a database belongs to the class of NoSQL databases requires the consideration of some of the following points:

The underlying data model is not relational

Although the relational data model is currently the most widely used data model, there are a number of other data models, such as wide column stores, key value systems, document stores, and graph databases. These are often characterized by a more flexible data schema.

There is consistent focus on distributed and horizontal scalability

Fast-growing user numbers usually also mean a fast-growing amount of data. In addition, data-intensive objects, such as films, music, or pictures, are often the items that need to be managed. The operation of a DBMS must therefore be able to react very quickly to a rapidly increasing demand for database capacity. With horizontal scalability, it is possible to change capacities by adding or removing another DBMS while the application is running. This creates a distributed database system in which all the data are stored on several

individual DBMSs. By distributing the data to several systems, the load of a single DBMS is also distributed to many systems. Where the internal capacities of the special hardware of a DBMS are adapted (scale-up principle) during the operation of a classic DBMS, the capacity of NoSQL systems is regulated by connecting further systems (scale-out principle).

Due to the distributed architecture, the system supports simple data replication.

Replicating the dataset of a DBMS is an important property for enabling horizontal scaling. Therefore, many NoSQL DBMSs offer a simple way to replicate the data stored in them, i.e., to store it multiple times. In some cases, only one database command is required.

The system is schema-free or has only weak schema restrictions

Since the internet boom in the 1990s, applications have been developed agilely. The functionality of many web applications evolves with the needs of a rapidly growing number of users and the business model of the operator. However, it must be permanently available to all users. The data schema usually changes with the scope of functions of an application. NoSQL databases shift the responsibility of schema definition and compliance with the limitations specified by the data schema into the logic of the application. Therefore, when the data schema is modified, the entire database does not have to be locked and restructured. With NoSQL databases, the database is versioned when schema changes are made; the conversion of the data is triggered as a background process. The system remains available, but there is a timespan during which the DBMS delivers old data that have not yet been converted in the new schema when a request is made. However, these consistency limitations are acceptable for many (entertainment) applications. In safety-critical systems or systems where large amounts of money are managed, on the other hand, reductions in data consistency requirements are unacceptable.

The system is usually based on a different consistency model: Eventually Consistent and BASE, but not ACID

For most relational databases, the ACID consistency model applies, which places very high demands on (among other things) the consistency of transactions. Many Web 2.0 and entertainment applications, in contrast to systems used by financial service providers such as banks and insurance companies, do not have particularly high consistency requirements. For example, messages or status information in social networks can be inconsistent for a short period of time without this inconsistency being noticed or harm being done. If, however, the system fails for only a brief moment, this can have a significant impact on user acceptance. Therefore, a BASE consistency model (Basically Available, Soft State, Eventual consistency), in which the criterion of availability is weighted higher than the criterion of data consistency, is sufficient for many applications. “Eventually consistent” in this context means that the dataset can be inconsistent for a short time, but after a certain period of time without further changes to the dataset, it will again reach a consistent state. Some but not all NoSQL systems support the BASE consistency model. The concrete requirements for availability and consistency determine the selection of a suitable DBMS.

The system offers a simple API

The technical interface to relational databases is the Structured Query Language, which has been mature for decades. It can be used to formulate very complex queries to the database. NoSQL systems without a relational data model do not offer an SQL interface. Here, requests to the DBMS are made via defined technical interfaces (an application programming interface, or API for short). These interfaces are usually kept very simple and do not (yet) support complex queries.

The NoSQL system is open source

Many original NoSQL systems are open source systems or originated from database systems originally developed as open source.

7.2 Selected Groups of NoSQL Systems

Key-Value Systems

Core principle of data organization

The principle of data organization of key-value systems is very simple: A value is stored for a key. A value can itself be a data structure, such as lists or quantities. In the Java programming language, the collection data structure map also implements a key value memory structure.

Figure 45: Data Organization in Key Value Systems

Key	Value, if necessary complex values
1	[price:10.99], [name:“On the Road to Nothingness“]
2	[price:13.99], [name:“Behind Us“]

Source: Jörg Dreikauss, 2022.

The simple data model of key value systems allows large and constantly growing amounts of data to be organized quickly and efficiently, as there are no schematic dependencies between individual datasets. However, the possibility of formulating queries on a key value system is given by the API of the manufacturer and therefore the formulation of complex queries is often difficult or not possible at all.

Typical representations

- Redis
- Riak

Wide Column Stores

Core principle of data organization

In wide column store databases, data are physically stored in tables, but unlike in relational databases, they are stored column by column rather than row by row. The data of a relational database are stored line by line:

Code

```
1, 10.99, „On the Road to Nothingness”,  
2, 13.99, “Behind Us“
```

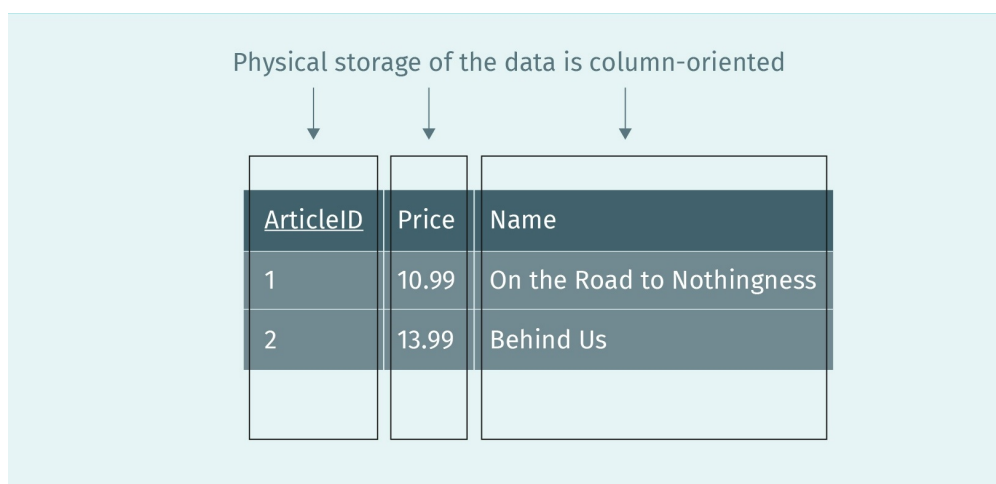
Data are stored in columns in column-oriented systems:

Code

```
1, 2, 10.99, 13.99, „On the road to nothingness”, “Behind us”
```

In this way, it is possible to read and change entire columns quickly, since all the data in a column are stored in memory one after another. In addition, columns can be grouped into families.

Figure 46: Data Organization for Wide Column Stores



Source: Jörg Dreikauss, 2022.

The column orientation shown in the figure above provides very good support for analytical functions such as those used in data-warehouse settings. Evaluation functions that refer to entire columns, such as aggregate functions, are very well supported by this type

of memory organization. Writing and reading complete datasets, on the other hand, is more time-consuming, since to completely read all data for a data set, the appropriate value must be sought out in different column stores.

Typical representations

The following DBMSs are based on column orientation but extend this concept to include key value structures:

- HBase
- Cassandra
- Amazon SimpleDB

Document Stores

Core principle of data organization

Document stores, unlike what the name might suggest, do not store documents that a user has written. The term “document” describes, as illustrated in the figure below, structured text such as XML or JSON that is used for the exchange of messages in technical system interfaces. Often, only the format of IDs is specified by the DBMS, but not a fixed data schema.

Figure 47: Schema of UPDATE

```
{
  "ID": "1" {
    "price" : "10.99"
    "name" : "On the Road to Nothingness"
  }
  "ID": "2" {
    "price" : "13.99"
    "name" : "Behind Us"
  }
}
```

Source: Jörg Dreikauss, 2022.

The simultaneous advantage and disadvantage of document stores is the complete shifting of responsibility for compliance with the required data schema to the logic of the application. On the one hand, document-based databases are quickly modifiable and any type of data can be immediately stored in them, but on the other hand, all consistency requirements must also be ensured by the application logic.

Typical representations

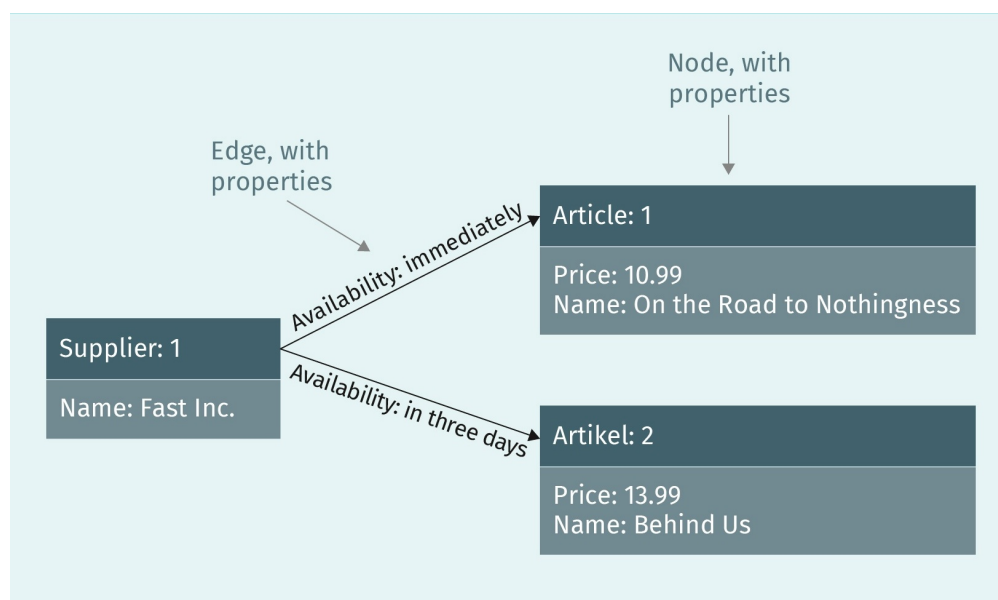
- CouchDB
- MongoDB

Graph Databases

Core principle of data organization

Graph databases use graphs as the basic data structure for organizing the information stored in them. A graph consists of the nodes and edges for each of the specific properties that can be stored. Information is structured like a network in many fields of expertise and knowledge. Examples of this are relationship graphs in social networks, or rail, road, and air transportation networks. With graph databases, such relationships between elements can be typed and stored in their natural form. The figure below illustrates the elements in graph databases using the node types “supplier” and “article,” connected by an edge.

Figure 48: Data Organization in Graph Databases



Source: Jörg Dreikauss, 2022.

With a graph structure in the background, a traversing of the data stock via the relationships between the nodes can take place much faster than in the simulation of a graph structure in relational databases.

Typical representatives

- AllegroGraph
- Neo4y

**SUMMARY**

Under the term NoSQL (“Not only SQL”), different database systems purposefully address the requirements of fast-growing internet applications, such as high availability, simple scaling, and the processing of large amounts of data. Selected categories of NoSQL systems are key value systems, wide column stores, document stores, and graph databases.

BACKMATTER

LIST OF REFERENCES

- Databases Online Lexicon. (2012). *Kardinalität von beziehungen* [Cardinality of relationships]. http://wikis.gm.fh-koeln.de/wiki_db/Datenbanken/Beziehung,Kardinalitaet-vo-n-Beziehungen
- Department of Mathematics and Computer Science. (2010). *Mondial Database*. <https://www.dbis.informatik.uni-goettingen.de/Mondial/OtherDBMSs/mondial-schema-mysql.sql>
- Edlich, S., Friedland, A., Hampe, J., & Brauer, B. (2011). *NoSQL. Einstieg in die welt nichtrelationaler web 2.0 datenbanken* [Entry into the world of non-relational web 2.0 databases]. Hanser Verlag.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems*. Pearson Education Limited.
- Elmasri, R., & Navathe, S. (2017). *Fundamentals of database systems* (7th ed., global ed.). Pearson.
- Foster, E., & Godbole, S. (2016). *Database systems: A pragmatic approach*. (2nd ed.). Apress.
- Harkins, S. (2011). *Surrogat- oder natürlicher schlüssel: So trifft man die richtige Entscheidung* [Surrogate or natural key: This is how you make the right decision]. ZDNet. <http://www.zdnet.de/41553179/surrogat-oder-natuerlicher-schluessel-so-trifft-man-die-richtige-entscheidung>
- Mondial Database. (n.d.). *Databases and information systems group: Practice database*. Institute for Informatics, Mondial. <http://www.dbis.informatik.uni-goettingen.de/Mondial>
- MySQL. (2019). *MySQL 5.7 reference manual: Data types*. <https://dev.mysql.com/doc/refman/5.7/en/data-types.html>
- NoSQL database. (n.d.). *List of NoSQL databases management systems*. <http://nosql-database.org>
- Schubert, M. (2007). *Datenbanken: Theorie, entwurf und programmierung relationaler datenbanken* [Databases: Theory, design and programming of relational databases]. (2nd ed.). Teubner Verlag.
- Steiner, R. (2014). *Grundkurs relationale datenbanken* [Basic course relational databases]. (8th ed.). Springer Vieweg.

- Steiner, R. (2021). *Grundkurs Relationale Datenbanken Einführung in die Praxis der Datenbankentwicklung für Ausbildung, Studium und IT-Beruf* [Basic course relational databases introduction to the practice of database development for apprenticeship, academic studies and IT career]. Springer Vieweg.
- Unterstein, M., & Matthiessen, G. (2012). *Relationale datenbanken und SQL in theorie und praxis* [Relational databases and SQL in theory and practice]. (5th ed.). Springer Vieweg.
- Unterstein, M., & Matthiessen, G. (2013). *Anwendungsentwicklung mit datenbanken* [Application development with databases]. (5th ed.). Springer Vieweg.

LIST OF TABLES AND FIGURES

Figure 1: Database, Database Management System, and Database System	13
Figure 2: Customer Relation Example	14
Table 1: Customer Data of an Online Shop	15
Table 2: Table with ADDRESS Relation	15
Figure 3: Illustration of the 1:1 Relationship Type	16
Figure 4: Table CUSTOMER with 1:1 Relationship to ADDRESS Relation	17
Figure 5: Illustration of the 1:N Relationship Type	17
Figure 6: ADDRESS table with 1:N Relationship to CUSTOMER Relation	18
Figure 7: Illustration of the N:M Relationship Type	19
Figure 8: Relationship Table for N:M Relationships	20
Table 3: CUSTOMER Relation Table	21
Table 4: ADDRESS Table Including Sample Data	21
Table 5: BILLINGADDRESS Table Including Sample Data	22
Table 6: CUSTOMER Table	26
Table 7: ADDRESS Table	26
Table 8: DELIVERYADDRESS Table	26
Table 9: CUSTOMER Table before Deletion	28
Table 10: ADDRESS Table before Deletion	28
Table 11: Relationship Table MAINRESIDENCE Before Deletion	28
Table 12: Relationship Table DELIVERYADDRESS Before Deletion	29

Table 13: Relationship Table INVOICEADDRESS Before Deletion	29
Table 14: Extract from the MONDIAL Database	34
Figure 9: Example of a Simple SELECT Request	36
Table 15: Result (Excerpt)	36
Table 16: Elements of a SELECT Command	37
Table 17: Result (Excerpt)	38
Table 18: Examples of Logical Operators	40
Table 19: Comparison Operators	41
Table 20: Date Functions	42
Table 21: String Functions	43
Table 22: Sequence Indicator	44
Table 23: Important Aggregate Functions in SQL	47
Figure 10: Example of Subqueries	49
Figure 11: Schema of Subqueries SELECT	49
Figure 12: Example of Subquery with Multiline Return	50
Table 24: Typical Comparison Operators for Quantities	51
Figure 13: Conception and Modeling of Relational Databases	55
Figure 14: Entities and Attributes in Martin Notation	56
Figure 15: Entities and Attributes in UML	56
Table 25: Types of 1:1 Relationships Including Optionality	57
Table 26: Types of 1:N Relationships Including Optionality	58
Table 27: Types of N:M Relationships Including Optionality	59
Figure 16: Dependencies Between Normal Forms	60

Table 28: Table for Article Delivery Example	60
Table 29: First Normal Form with Compound Primary Key	61
Table 30: First Normal Form with Transformed Table Including Simple Primary Key.	61
Figure 17: Dependencies in First Normal Form	63
Table 31: ArticleDelivery, Second Normal Form	63
Table 32: Customer, Second Normal Form	64
Table 33: Article, Second Normal Form	64
Figure 18: Dependencies in the Second Normal Form	65
Figure 19: Third Normal Form with Compound Primary Key	66
Figure 20: Third Normal Form with a Simple Primary Key	67
Figure 21: Activities in the Design Process for Databases	71
Figure 22: Example of a Conceptual Data Model in the Form of a UML Class Diagram ...	74
Table 34: Illustration of 1:1 Relationships	75
Table 35: Illustration of C:C Relationships	77
Table 36: Illustration of 1:N relationships	78
Table 37: Mapping of N:M Relationships	80
Table 38: Mapping of Recursive Relationships	81
Figure 23: Scenario for Inheritance Example	82
Figure 24: Physical Data Model for Single-Table Strategy	83
Table 39: Advantages and Disadvantages of the Single-Table Strategy	83
Figure 25: Example for Joined Subclass Table Strategy	84
Table 40: Advantages and Disadvantages of the Joined Subclass Table Strategy	84
Figure 26: Example of Table Per Class Strategy	85

Table 41: Advantages and Disadvantages of the Table per Class Strategy	85
Table 42: Text Types in SQL, Selection	86
Table 43: Number Types in SQL, Selection	87
Table 44: Date Types in SQL, Selection	87
Table 45: Additional Data Types in SQL, Selection	88
Table 46: Elements for Column Definition	90
Table 47: Address	91
Table 48: Address	92
Figure 27: Example for Foreign Keys	93
Table 49: Preservation of Referential Integrity	94
Table 50: Article	98
Table 51: Film	99
Figure 28: SQL Example of a Compound Set	101
Figure 29: Example Database for the Illustration of Collaborations Database: JOINTTEST	101
Table 52: Important Compound Types of Relational Databases	102
Table 53: Internal Bond with WHERE	104
Figure 30: Schema for Set Operations	105
Figure 31: Example for the Set Operation UNION	105
Figure 32: Example for CREATE VIEW	107
Figure 33: Reuse of a VIEW	107
Figure 34: Schema of an INSERT Statement	110
Table 54: Elements in the INSERT Statement	110
Figure 35: Schema of an INSERT Statement	111

Figure 36: Examples of Automatic Type Conversions	113
Figure 37: Schema of UPDATE	116
Table 55: Elements of ALTER TABLE	117
Figure 38: Example of a Constraint: Foreign Key	118
Figure 39: Data Schema for Articles	120
Figure 40: Schema of a Transaction	121
Figure 41: Example of Lost Update	123
Figure 42: Example of Dirty Read	124
Figure 43: Example of Non-Repeatable Read	125
Figure 44: Example of Phantom Read	126
Table 56: Isolation Levels of Transactions	126
Figure 45: Data Organization in Key Value Systems	132
Figure 46: Data Organization for Wide Column Stores	133
Figure 47: Schema of UPDATE	134
Figure 48: Data Organization in Graph Databases	135



IU Internationale Hochschule GmbH
IU International University of Applied Sciences
Juri-Gagarin-Ring 152
D-99084 Erfurt



Mailing Address
Albert-Proeller-Straße 15-19
D-86675 Buchdorf



media@iu.org
www.iu.org



Help & Contacts (FAQ)
On myCampus you can always find answers
to questions concerning your studies.